

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

HỒ VĂN VIỆT HOÀNG
NGUYỄN PHÚC HIẾU

KHÓA LUẬN TỐT NGHIỆP
NGHIÊN CỨU, THIẾT KẾ VÀ HIỆN THỰC IP NHẬN
DIỆN TỪ KHÓA TỤC TỬ TRONG LỚP HỌC

RESEARCH, DESIGN AND IMPLEMENTATION OF A PROFANITY
KEYWORD SPOTTING IP FOR CLASSROOM ENVIRONMENTS

CỬ NHÂN NGÀNH KỸ THUẬT MÁY TÍNH

TP. HỒ CHÍ MINH, 2026

**ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH**

**HỒ VĂN VIỆT HOÀNG - 22520461
NGUYỄN PHÚC HIẾU - 22520441**

**KHÓA LUẬN TỐT NGHIỆP
NGHIÊN CỨU, THIẾT KẾ VÀ HIỆN THỰC IP NHẬN
DIỆN TỪ KHÓA TỤC TỬ TRONG LỚP HỌC**

**RESEARCH, DESIGN AND IMPLEMENTATION OF A PROFANITY
KEYWORD SPOTTING IP FOR CLASSROOM ENVIRONMENTS**

CỬ NHÂN NGÀNH KỸ THUẬT MÁY TÍNH

**GIẢNG VIÊN HƯỚNG DẪN
THS. TRƯƠNG VĂN CƯỜNG**

TP. HỒ CHÍ MINH, 2026

THÔNG TIN HỘI ĐỒNG CHẤM KHÓA LUẬN TỐT NGHIỆP

Hội đồng chấm khóa luận tốt nghiệp, thành lập theo Quyết định số 710/QĐ-ĐHCNTT ngày 19 tháng 06 năm 2026 của Hiệu trưởng Trường Đại học Công nghệ Thông tin.

LỜI CẢM ƠN

Đầu tiên, chúng em xin gửi lời cảm ơn chân thành đến Ban Giám hiệu và quý thầy cô trường Đại học Công Nghệ Thông Tin (ĐHQG-HCM). Những kiến thức quý báu và sự chỉ bảo tận tình của quý thầy cô trong suốt quá trình học tập tại trường đã trang bị cho chúng em nền tảng vững chắc về chuyên môn cũng như kỹ năng mềm, là hành trang vô giá cho chúng em trong công việc và cuộc sống sau này.

Chúng em cũng xin gửi lời tri ân đến các thầy cô khoa Kỹ Thuật Máy Tính đã luôn tạo mọi điều kiện thuận lợi, môi trường học tập tốt nhất để chúng em có thể trau dồi và phát triển bản thân.

Đặc biệt, để hoàn thành khóa luận tốt nghiệp này, chúng em xin gửi lời cảm ơn sâu sắc nhất tới Thầy Trương Văn Cương. Thầy đã luôn tận tình hướng dẫn, định hướng nghiên cứu và hỗ trợ chúng em rất nhiều về mặt thiết bị cũng như kiến thức chuyên môn trong suốt thời gian thực hiện đề tài. Sự nhiệt huyết và chỉ bảo của Thầy là động lực lớn giúp chúng em vượt qua những khó khăn để hoàn thành đồ án một cách tốt nhất.

Đồng thời, chúng em cũng xin cảm ơn bạn bè đã luôn bên cạnh hỗ trợ, chia sẻ kinh nghiệm và giúp đỡ chúng em trong quá trình thực hiện.

Cuối cùng, chúng con xin gửi lời biết ơn sâu sắc đến gia đình và ba mẹ đã luôn quan tâm, hỗ trợ và ủng hộ chúng con vô điều kiện trên con đường học vấn.

Mặc dù đã rất cố gắng, nhưng khóa luận khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự đóng góp ý kiến của quý thầy cô để đề tài được hoàn thiện hơn. Chúng em xin chân thành cảm ơn!

TP. Hồ Chí Minh, tháng 7 năm 2026

Nhóm sinh viên thực hiện

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU.....	3
1.1. Đặt vấn đề.....	3
1.2. Mục tiêu đề tài.....	4
1.3. Nội dung thực hiện đề tài	4
CHƯƠNG 2. TÌM HIỂU TỔNG QUAN	8
2.1. Cơ sở lý thuyết	8
2.1.1. Cơ chế sinh thanh và tính chất vật lý của tín hiệu tiếng nói	8
2.1.2. Đặc trưng phổ năng lượng và âm vị học.....	8
2.1.3. Yêu cầu kỹ thuật khi triển khai trên FPGA	9
2.2. Quy trình trích xuất đặc trưng MFCC trong hệ thống	12
2.2.1. Tiền nhân (Pre-emphasis)	13
2.2.2. Chia khung (Framing).....	15
2.2.3. Nhân cửa sổ (Windowing)	16
2.2.4. Biến đổi Fourier nhanh và phổ công suất	18
2.2.5. Bộ lọc Mel	20
2.2.6. Logarit năng lượng	21
2.2.7. DCT và vector MFCC	22
2.2.8. Bộ đệm đặc trưng (Feature RAM Buffer)	23
2.3. Khối phân loại mạng nơ-ron, lượng tử hóa và huấn luyện MLP	25
2.3.1. Cấu trúc mô hình MLP 1274-32-20.....	27
2.3.2. Lượng tử hóa mô hình MLP và đồng bộ thang đo	29
2.3.3. Xuất tham số và yêu cầu nhất quán Python-fixed-point-RTL.....	31
2.4. Công trình nghiên cứu liên quan	32
2.4.1. Nghiên cứu KWS quốc tế	32
2.4.2. Các nghiên cứu tiếng Việt trên phần cứng	33
2.4.3. Khoảng trống nghiên cứu và định vị đề tài.....	34
CHƯƠNG 3. THIẾT KẾ VÀ HIỆN THỰC	36
3.1. Mô tả kiến trúc tổng quan	36
3.2. Thiết kế chi tiết các khối chức năng.....	38
3.2.1. Khối Tiền nhân (Pre-emphasis)	41
3.2.2. Khối Chia khung (Framing Control)	43
3.2.3. Khối Cửa sổ hóa (Windowing).....	44
3.2.4. Khối Biến đổi Fourier và Phổ năng lượng (FFT & Power Spectrum) ...	46
3.2.5. Khối Bộ lọc Mel (Mel Filterbank).....	47
3.2.6. Khối Logarit năng lượng (Log-Mel)	49
3.2.7. Khối Biến đổi Cosine rời rạc (DCT/MFCC)	50
3.2.8. Khối Bộ đệm đặc trưng (Feature RAM Buffer)	51
3.2.9. Khối Phân loại mạng nơ-ron (MLP Core)	53
3.3. Kiến trúc SoC và vai trò PS-PL	56
3.3.1. Nền tảng KV260 và thành phần PS/PL	56
3.3.2. Luồng nạp, điều khiển và trao đổi dữ liệu	56
3.4. Kiến trúc phần cứng và phần mềm.....	57
3.4.1. Kiến trúc phần cứng.....	57
3.4.2. Kiến trúc phần mềm.....	59
3.5. Tích hợp top-level và giao tiếp PS-PL	60
3.5.1. Giao tiếp top-level	60
3.5.2. Ảnh xạ thành ghi phần mềm-phần cứng.....	60

3.5.3. Trình tự xử lý phần mềm	61
CHƯƠNG 4. MÔ HÌNH KIỂM CHỨNG.....	63
4.1. Mô hình kiểm chứng tổng quan	63
4.2. Mô hình tham chiếu phần mềm và dữ liệu kiểm thử	64
4.3. Mục tiêu, phạm vi và tiêu chí đánh giá	64
4.4. Cơ chế đối sánh dữ liệu Python-RTL và chỉ số sai số	67
4.5. Kích bản kiểm thử, coverage và assertion	69
4.6. Kế hoạch kiểm chứng và báo cáo kết quả kiểm chứng	72
CHƯƠNG 5. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ	74
5.1. Môi trường thực nghiệm và dữ liệu đánh giá.....	74
5.2. Kết quả kiểm chứng mức phần mềm	74
5.3. Kết quả kiểm chứng RTL theo từng module.....	80
5.3.1. Kiểm chứng khối Pre-emphasis.....	81
5.3.2. Kiểm chứng khối Framing.....	82
5.3.3. Kiểm chứng khối Windowing	83
5.3.4. Kiểm chứng khối FFT/Power Spectrum.....	84
5.3.5. Kiểm chứng khối Mel Filterbank	85
5.3.6. Kiểm chứng khối Log-Mel	85
5.3.7. Kiểm chứng khối DCT/MFCC	86
5.3.8. Kiểm chứng Feature RAM Buffer.....	86
5.3.9. Kiểm chứng MLP Core.....	87
5.3.10. Kiểm chứng top-level RTL full-pipeline.....	88
5.4. Kết quả triển khai trên phần cứng FPGA KV260	90
5.5. Tài nguyên, timing và công suất	95
5.5.1. Tài nguyên phần cứng sau implementation	96
5.5.2. Kết quả timing sau implementation	97
5.5.3. Phân tích công suất tiêu thụ	98
5.6. So sánh với các công trình liên quan.....	101
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	103
6.1. Kết luận chung	103
6.2. Hạn chế của đề tài	104
6.3. Hướng phát triển	105
6.3.1. Mở rộng và chuẩn hóa dữ liệu	105
6.3.2. Mở rộng số lượng từ khóa và cập nhật hệ thống	105
6.3.3. Nâng cấp mô hình và tối ưu kiến trúc phần cứng.....	106
6.3.4. Tích hợp âm thanh thời gian thực và hoàn thiện đánh giá	106
TÀI LIỆU THAM KHẢO.....	107
PHỤ LỤC A	110

DANH MỤC HÌNH

Hình 1.1. Quy trình nghiên cứu và hiện thực hệ thống	6
Hình 2.1. Dạng sóng tín hiệu tiếng nói trước và sau bước tiền nhấn trên Python ...	14
Hình 2.2. Quá trình chia frame 512 mẫu và cơ chế chồng lấn hop 160 mẫu	16
Hình 2.3. Hệ số Hamming và dạng sóng frame trước, sau bước cửa sổ hóa	18
Hình 2.4. Phổ biên độ FFT và phổ công suất trung bình 257 bin sau FFT	19
Hình 2.5. Bộ lọc tam giác Mel và vector năng lượng Mel	20
Hình 2.6. Tác động của phép logarit lên dải động của vector năng lượng Mel	22
Hình 2.7. Heatmap MFCC kích thước 13×98 của một mẫu từ khóa	23
Hình 2.8. Vector đặc trưng MFCC 1274 phần tử	24
Hình 2.9. Kiến trúc mô hình MLP 1274-32-20 và luồng dữ liệu đặc trưng	29
Hình 3.1. Kiến trúc tổng quan của hệ thống nhận diện từ khóa	36
Hình 3.2. Kiến trúc top-level KWS IP Core	37
Hình 3.3. Sơ đồ kết nối pipeline MFCC trong thiết kế RTL	40
Hình 3.4. Sơ đồ khối module pre_emphasis	42
Hình 3.5. Sơ đồ khối module framing	44
Hình 3.6. Sơ đồ khối module windowing	45
Hình 3.7. Sơ đồ khối module fft512_power_axis	47
Hình 3.8. Sơ đồ khối module mel_filterbank	48
Hình 3.9. Sơ đồ khối module log_block	49
Hình 3.10. Sơ đồ khối module DCT/MFCC	51
Hình 3.11. Kiến trúc bộ đệm đặc trưng theo block 98 frame	52
Hình 3.12. Datapath xử lý của MLP Core	53
Hình 3.13. FSM điều khiển MLP Core	55
Hình 3.14. Kiến trúc SoC PS-PL khi tích hợp KWS IP	56
Hình 3.15. Kiến trúc phần cứng của KWS IP trong PL	58
Hình 3.16. Luồng chương trình C kiểm thử trên PS/Linux	59
Hình 3.17. Sơ đồ ánh xạ giao tiếp AXI4-Lite giữa CPU và KWS IP	61
Hình 3.18. Vivado Block Design và kết nối PS-PL của hệ thống	62
Hình 4.1. Mô hình kiểm chứng phần mềm, RTL và FPGA	63
Hình 4.2. Đồ thị so sánh giá trị hệ số MFCC giữa Python và RTL	68
Hình 5.1. Log huấn luyện và đánh giá Python floating-point trên tập validation	75
Hình 5.2. Log huấn luyện fixed-point trên tập validation	75
Hình 5.3. Đường cong train/validation loss và accuracy Python floating-point	76
Hình 5.4. Ma trận nhầm lẫn mô hình Python floating-point validation 300 mẫu ...	76
Hình 5.6. Log batch validation 300 mẫu ở mức RTL pre-synthesis simulation	77
Hình 5.7. Log batch toàn bộ 2.000 mẫu ở mức RTL pre-synthesis simulation	78
Hình 5.8. Log kiểm chứng top-level RTL pre-synthesis cho một testcase đại diện	78
Hình 5.9. Log post-synthesis functional simulation	79
Hình 5.10. Log batch validation 300 mẫu trên phần cứng FPGA KV260	79
Hình 5.11. Waveform kiểm chứng khối Pre-emphasis	82
Hình 5.12. Waveform kiểm chứng khối Framing	83

Hình 5.13. Waveform kiểm chứng khối Windowing	83
Hình 5.14. Waveform kiểm chứng khối FFT/Power Spectrum	84
Hình 5.15. Waveform kiểm chứng khối Mel Filterbank	85
Hình 5.16. Waveform kiểm chứng khối Log-Mel.....	85
Hình 5.17. Waveform kiểm chứng khối DCT/MFCC.....	86
Hình 5.18. Waveform kiểm chứng Feature RAM Buffer 98 frame	87
Hình 5.19. Waveform kiểm chứng MLP Core cấu hình 1274-32-20.....	88
Hình 5.20. Waveform kiểm chứng top-level cho một testcase từ khóa đại diện	88
Hình 5.21. Setup thực nghiệm trên phần cứng FPGA KV260	90
Hình 5.22. Log nạp bitstream trên KV260	91
Hình 5.23. Log kiểm thử một file PCM trên KV260	91
Hình 5.24. So sánh giá trị logit mô hình Python và RTL của một mẫu đại diện	94
Hình 5.25. Thống kê tài nguyên phần cứng sau implementation.....	96
Hình 5.26. Timing summary sau implementation	97
Hình 5.27. Tổng công suất on-chip ước lượng bởi Vivado.....	98
Hình 5.28. Phân bố công suất động và tĩnh của thiết kế	98

DANH MỤC BẢNG

Bảng 1.1. Danh sách 20 từ khóa/nhãn tục tĩu tiếng Việt sử dụng trong dataset	4
Bảng 1.2. Yêu cầu và thông số thiết kế	5
Bảng 2.1. Độ rộng dữ liệu và quy ước fixed-point của thiết kế	10
Bảng 2.2. Cấu trúc và số lượng tham số của MLP	29
Bảng 2.3. Quy ước lượng tử hóa và xuất tham số MLP	30
Bảng 2.4. Một số công trình KWS quốc tế liên quan	32
Bảng 2.5. Một số công trình nhận dạng tiếng Việt trên phần cứng	34
Bảng 3.1. Thông số tổng quát của hệ thống	38
Bảng 3.2. Danh sách module RTL chính	38
Bảng 3.3. Chu kỳ clock và độ trễ của các khối RTL	40
Bảng 3.4. Giao diện tín hiệu module pre_emphasis	42
Bảng 3.5. Giao diện tín hiệu module framing	44
Bảng 3.6. Giao diện tín hiệu module windowing	45
Bảng 3.7. Giao diện tín hiệu module fft512_power_axis	47
Bảng 3.8. Giao diện tín hiệu module mel_filterbank	48
Bảng 3.9. Giao diện tín hiệu module log_block	50
Bảng 3.10. Giao diện tín hiệu module MFCC	51
Bảng 3.11. Giao diện tín hiệu module mfcc_feature_buffer	53
Bảng 3.12. Giao diện tín hiệu module mlp_core	55
Bảng 3.13. Vai trò các thành phần của nền tảng KV260 trong đề tài	57
Bảng 3.14. Giao tiếp top-level của KWS IP	60
Bảng 3.15. Ánh xạ thanh ghi phần mềm-phần cứng của KWS IP	60
Bảng 4.1. Kế hoạch kiểm chứng	66
Bảng 5.1. Tổng hợp kết quả kiểm chứng RTL theo từng khối	80
Bảng 5.2. Tổng hợp kết quả đánh giá theo các mức thực nghiệm	92
Bảng 5.3. Thống kê tài nguyên phần cứng sử dụng (Post-Implementation)	96
Bảng 5.4. Tóm tắt kết quả hậu implementation của thiết kế	100
Bảng 5.5. So sánh định lượng với các công trình liên quan	101

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tên đầy đủ
AXI	Advanced eXtensible Interface
BRAM	Block RAM
CSV	Comma-Separated Values
DCT	Discrete Cosine Transform
DSP	Digital Signal Processing / DSP Slice
DUT	Design Under Test
FFT	Fast Fourier Transform
Fmax	Maximum Operating Frequency
FPGA	Field-Programmable Gate Array
FPU	Floating-Point Unit
FSM	Finite State Machine
HEX	Hexadecimal file format
IP	Intellectual Property core
KWS	Keyword Spotting
LOD	Leading-One Detector
LUT	Look-Up Table
MAC	Multiply-Accumulate
MAE	Mean Absolute Error
MFCC	Mel-Frequency Cepstral Coefficients
MLP	Multi-Layer Perceptron
PCM	Pulse-Code Modulation
PL	Programmable Logic
PS	Processing System
RMSE	Root Mean Square Error
RTL	Register Transfer Level
SVA	SystemVerilog Assertion
TNS	Total Negative Slack
UIO	Userspace I/O
WHS	Worst Hold Slack
WNS	Worst Negative Slack
WPWS	Worst Pulse Width Slack

TÓM TẮT KHÓA LUẬN

Khóa luận trình bày quá trình nghiên cứu, thiết kế và kiểm chứng một IP nhận diện từ khóa tục tĩu tiếng Việt trong môi trường lớp học. Bài toán được xây dựng dưới dạng Keyword Spotting, với mỗi đoạn âm thanh đã chuẩn hóa, thiết kế trả về mã `result_id` tương ứng với một trong 20 từ khóa/nhãn đã định nghĩa. Trên mô phỏng RTL, thiết kế đạt 295/300 mẫu đúng trên tập validation và 1.996/2.000 mẫu đúng trên toàn bộ dataset, không ghi nhận timeout hoặc lỗi đọc dữ liệu. Khi triển khai trên KV260, thiết kế đạt 294/300 trên batch validation và 1.995/2.000 trên toàn bộ dataset.

Về phương pháp nghiên cứu, dữ liệu WAV được chuẩn hóa về PCM signed 16-bit, mono, tần số lấy mẫu 16 kHz. Python được sử dụng để xây dựng mô hình tham chiếu, trích xuất MFCC, huấn luyện MLP và đánh giá mô hình fixed-point trước khi xuất hệ số sang thiết kế phần cứng. Quy trình này tạo cơ sở đối chiếu giữa mô hình số thực, mô hình fixed-point và kết quả phần cứng.

Ở mức phần cứng, đề tài hiện thực chuỗi xử lý end-to-end PCM-MFCC-Feature Buffer-MLP bằng số học fixed-point trên FPGA. Pipeline MFCC tạo 13 hệ số cho mỗi frame, 98 frame liên tiếp được ghép thành vector 1.274 phần tử và đưa vào MLP 1274-32-20. Các phép tính được lượng tử hóa theo Q-format thống nhất để phù hợp với tài nguyên LUT, DSP và BRAM.

Từ mục tiêu và phạm vi đã đặt ra, khóa luận tập trung vào ba đóng góp chính. Thứ nhất, đề tài xây dựng bài toán và dataset 20 từ khóa/nhãn nhạy cảm tiếng Việt. Thứ hai, đề tài thiết kế pipeline end-to-end PCM-MFCC-Feature Buffer-MLP cho bài toán KWS. Thứ ba, đề tài hiện thực toàn bộ hệ thống theo số học fixed-point trên FPGA. Bên cạnh đó, khóa luận xây dựng luồng kiểm chứng phần mềm-RTL-FPGA để đánh giá độ chính xác, timeout/error, tài nguyên, timing và power. Thiết kế sử dụng 26.053 LUT, 21.879 FF, 74 BRAM, 131 DSP, đạt WNS = 0,225 ns ở clock 100 MHz, Fmax ước lượng 102,3 MHz và công suất on-chip ước lượng 0,595 W.

ABSTRACT

This thesis presents the research, design, implementation, and verification of an IP core for Vietnamese profanity keyword spotting in classroom environments. The system classifies each normalized audio segment into one of 20 predefined keywords or labels and returns the corresponding result_id. RTL simulation achieves 295/300 correct predictions on the validation set and 1,996/2,000 on the full dataset, without timeout or data-reading errors. On the Xilinx Kria KV260, the design achieves 294/300 on the validation batch and 1,995/2,000 on the full dataset.

In the proposed methodology, WAV files are normalized to signed 16-bit mono PCM at 16 kHz. Python is used to construct the reference model, extract MFCC features, train the MLP classifier, evaluate the fixed-point model, and export hardware coefficients. This workflow enables consistent comparison among the floating-point model, fixed-point model, RTL simulation, and FPGA execution.

The hardware implements an end-to-end PCM–MFCC–Feature Buffer–MLP processing chain using fixed-point arithmetic. The MFCC front-end produces 13 coefficients per frame. A sequence of 98 frames is concatenated into a 1,274-element feature vector and supplied to a 1274-32-20 MLP classifier. Consistent Q-formats are applied throughout the design to maintain numerical agreement while efficiently using LUT, DSP, and BRAM resources.

The thesis makes three main contributions. Constructing a 20-class Vietnamese profanity keyword dataset, designing an end-to-end KWS hardware architecture, and implementing the complete system using fixed-point arithmetic on FPGA. A software–RTL–FPGA verification flow is also developed to evaluate accuracy, timeout/error events, resource utilization, timing, and power. The final design uses 26,053 LUTs, 21,879 FFs, 74 BRAMs, and 131 DSPs. It achieves WNS = 0.225 ns at 100 MHz, a derived Fmax of approximately 102.3 MHz, and a Vivado post-implementation on-chip power estimate of 0.595 W.

CHƯƠNG 1. GIỚI THIỆU

1.1. Đặt vấn đề

Trong môi trường giáo dục, văn hóa giao tiếp học đường là một thành tố quan trọng góp phần hình thành môi trường học tập an toàn và lành mạnh, tác động đến quan hệ thầy trò, bạn bè, mức độ gắn kết cũng như kết quả học tập của học sinh [1]. Tuy nhiên, việc kiểm soát ngôn từ trong lớp học không phải lúc nào cũng được thực hiện hiệu quả, đặc biệt ở các lớp đông học sinh hoặc khi nhiều nhóm thảo luận diễn ra đồng thời. Những phát ngôn lệch chuẩn, bao gồm từ ngữ tục tĩu hoặc mang tính xúc phạm, có thể xuất hiện trong thời gian ngắn, khiến giáo viên khó phát hiện và can thiệp kịp thời [2].

Về mặt triển khai, nhận diện âm thanh có thể được thực hiện trên điện thoại, máy tính cá nhân hoặc máy chủ cloud. Các phương án này thuận tiện cho giai đoạn phát triển thuật toán, nhưng trong bối cảnh lớp học, hệ thống còn phải đáp ứng các yêu cầu về xử lý tại chỗ, bảo mật dữ liệu âm thanh, độ trễ ổn định và mức tiêu thụ tài nguyên có thể kiểm soát. Vì vậy, các giải pháp phụ thuộc nhiều vào phần mềm hoặc hạ tầng mạng chưa thật sự phù hợp trong mọi điều kiện vận hành. Các nghiên cứu KWS ở thiết bị biên cũng cho thấy bài toán này thường cần mô hình nhỏ, xử lý tại chỗ và giảm phụ thuộc vào hạ tầng mạng [11], [12].

Xuất phát từ các yêu cầu trên, FPGA được chọn làm nền tảng nghiên cứu cho IP nhận diện từ khóa trong khóa luận này. Khác với cách triển khai thuần phần mềm, FPGA cho phép xử lý trực tiếp luồng âm thanh tại thiết bị, giúp giảm độ trễ và hỗ trợ phản hồi kịp thời trong môi trường lớp học. Bên cạnh đó, các bước xử lý lặp lại trong hệ thống KWS như tiền xử lý tín hiệu, FFT, Mel filterbank, DCT và các phép nhân, cộng trong MLP có thể được hiện thực thành các khối phần cứng chuyên dụng. Cách tiếp cận này cho phép khai thác pipeline, song song hóa và kiểm soát trực tiếp độ rộng fixed-point, từ đó đánh giá được đồng thời độ chính xác, tài nguyên phần cứng, timing và công suất trên cùng một nền tảng triển khai [21], [22].

1.2. Mục tiêu đề tài

Đề tài hướng tới mục tiêu thiết kế luồng IP xử lý âm thanh từ mô hình thuật toán đến thiết kế RTL và kiểm thử trên phần cứng. Các mục tiêu cụ thể gồm:

- Nghiên cứu cơ sở lý thuyết của KWS, đặc tính tín hiệu tiếng nói tiếng Việt, lựa chọn tần số lấy mẫu 16 kHz, định dạng PCM 16-bit và quy trình trích xuất đặc trưng MFCC dựa trên các tài liệu xử lý tiếng nói và xử lý tín hiệu số [3], [4].
- Xây dựng mô hình tham chiếu Python gồm pipeline MFCC, mô hình MLP, quy trình huấn luyện, đánh giá floating-point và mô hình fixed-point để sinh golden data và file hệ số cho thiết kế phần cứng.
- Thiết kế và hiện thực pipeline phần cứng gồm MFCC, Feature Buffer và MLP Core bằng số học fixed-point, có giao tiếp top-level phù hợp để tích hợp vào hệ thống PS-PL trên KV260 [21], [22].
- Xây dựng quy trình kiểm chứng định lượng gồm module-level test, self-checking testbench, scoreboard, batch regression, functional coverage, assertion checking và kiểm thử board-level bằng chương trình C/Linux.

1.3. Nội dung thực hiện đề tài

- Đối tượng nhận dạng: Phạm vi nhận dạng của đề tài được giới hạn trong bài toán Keyword Spotting, với 20 từ khóa/nhãn tục tũ tiếng Việt đã được định nghĩa trước trong dataset. Hệ thống trả về mã nhận diện tương ứng với từ khóa trong tập đã huấn luyện.
- Danh sách 20 từ khóa/nhãn và mã result_id sử dụng trong dataset được trình bày ở Bảng 1.1. Các từ khóa này chỉ được dùng cho mục đích nghiên cứu, huấn luyện và kiểm chứng hệ thống KWS trong phạm vi khóa luận.

Bảng 1.1. Danh sách 20 từ khóa/nhãn tục tũ tiếng Việt sử dụng trong dataset.

result_id	Nhãn dataset	result_id	Nhãn dataset
0	01_địt	10	11_xạo lồn
1	02_lồn	11	12_súc vật

2	03_cặ	12	13_khốn nạn
3	04_éo	13	14_mất dạy
4	05_buổi	14	15_thằng chó
5	06_đĩ	15	16_hãm lòn
6	07_địt mẹ	16	17_xâm cút
7	08_đụ má	17	18_ngu vãi lòn
8	09_vãi lòn	18	19_cái lòn má
9	10_óc chó	19	20_vãi cả cút

- Dữ liệu đầu vào: Mỗi mẫu âm thanh được chuẩn hóa về mono, PCM signed 16-bit, tần số lấy mẫu 16 kHz. Theo định lý lấy mẫu Nyquist, tần số 16 kHz cho phép biểu diễn các thành phần đến 8 kHz, dải này phù hợp với thông tin chính của tiếng nói và giúp giảm chi phí tính toán so với các cấu hình lấy mẫu cao hơn [5], [28]. Định dạng PCM 16-bit biểu diễn mẫu âm thanh dưới dạng số nguyên có dấu, thuận lợi cho việc đối chiếu Python và hiện thực datapath fixed-point trong thiết kế RTL [28].

Giải pháp kỹ thuật: Thiết kế được triển khai trên nền tảng phần cứng FPGA Xilinx Kria KV260, sử dụng tài nguyên LUT/FF, DSP48E2 và BRAM của vùng PL. Các phép tính được đưa về fixed-point để kiểm soát tài nguyên, độ trễ và sai số lượng tử hóa [19], [21], [22].

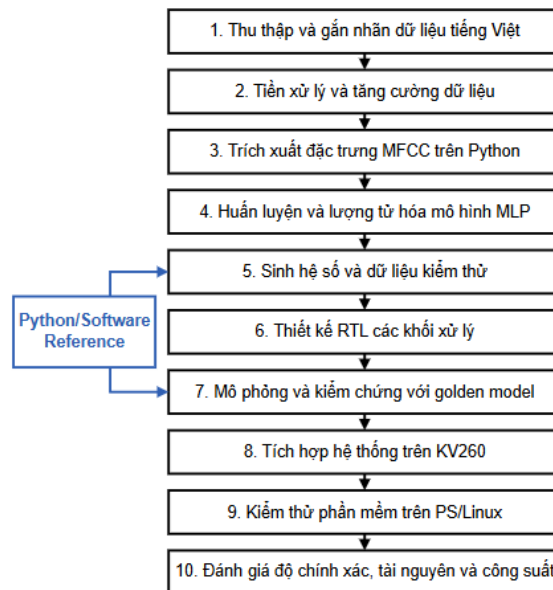
- Phương pháp thực hiện: Python tạo mô hình tham chiếu, dữ liệu golden và file HEX, thiết kế RTL hiện thực pipeline phần cứng, Vivado/XSim kiểm chứng bằng waveform, scoreboard và batch regression, KV260 kiểm thử bằng chương trình C/Linux thông qua UIO/mmap.

Bảng 1.2. Yêu cầu và thông số thiết kế.

Yêu cầu	Thông số sử dụng trong đề tài
Tín hiệu đầu vào	PCM 16-bit, mono, 16 kHz
Độ dài mẫu âm thanh	1 giây danh định, xử lý 16.032 mẫu để tạo đủ 98 frame
Số từ khóa/nhân nhận diện	20 từ khóa/nhân tục tiêu tiếng Việt
Đặc trưng âm thanh	13 hệ số MFCC / frame
Độ dài frame	512 mẫu

Yêu cầu	Thông số sử dụng trong đề tài
Hop size	160 mẫu
Số frame ngữ cảnh	98 frame
Kích thước vector MLP	$98 \times 13 = 1274$
Cấu trúc MLP	1274-32-20
Nền tảng triển khai	Xilinx KV260
Công cụ	Xilinx Vivado, Python golden model, chương trình C kiểm thử trên Linux
Kết quả đầu ra	result_valid, result_id[5:0] trong dải 0-19

Phạm vi đề tài được giới hạn ở KWS với tập từ khóa hữu hạn. Hệ thống không phân tích ngữ nghĩa hội thoại và chưa xử lý trực tiếp microphone thời gian thực. Việc đánh giá accuracy được thực hiện trên dataset tiếng Việt tự xây dựng, do đó các kết quả chỉ có ý nghĩa trong phạm vi dataset, mô hình MLP và thiết kế RTL tương ứng.



Hình 1.1. Quy trình nghiên cứu và hiện thực hệ thống.

Hình 1.1 tóm tắt cách đề tài được triển khai từ dữ liệu tiếng Việt đến mô hình phần mềm, sau đó từng bước đưa xuống thiết kế phần cứng và kiểm thử trên KV260. Luồng Python đóng vai trò tham chiếu, tạo đặc trưng MFCC, hệ số MLP và dữ liệu kỳ vọng. Luồng phần cứng hiện thực các phép toán bằng RTL fixed-point và được đối chiếu liên tục với dữ liệu tham chiếu. Nhờ cách tổ chức này, sai lệch có thể được kiểm chứng tại từng khối thay vì chỉ phát hiện ở result_id cuối cùng.

CHƯƠNG 2. TÌM HIỂU TỔNG QUAN

2.1. Cơ sở lý thuyết

2.1.1. Cơ chế sinh thanh và tính chất vật lý của tín hiệu tiếng nói

Tiếng nói là tín hiệu âm học được tạo ra từ quá trình phát âm của con người, trong đó luồng khí từ phổi đóng vai trò nguồn năng lượng chính tạo nên cơ chế phát âm. Khi đi qua thanh quản, luồng khí này có thể làm dây thanh dao động để tạo âm hữu thanh, hoặc tạo dòng nhiễu khí động đối với các âm vô thanh. Sau đó, âm thanh được cộng hưởng và biến đổi bởi khoang họng, khoang miệng và khoang mũi trước khi phát ra ngoài. Cách mô tả này thường được gọi là mô hình nguồn-bộ lọc, source-filter model [4]. Trong mô hình này, nguồn âm quyết định các đặc tính như cao độ cơ bản và hữu thanh/vô thanh, còn phần bộ lọc, tức đường thanh âm, quyết định đường bao phổ và các vùng cộng hưởng đặc trưng của tiếng nói.

Xét trong miền thời gian dài, tiếng nói là tín hiệu không dừng, biên độ, tần số cơ bản và phân bố năng lượng thay đổi theo sự chuyển tiếp giữa các âm vị. Vì vậy, nếu áp dụng Fourier cho toàn bộ từ hoặc toàn bộ câu, ta chỉ thu được thành phần tần số tổng quát mà không biết rõ thời điểm từng âm xuất hiện. Trong các khoảng ngắn khoảng 20-32 ms, tiếng nói thường được xem là gần dừng, đây là cơ sở của phân tích ngắn hạn trong các hệ thống nhận dạng tiếng nói [4], [30].

Với tần số lấy mẫu 16 kHz, một frame 512 mẫu của đề tài tương ứng $512/16000 = 0,032 \text{ s} = 32 \text{ ms}$.

$$T_{\text{frame}} = \frac{N}{f_s} = \frac{512}{16000} = 0,032 \text{ s} = 32 \text{ ms} \quad (2.1)$$

Khoảng thời gian 32 ms đủ dài để cung cấp độ phân giải tần số cho FFT 512 điểm, đồng thời vẫn nằm trong miền phân tích ngắn hạn thường dùng cho tiếng nói. Bước nhảy 160 mẫu tương ứng 10 ms, tạo mật độ khoảng 100 frame/giây và giúp theo dõi sự biến đổi của âm vị theo thời gian [4], [30].

2.1.2. Đặc trưng phổ năng lượng và âm vị học

a. Tần số cơ bản và họa âm

Đối với âm hữu thanh, dao động gần tuần hoàn của dây thanh tạo ra tần số cơ bản F0. Các thành phần tại bội số nguyên của F0 tạo thành hệ họa âm. Tuy nhiên, nội dung âm vị không chỉ phụ thuộc vào F0 mà còn phụ thuộc mạnh vào đường bao phổ do cộng hưởng của ống thanh quản. Các đỉnh cộng hưởng chính được gọi là formant F1, F2, F3, trong đó F1 và F2 có vai trò đặc biệt quan trọng đối với sự phân biệt nguyên âm [4].

Trong miền thời gian, hai âm vị có thể tạo dạng sóng phức tạp và khó so sánh trực tiếp. Trong miền tần số, vị trí và biên độ tương đối của các đỉnh phổ cung cấp biểu diễn ổn định hơn về cấu hình phát âm [4]. Đây là lý do hệ thống không đưa trực tiếp 16.000 mẫu PCM vào MLP mà chuyển tín hiệu sang tập đặc trưng phổ gọn hơn.

Một số phụ âm sát và âm tắc có thành phần năng lượng cao tần rõ hơn so với nguyên âm. Nếu độ nghiêng phổ tự nhiên của tiếng nói không được bù, các thành phần này có thể bị che lấp bởi năng lượng thấp tần. Vì vậy, bước tiền nhân được sử dụng trước khi chia frame và phân tích FFT [4], [30].

b. Thanh điệu tiếng Việt và ảnh hưởng đến KWS

Tiếng Việt là ngôn ngữ có thanh điệu, sự thay đổi cao độ và đường F0 có thể làm thay đổi ý nghĩa của âm tiết. Trong hội thoại thực tế, các từ khóa ngắn còn có thể bị nói lướt, nối âm, giảm âm lượng hoặc biến đổi cao độ. Vì vậy, hệ thống không nên chỉ dựa vào năng lượng tức thời của một frame đơn lẻ, mà cần kết hợp đặc trưng phổ MFCC với ngữ cảnh nhiều frame để mô tả diễn biến của cả từ khóa [4], [30].

Về mặt phổ năng lượng, tiếng nói thường tập trung năng lượng ở vùng tần số thấp và trung bình, trong khi thông tin phân biệt âm vị còn phụ thuộc vào đường bao phổ, formant và các chuyển tiếp ngắn [4]. Với tần số lấy mẫu 16 kHz, hệ thống giữ các bin FFT từ 0 đến 8 kHz, đủ để biểu diễn vùng tần số hữu ích của tiếng nói trong phạm vi thiết kế [5], [28].

2.1.3. Yêu cầu kỹ thuật khi triển khai trên FPGA

a. Số học dấu phẩy tĩnh

Máy tính thông thường sử dụng số dấu phẩy động IEEE 754 để biểu diễn số thực với dải động lớn. Trên FPGA, các phép toán floating-point có thể được hiện thực bằng IP chuyên dụng nhưng thường làm tăng độ trễ, số LUT/FF và DSP. Để kiểm soát tài nguyên và bảo đảm hành vi số học có thể dự đoán, thiết kế sử dụng số học dấu phẩy tĩnh, đây cũng là hướng phổ biến khi triển khai mô hình học máy nhỏ trên phần cứng tài nguyên hữu hạn [19], [21], [22].

Một giá trị thực x được lượng tử hóa thành số nguyên x_q với F bit phần lẻ theo công thức:

$$x_q = \text{round}(x \cdot 2^F) \quad (2.2)$$

Giá trị gần đúng sau khi lượng tử hóa là:

$$\hat{x} = \frac{x_q}{2^F} \quad (2.3)$$

Nếu sử dụng phép làm tròn về số nguyên gần nhất, sai số lượng tử cực đại thỏa:

$$|e_q| = |x - \hat{x}| \leq \frac{1}{2^{F+1}} \quad (2.4)$$

Tăng số F giúp giảm sai số lượng tử hóa, nhưng đồng thời làm giảm số bit dành cho phần nguyên và tăng nguy cơ tràn. Do đó, Q-format phải được chọn theo dải động của từng tầng thay vì dùng một định dạng duy nhất cho toàn hệ thống [19].

Bảng 2.1. Độ rộng dữ liệu và quy ước fixed-point của thiết kế.

Thành phần	Độ rộng RTL	Quy ước biểu diễn	Vai trò
PCM đầu vào	16-bit signed	Số nguyên PCM	Mẫu âm thanh mono 16 kHz
Hệ số α , Hamming, Mel, DCT	16-bit signed	Q1.15	Hệ số có trị tuyệt đối xấp xỉ không vượt quá 1
FFT thực/ảo	20-bit signed mỗi thành phần	Theo scale của lõi fftmain	Kết quả FFT phức
Phổ công suất	48-bit unsigned	$\text{Re}^2 + \text{Im}^2$	Năng lượng của 257 bin
Bộ tích lũy Mel	64-bit signed	Power $\times$ Q1.15 tích lũy	Hạn chế overflow

Thành phần	Độ rộng RTL	Quy ước biểu diễn	Vai trò
Đầu ra Mel	64-bit signed	Accumulator dịch phải 15 bit	Đầu vào khối log
Log-Mel	32-bit signed	Q16.16, log cơ số 2	Đầu vào DCT
MFCC	32-bit signed	Q16.16	Đặc trưng lưu trong Feature RAM
Trọng số MLP	32-bit storage	Q4.12 theo script xuất hệ số	Tham số W1, W2
Bias MLP	32-bit storage	Q28	Cùng thang với tích Q16.16 $\times$ Q4.12
Accumulator MLP	64-bit signed	Miền tích MAC	Hạn chế overflow khi cộng nhiều tích

b. Làm tròn, cắt bit, bão hòa và tràn số

Sau phép nhân fixed-point, kết quả có số bit phần lẻ bằng tổng số bit phần lẻ của hai toán hạng. Để đưa kết quả về gần với kết quả floating-point, phần cứng phải dịch phải số học. Nếu chỉ cắt các bit thấp, sai số fixed-point có thể lệch về một phía, vì vậy các khối như pre-emphasis và windowing cộng hoặc trừ một nửa LSB trước khi dịch nhằm làm tròn về giá trị gần nhất [19].

Bão hòa và wrap-around phải được phân biệt. Bão hòa kẹp kết quả vào giá trị lớn nhất hoặc nhỏ nhất có thể biểu diễn, còn wrap-around giữ các bit thấp và có thể làm đảo dấu khi overflow.

c. Kiến trúc xử lý song song và pipeline

Để đạt được khả năng xử lý ổn định, hệ thống áp dụng kiến trúc pipeline và cơ chế handshaking. Hai kỹ thuật này phù hợp với đặc điểm của FPGA, nơi nhiều khối có thể hoạt động đồng thời trên các mẫu dữ liệu khác nhau [21], [22].

- Pipeline: Dữ liệu âm thanh truyền qua các khối chức năng nối tiếp nhau, gồm Pre-emphasis, Framing, Windowing, FFT, Mel Filterbank, Log và DCT. Các

khối MFCC được nối theo kiến trúc xử lý dòng và có thể chồng lấp hoạt động khi điều kiện valid/ready cho phép. Tuy nhiên, các khối không có cùng thông lượng; một số khối sử dụng FSM và xử lý tuần tự theo frame. MLP Core tái sử dụng MAC theo thời gian, không phải kiến trúc fully pipelined.

- Handshaking: Cơ chế valid/ready được dùng để đồng bộ dữ liệu giữa các khối có tốc độ xử lý khác nhau. Một phần tử dữ liệu chỉ được truyền khi tín hiệu valid của khối phát và tín hiệu ready của khối nhận cùng được kích hoạt.

d. Tối ưu hóa bộ nhớ

Tài nguyên bộ nhớ nội trên FPGA là hữu hạn, đặc biệt khi thiết kế cần lưu hệ số, bảng tra, dữ liệu frame và đặc trưng MFCC [21], [22]. Vì vậy, hệ thống không lưu toàn bộ file âm thanh trong phần cứng, mà chỉ lưu các phần tử cần thiết cho từng giai đoạn xử lý.

- Circular Buffer: Chỉ lưu cục bộ các mẫu cần thiết cho một frame xử lý, giúp tái sử dụng dữ liệu chồng lấn mà không cần dịch toàn bộ 512 mẫu.
- ROM Look-up Table: Các giá trị hằng số như hệ số cửa sổ, hệ số Mel, hệ số DCT, trọng số và bias MLP được tính trước và nạp vào ROM, giúp giảm tải tính toán trong lúc suy luận.

2.2. Quy trình trích xuất đặc trưng MFCC trong hệ thống

MFCC (Mel-Frequency Cepstral Coefficients) được lựa chọn vì có khả năng mô tả đường bao phổ của tiếng nói, giảm chiều dữ liệu và tạo vector đặc trưng phù hợp cho bộ phân loại nhỏ [3], [30]. Thay vì đưa trực tiếp chuỗi PCM dài vào MLP, pipeline MFCC chuyển tín hiệu qua các miền biểu diễn liên tiếp, từ thời gian sang tần số, từ phổ tuyến tính sang thang Mel, sau đó nén thành các hệ số cepstral.

Trong đề tài, pipeline trích xuất đặc trưng gồm 7 giai đoạn nối tiếp, mỗi giai đoạn được sử dụng với mục đích rõ ràng và đều được chọn để phù hợp với hiện thực fixed-point trên FPGA:

1. Tiền nhấn (Pre-emphasis): Bù xu hướng suy giảm năng lượng ở vùng cao tần và làm nổi bật phụ âm, chuyển tiếp nhanh trong tiếng nói [4], [30].

2. Chia khung (Framing): Cắt tín hiệu thành các đoạn ngắn 32 ms để có thể xem gần đúng khi phân tích FFT [4], [30].
3. Cửa sổ hóa (Windowing): Làm mềm biên frame để giảm rò rỉ phổ khi chuyển sang miền tần số [5].
4. FFT và phổ công suất: Chuyển từng frame sang miền tần số và giữ thông tin năng lượng của 257 bin không âm [5], [29].
5. Mel Filterbank: Gom phổ tuyến tính thành 40 băng năng lượng theo thang gần với cảm nhận thính giác [3], [30].
6. Logarit năng lượng: Nén dải động của năng lượng Mel để các băng phổ yếu không bị che lấp bởi một vài đỉnh năng lượng lớn [3], [30].
7. DCT: Giảm tương quan giữa các băng Log-Mel và nén đặc trưng còn 13 hệ số MFCC cho mỗi frame [3], [30].

2.2.1. Tiền nhấn (Pre-emphasis)

a. Cơ sở lý thuyết

Tiếng nói tự nhiên thường có năng lượng thấp tần mạnh hơn cao tần do đặc tính nguồn thanh môn và bức xạ tại môi, hiện tượng này thường được gọi là spectral tilt [4], [30]. Nếu đưa tín hiệu trực tiếp vào FFT, các thành phần cao tần quan trọng cho phụ âm và chuyển tiếp âm vị có thể bị yếu tương đối so với vùng thấp tần. Vì vậy, bước tiền nhấn được sử dụng để làm phẳng tương đối đường bao phổ trước khi chia frame.

b. Nguyên lý hoạt động

Bộ lọc tiền nhấn được mô tả bằng hàm truyền bậc một trong miền z :

$$H(z) = 1 - \alpha z^{-1} \quad (2.5)$$

Phương trình sai phân tương ứng là:

$$y[n] = x[n] - \alpha x[n - 1] \quad (2.6)$$

Trong đó, $x[n]$ là mẫu đầu vào, $y[n]$ là mẫu sau tiền nhấn và α là hệ số lọc.

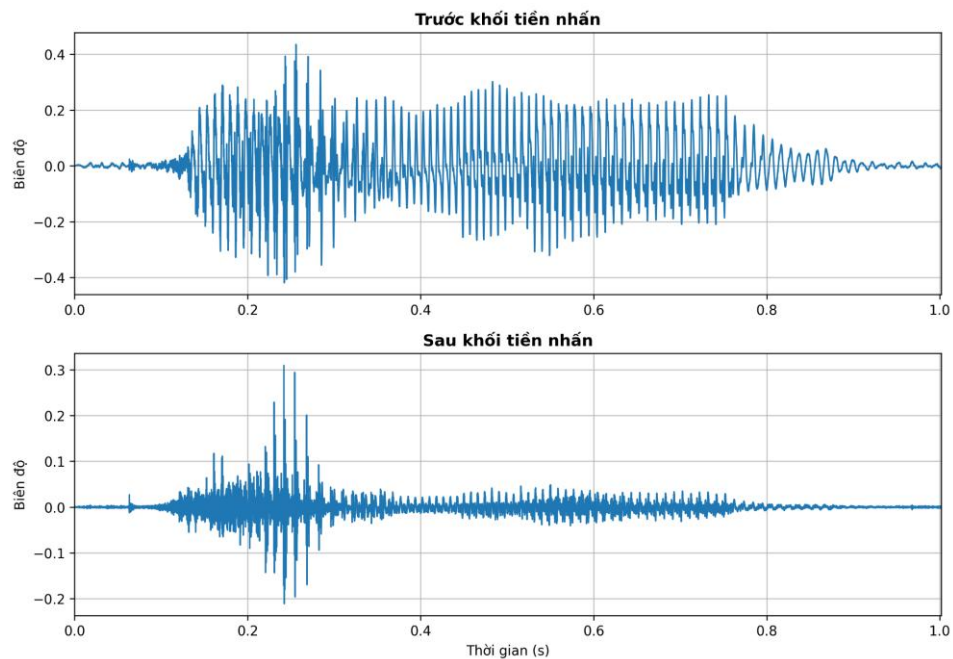
Đề tài chọn $\alpha = 0,97$, khi biểu diễn Q1.15, hệ số nguyên là:

$$\alpha_q = \text{round}(0,97 \cdot 2^{15}) = 31785 \quad (2.7)$$

Mẫu đầu tiên sau reset sử dụng $x[-1] = 0$. Trong thiết kế RTL, tích $\alpha x[n-1]$ được làm tròn trước khi dịch phải 15 bit. Chi tiết này được mô phỏng giống trong Python bit-accurate để tránh sai khác tích lũy giữa mô hình tham chiếu và thiết kế phần cứng.

c. Kết quả đạt được

Hình 2.1 minh họa một đoạn tín hiệu trước và sau khi áp dụng bộ lọc tiền nhấn với $\alpha = 0,97$.



Hình 2.1. Dạng sóng tín hiệu tiếng nói trước và sau bước tiền nhấn trên Python.

Trong Hình 2.1, trục ngang biểu diễn thời gian theo giây, trục dọc biểu diễn biên độ tín hiệu chuẩn hóa. Đồ thị phía trên là tín hiệu PCM gốc ở tần số lấy mẫu 16 kHz. Các đoạn hữu thanh tạo dao động tuần hoàn với biên độ tương đối lớn, trong khi đường bao thay đổi chậm theo nội dung phát âm.

Đồ thị phía dưới là tín hiệu sau tiền nhấn. Do phép sai phân giữa mẫu hiện tại và mẫu trước đó, các thành phần biến thiên chậm bị giảm tương đối, còn các sườn thay đổi nhanh được làm nổi bật. Mục tiêu của bước này không phải khuếch đại toàn bộ tín hiệu, mà là thay đổi phân bố phổ để phụ âm và chuyển tiếp cao tần đóng góp

rõ hơn trong phổ FFT. Sau tiền nhân, dữ liệu được chuyển sang khối framing để phân tích theo từng đoạn ngắn.

2.2.2. Chia khung (Framing)

a. Cơ sở lý thuyết

Một từ khóa kéo dài qua nhiều âm vị, trong khi đặc tính âm học của tiếng nói thay đổi liên tục theo thời gian. Nếu lấy FFT một lần cho toàn bộ đoạn 1 giây, hệ thống sẽ tính năng lượng tổng quát theo tần số nhưng mất thông tin về thời điểm xuất hiện của từng âm. Vì vậy, tín hiệu sau tiền nhân được chia thành các frame ngắn để có thể xem gần đúng trong từng khoảng 32 ms [4], [30]. Frame thứ m được biểu diễn bởi:

$$x_m[n] = x[n + mH], \quad 0 \leq n < N \quad (2.8)$$

b. Nguyên lý hoạt động

Trong đó, H là bước nhảy giữa hai frame liên tiếp. Với $N = 512$ và $H = 160$, độ chồng lấn giữa hai frame là:

$$N_{\text{overlap}} = N - H = 512 - 160 = 352 \text{ mẫu} \quad (2.9)$$

Tỷ lệ chồng lấn tương ứng là:

$$R_{\text{overlap}} = \frac{N - H}{N} = \frac{352}{512} = 68,75\% \quad (2.10)$$

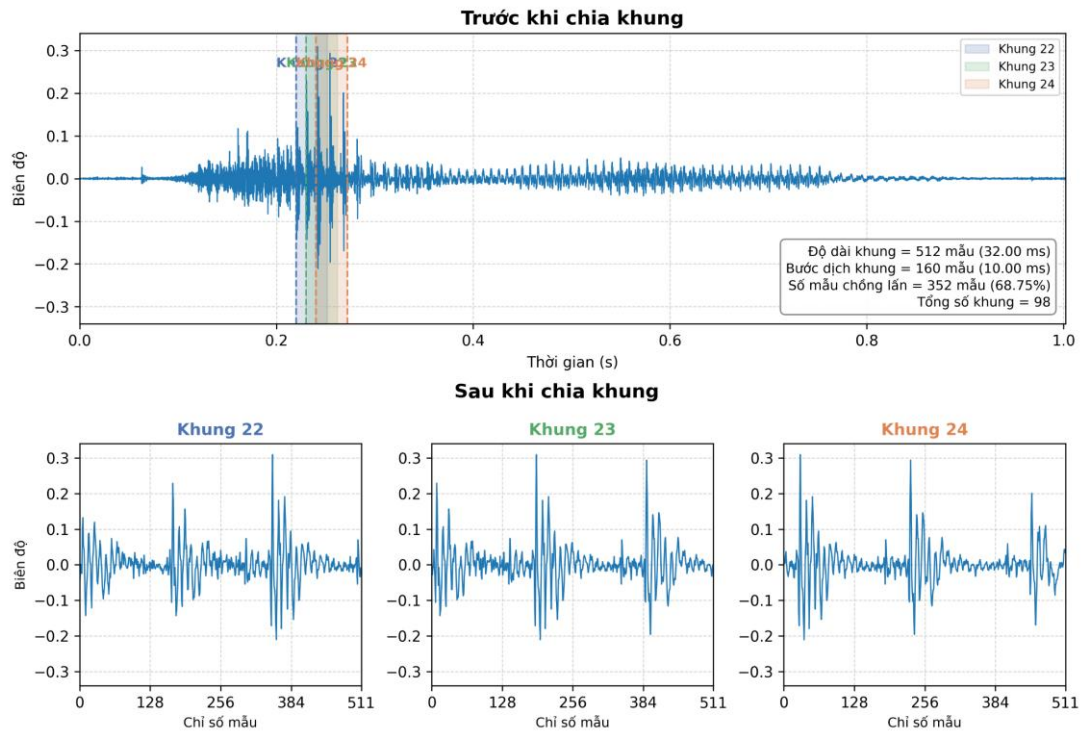
c. Kết quả đạt được

Với tần số lấy mẫu 16 kHz, frame 512 mẫu dài 32 ms và hop 160 mẫu dài 10 ms. Cấu hình này cân bằng giữa độ phân giải tần số và độ phân giải thời gian, đồng thời tạo đủ 98 frame cho đoạn âm thanh 1 giây danh định [4], [30].

Ở đồ thị phía trên của Hình 2.2, trục ngang là thời gian của toàn bộ tín hiệu, còn trục dọc là biên độ chuẩn hóa. Các vùng màu đánh dấu vị trí của những frame liên tiếp trên cùng chuỗi đầu vào. Vì hop chỉ bằng 160 mẫu trong khi frame dài 512 mẫu, các vùng frame chồng lên nhau đáng kể.

Ba đồ thị phía dưới biểu diễn riêng các frame đại diện. Trục ngang được chuyển từ thời gian toàn cục sang chỉ số mẫu cục bộ 0-511, còn trục dọc vẫn là biên độ. Mỗi

frame chứa 512 mẫu liên tục, frame kế tiếp bắt đầu muộn hơn frame trước 160 mẫu nên tái sử dụng 352 mẫu cũ. Trước framing, dữ liệu là một chuỗi duy nhất, sau framing, dữ liệu trở thành tập các vector cùng chiều dài, mỗi vector mô tả trạng thái âm học trong 32 ms.



Hình 2.2. Quá trình chia frame 512 mẫu và cơ chế chồng lấn hop 160 mẫu.

Cơ chế overlap giúp giảm nguy cơ bỏ sót chuyển tiếp âm vị ở biên frame và tạo chuỗi đặc trưng dày theo thời gian để hình thành context 98 frame. Trong thiết kế RTL, khối framing sử dụng RAM vòng 512 phần tử. Khi nhận mẫu mới, dữ liệu được ghi tại `wr_ptr`, khi cần xuất frame, `rd_ptr` quét tuần tự 512 mẫu trong RAM. Cách tổ chức này tái sử dụng 352 mẫu chồng lấn mà không cần dịch toàn bộ 512 thanh ghi.

2.2.3. Nhân cửa sổ (Windowing)

a. Cơ sở lý thuyết

Việc cắt một đoạn hữu hạn từ tín hiệu dài tương đương nhân tín hiệu với cửa sổ chữ nhật. Khi FFT xem frame hữu hạn như một tín hiệu tuần hoàn, sự không liên tục tại hai biên frame có thể làm năng lượng của một thành phần tần số lan sang các bin

lân cận, hiện tượng này gọi là rò rỉ phổ [5]. Rò rỉ phổ làm mờ vị trí các đỉnh năng lượng và ảnh hưởng trực tiếp đến đầu ra Mel filterbank.

b. Nguyên lý hoạt động

Đề tài sử dụng cửa sổ Hamming vì cửa sổ này làm giảm biên frame nhưng vẫn giữ vùng giữa frame gần nguyên vẹn [5]. Hệ số cửa sổ được tính như sau:

$$w[n] = 0,54 - 0,46\cos\left(\frac{2\pi n}{N-1}\right), \quad 0 \leq n < N \quad (2.11)$$

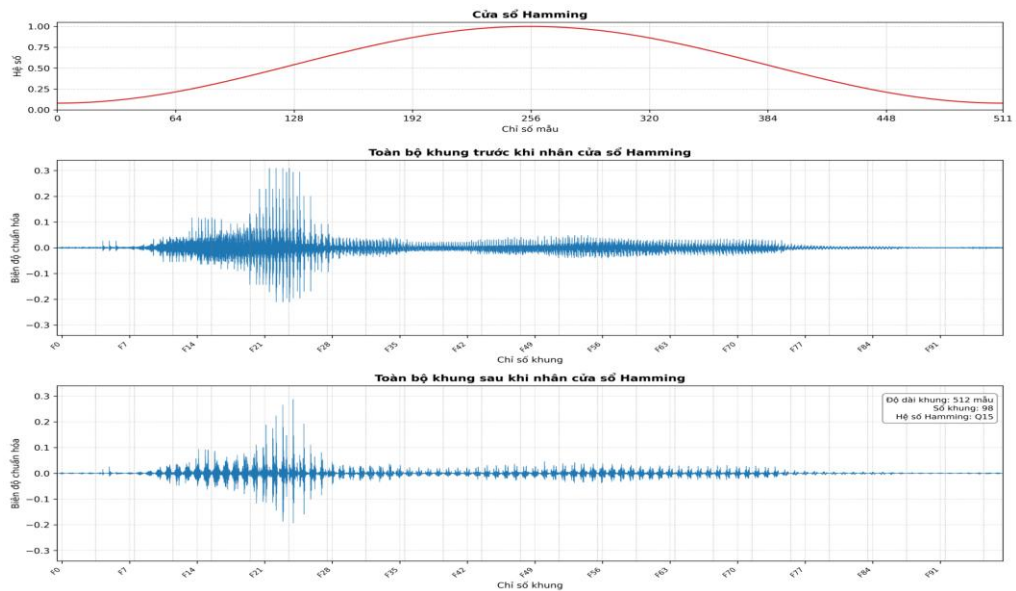
Tín hiệu sau cửa sổ hóa là:

$$x_w[n] = x_m[n] \cdot w[n] \quad (2.12)$$

Khác với cửa sổ Hann, Hamming không giảm đúng bằng 0 tại hai đầu mà có giá trị xấp xỉ 0,08. Đặc tính này giúp giảm búp phụ so với cửa sổ chữ nhật trong khi vẫn giữ búp chính tương đối hẹp [5].

c. Kết quả đạt được

Hình 2.3 mô tả hệ số Hamming và dạng sóng trước và sau khi nhân windowing. Đồ thị trên cùng có trục ngang là chỉ số hệ số từ 0 đến 511, trục dọc là giá trị cửa sổ. Đồ thị giữa là frame trước windowing, còn đồ thị dưới là kết quả sau phép nhân $x_w[n] = x_m[n] \times w[n]$.



Hình 2.3. Hệ số Hamming và dạng sóng frame trước, sau bước cửa sổ hóa.

Trước windowing, biên độ ở đầu và cuối frame có thể khác 0, tạo điểm gián đoạn khi frame lặp tuần hoàn trong phép DFT. Sau windowing, các mẫu gần biên bị suy giảm mạnh, còn vùng giữa frame được giữ gần nguyên vẹn. Nhờ đó, phổ FFT tập trung hơn quanh các thành phần thực và ít bị lan năng lượng sang bin lân cận. Trong thiết kế RTL, 512 hệ số Q1.15 được lưu trong ROM, mỗi mẫu 16 bit được nhân với hệ số tương ứng, làm tròn và dịch phải 15 bit để trở lại độ rộng 16 bit.

2.2.4. Biến đổi Fourier nhanh và phổ công suất

a. Cơ sở lý thuyết

Sau framing và windowing, mỗi frame vẫn là tín hiệu theo thời gian. Để biết năng lượng của frame nằm ở những dải tần nào, hệ thống cần chuyển dữ liệu sang miền tần số bằng DFT. DFT của frame đã cửa sổ hóa được định nghĩa như sau:

$$X[k] = \sum_{n=0}^{N-1} x_w[n] e^{-j2\pi kn/N}, \quad 0 \leq k < N \quad (2.13)$$

b. Nguyên lý hoạt động

DFT cần $O(N^2)$ phép toán. FFT khai thác tính đối xứng và tuần hoàn của hệ số quay để giảm độ phức tạp xuống $O(N \log N)$ [5], [29]. Với $N = 512$, cấu trúc radix-2 phù hợp cho hiện thực phần cứng.

Đầu ra FFT là số phức:

$$X[k] = \text{Re}[k] + j\text{Im}[k] \quad (2.14)$$

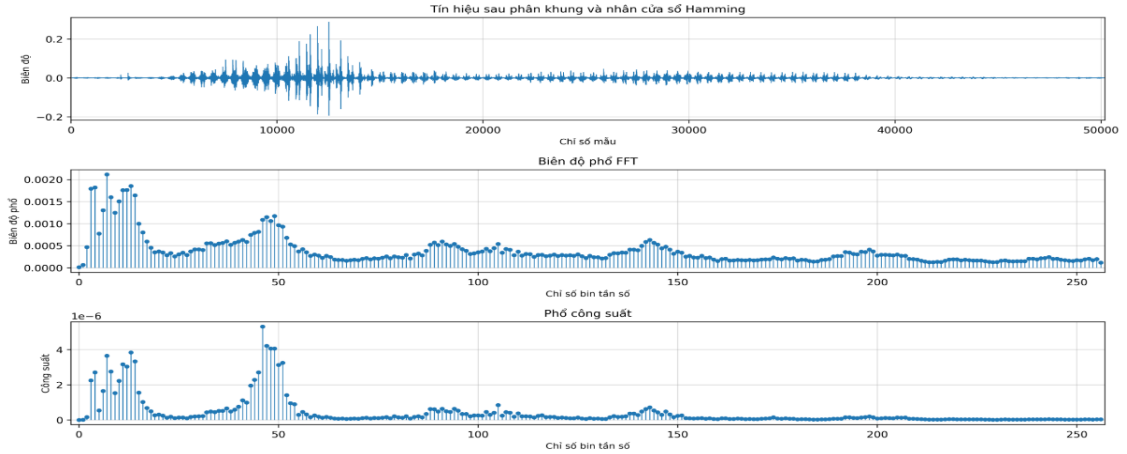
Trong KWS, hệ thống sử dụng phổ công suất và bỏ qua pha tuyệt đối của tín hiệu, phổ công suất được tính như sau:

$$P[k] = \text{Re}^2[k] + \text{Im}^2[k], \quad 0 \leq k \leq 256 \quad (2.15)$$

Vì đầu vào là tín hiệu thực, phổ có tính đối xứng liên hợp, nên thiết kế chỉ cần giữ 257 bin từ 0 đến 256, tương ứng 0 Hz đến tần số Nyquist 8 kHz [5].

c. Kết quả đạt được

Hình 2.4 trình bày tín hiệu sau phân khung và nhân cửa sổ Hamming, phổ biên độ FFT và phổ công suất của 257 bin sau FFT.



Hình 2.4. Phổ biên độ FFT và phổ công suất trung bình 257 bin sau FFT.

Hình 2.4 biểu diễn ba mức quan sát liên tiếp trong pipeline. Đồ thị phía trên là tín hiệu sau khi chia khung và nhân cửa sổ Hamming, đồ thị giữa là phổ biên độ FFT, còn đồ thị phía dưới là phổ công suất theo chỉ số bin tần số. Các đỉnh năng lượng tập trung nhiều hơn ở vùng tần số thấp và trung bình, phù hợp với đặc trưng phổ của tiếng nói [4]. Việc chỉ giữ 257 bin không âm giúp giảm số lượng phần tử cần xử lý trước khi đưa sang bộ lọc Mel.

Trước FFT, thông tin được sắp xếp theo thời gian, sau FFT, mỗi phần tử mô tả mức đóng góp của một dải tần. Những đỉnh ở vùng thấp và trung tần phản ánh thành phần cơ bản, họa âm và formant của frame. Việc giảm từ 512 mẫu thời gian xuống 257 bin không làm mất thông tin biên độ cần thiết vì nửa phổ còn lại đối xứng.

2.2.5. Bộ lọc Mel

a. Cơ sở lý thuyết

Tai người cảm nhận cao độ không tuyến tính theo tần số vật lý. Thang Mel tăng độ phân giải ở vùng thấp tần và giảm độ phân giải ở vùng cao tần, phù hợp với cách con người cảm nhận tiếng nói [3], [30]. Trong hệ thống MFCC, bộ lọc Mel được dùng để chuyển phổ tuyến tính của FFT thành các băng năng lượng gần với cảm nhận thính giác. Quan hệ giữa tần số Hz và thang Mel được mô tả bởi:

$$m(f) = 2595 \log_{10} \left(1 + \frac{f}{700} \right) \quad (2.16)$$

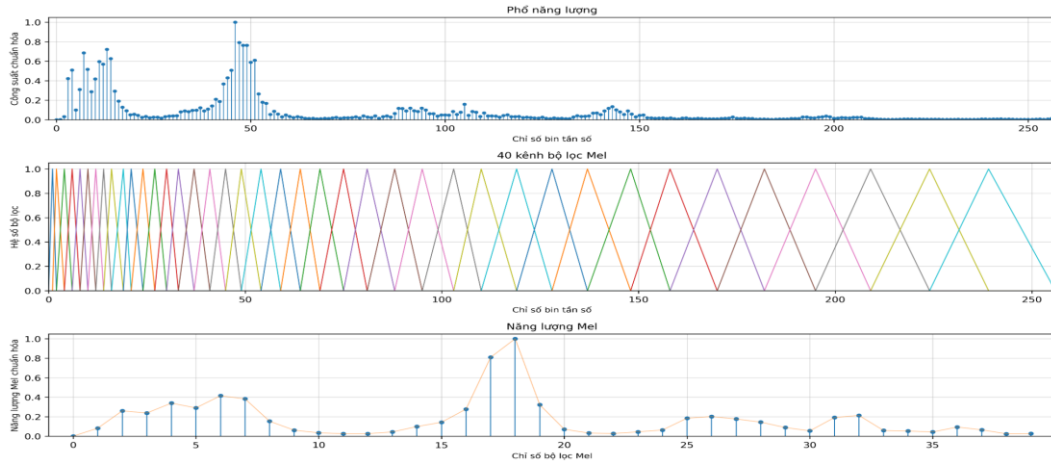
b. Nguyên lý hoạt động

Hệ thống sử dụng 40 bộ lọc tam giác $H_m[k]$ phủ lên 257 bin FFT. Năng lượng Mel thứ m được tính bằng cách nhân phổ công suất với bộ lọc tương ứng và cộng trên toàn bộ bin:

$$E_m = \sum_{k=0}^{256} P[k]H_m[k], \quad m = 0, 1, \dots, 39 \quad (2.17)$$

Các bộ lọc được đặt chồng lấn để biến phổ liên tục thành 40 năng lượng băng tần. Hệ số Mel được lượng tử hóa Q1.15, bộ tích lũy 64 bit được dùng để giảm nguy cơ overflow khi cộng nhiều tích $\text{power} \times \text{hệ số}$.

c. Kết quả đạt được



Hình 2.5. Bộ lọc tam giác Mel và vector năng lượng Mel.

Hình 2.5 gồm ba đồ thị. Đồ thị phía trên biểu diễn phổ công suất đầu vào, đồ thị giữa biểu diễn 40 bộ lọc tam giác trên trục bin tần số, mỗi đường là một bộ lọc có giá trị lớn nhất tại tần số trung tâm và giảm tuyến tính về 0 tại hai biên. Các tam giác dày hơn ở vùng tần số thấp và thưa hơn ở vùng cao, đúng với đặc tính thang Mel.

Đồ thị phía dưới biểu diễn vector 40 năng lượng Mel sau khi lọc. Trục ngang là chỉ số băng Mel 0-39, trục dọc là năng lượng sau tích lũy đã chuẩn hóa. So với 257 bin FFT, vector Mel giảm đáng kể số chiều nhưng vẫn giữ vị trí tương đối của các vùng phổ nổi bật. Trước bước Mel, dữ liệu còn gắn với lưới tần số tuyến tính, sau bước Mel, dữ liệu được phân bố theo độ phân giải gần với cảm nhận thính giác.

2.2.6. Logarit năng lượng

a. Cơ sở lý thuyết

Năng lượng Mel có dải động lớn, một số băng có thể lớn hơn các băng khác nhiều bậc độ lớn. Trong quy trình MFCC, phép log được dùng để nén dải động trước khi thực hiện DCT [3], [30]. Nếu đưa trực tiếp năng lượng tuyến tính vào DCT và MLP, các băng mạnh có thể chi phối phép tính và làm giảm ảnh hưởng của các cấu trúc phổ nhỏ. Phép log được mô tả theo:

$$L_m = \log_2(\max(E_m, \epsilon)) \quad (2.18)$$

Trong đó epsilon là giá trị nhỏ hoặc quy tắc kẹp được dùng để xử lý trường hợp năng lượng bằng 0. Thiết kế RTL sử dụng log cơ số 2, mô hình golden được xây dựng cùng cơ số và cùng scale để kết quả có thể đổi chiều.

b. Nguyên lý hoạt động

Đối với số nguyên dương X, đặt k là vị trí bit 1 cao nhất và chuẩn hóa X về dạng m trong khoảng [1,2), khi đó:

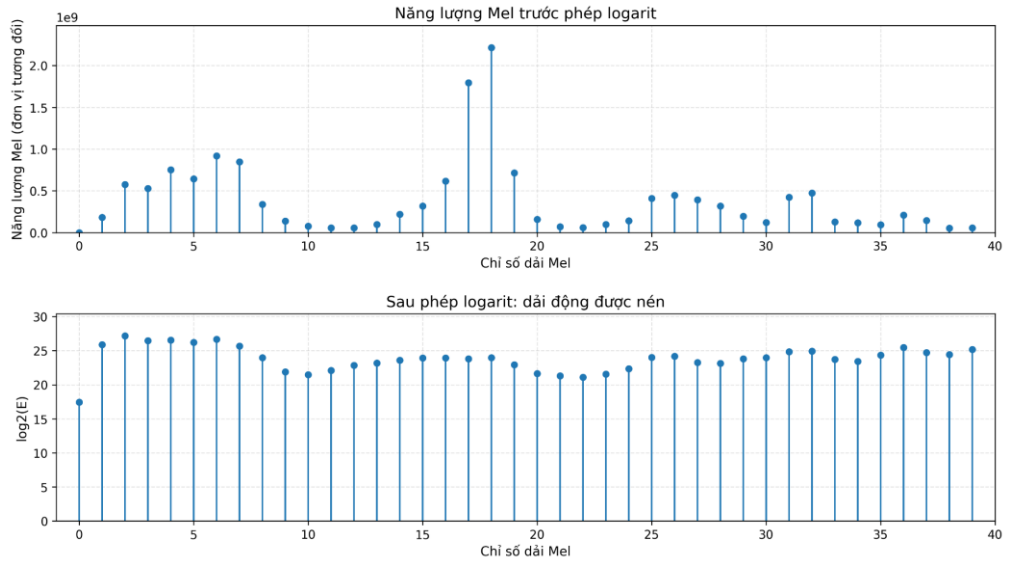
$$X = 2^k \cdot m, \quad 1 \leq m < 2, \quad \log_2(X) = k + \log_2(m) \quad (2.19)$$

Phần nguyên k được xác định bằng Leading-One Detector. Tám bit sau MSB tạo địa chỉ cho LUT 256 phần tử xấp xỉ phần thập phân. Cách tách phần nguyên và tra LUT giúp hiện thực log2 bằng phần cứng gọn hơn so với phép log floating-point trực tiếp [20]. Kết quả được ghép thành Q16.16.

c. Kết quả đạt được

Trong Hình 2.6, trục ngang của cả hai đồ thị là chỉ số băng Mel. Trục dọc của đồ thị trên là năng lượng tuyến tính, còn trục dọc của đồ thị dưới là $\log_2(E_m)$.

Trước phép log, một số đỉnh lớn làm các giá trị nhỏ gần như không thể quan sát trên cùng thang. Sau phép log, khoảng cách tương đối giữa đỉnh lớn và băng yếu được thu hẹp, khiến cấu trúc toàn bộ vector trở nên rõ hơn. Bước này không thay đổi thứ tự các băng vì log là hàm đơn điệu, nhưng làm giảm sự phụ thuộc vào biên độ tuyệt đối của người nói và khoảng cách thu âm.



Hình 2.6. Tác động của phép logarit lên dải động của vector năng lượng Mel.

2.2.7. DCT và vector MFCC

a. Cơ sở lý thuyết

Các băng Log-Mel lân cận thường còn tương quan cao vì phổ tiếng nói biến thiên trơn. Trong MFCC, DCT được dùng để giảm tương quan và nén phần lớn thông tin đường bao phổ vào các hệ số bậc thấp [3], [30]. Đề tài sử dụng DCT-II:

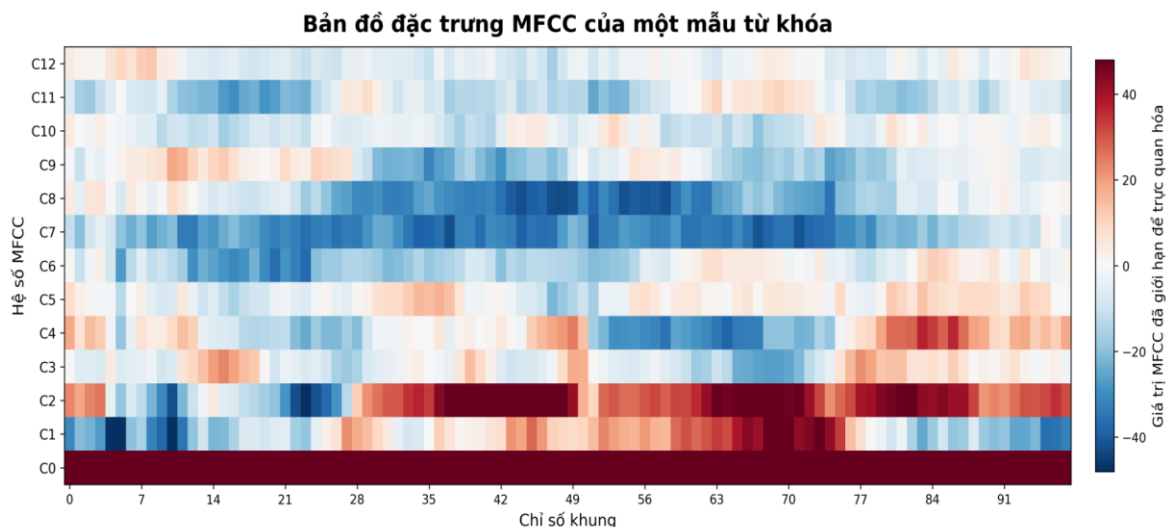
$$c_p = \sum_{m=0}^{39} L_m \cos \left[\frac{\pi p(m + 0,5)}{40} \right], \quad p = 0, 1, \dots, 12 \quad (2.20)$$

b. Nguyên lý hoạt động

Hệ số C0 phản ánh mức năng lượng hoặc log-energy tổng quát, còn các hệ số tiếp theo mô tả hình dạng đường bao phổ với mức chi tiết tăng dần. Hệ thống giữ 13 hệ số C0 đến C12, các hệ số bậc cao hơn bị loại để giảm chiều dữ liệu và hạn chế nhiễu chi tiết.

c. Kết quả đạt được

Hình 2.7 biểu diễn ma trận MFCC gồm 13 hệ số theo 98 frame thời gian. Trục ngang là chỉ số frame, trục dọc là hệ số MFCC C0-C12. Màu sắc thể hiện biên độ hệ số sau khi giới hạn dải hiển thị để tránh hệ số C0 chi phối toàn bộ bản đồ màu.



Hình 2.7. Heatmap MFCC kích thước 13×98 của một mẫu từ khóa.

Trước DCT, một frame được biểu diễn bằng 40 Log-Mel có tương quan. Sau DCT, frame được nén thành 13 số. Mức giảm chiều từ 40 xuống 13 giúp giảm số lần ghi Feature RAM và giảm số trọng số tầng đầu của MLP mà vẫn giữ các đặc trưng phổ chính. Các vết thay đổi theo trục thời gian trong Hình 2.7 cho thấy từ khóa không nằm ở một frame đơn lẻ, mà trải qua nhiều frame liên tiếp, đây là lý do cần bộ đệm đặc trưng 98 frame.

2.2.8. Bộ đệm đặc trưng (Feature RAM Buffer)

a. Cơ sở lý thuyết

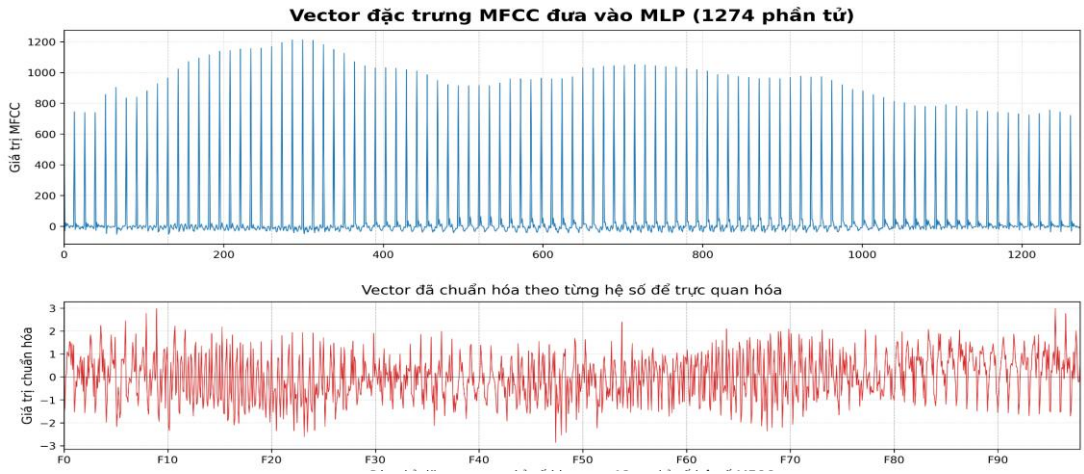
Quy trình trích xuất MFCC hoạt động theo cơ chế phân tích ngắn hạn, sinh ra một vector đặc trưng 13 chiều sau mỗi hop 10 ms [3], [30]. Tuy nhiên, một từ khóa trọn vẹn thường kéo dài qua nhiều frame và bao gồm chuỗi nhiều âm vị nối tiếp nhau. Nếu chỉ phân loại từng frame riêng lẻ, hệ thống dễ mất thông tin về thứ tự phát âm và chuyển tiếp giữa các âm.

MLP trong đề tài là mô hình truyền thẳng, không có bộ nhớ hồi tiếp như RNN/LSTM. Vì vậy, bộ đệm đặc trưng được dùng để gom nhiều frame liên tiếp thành một vector ngữ cảnh, giúp MLP quan sát diễn biến của từ khóa theo thời gian mà không cần thêm cơ chế trạng thái hồi tiếp phức tạp.

b. Nguyên lý hoạt động

Hệ thống sử dụng Feature RAM để lưu 98 frame MFCC liên tiếp. Mỗi frame gồm 13 hệ số, do đó vector đầu vào của MLP có kích thước $98 \times 13 = 1274$. Thiết kế RTL hiện tại hoạt động theo cơ chế block 98 frame, controller ghi đủ một block đặc trưng, phát mlp_start cho MLP xử lý và sau đó quay lại thu block tiếp theo.

c. Kết quả đạt được



Hình 2.8. Vector đặc trưng MFCC 1274 phần tử.

Trong Hình 2.8, các hệ số MFCC của 98 frame được sắp xếp thành vector tuyến tính theo quy tắc địa chỉ $feature_addr = frame_index \times 13 + cep_index$. Đây là đúng thứ tự dữ liệu được đưa vào MLP 1274-32-20, vì vậy hình này cũng minh họa cách Feature RAM tổ chức dữ liệu trước khi suy luận.

$$feature_addr = frame_index \times 13 + cep_index \quad (2.21)$$

$$N_{input} = 98 \times 13 = 1274 \quad (2.22)$$

Vì địa chỉ hợp lệ nằm trong 0-1273 nên bus địa chỉ cần 11 bit. Thiết kế RTL hiện tại hoạt động theo block, controller ghi lần lượt 1274 hệ số MFCC vào Feature RAM, khi feature_count đạt 1273, controller phát mlp_start. Khi MLP phát mlp_done, controller chốt result_id, phát result_valid và đặt lại bộ đếm để thu block tiếp theo. Thiết kế chưa thực hiện sliding window và không loại frame cũ nhất sau

mỗi frame mới. Việc mô tả đúng cơ chế block-based rất quan trọng khi tính latency và khi xây dựng chương trình C gửi dữ liệu.

2.3. Khối phân loại mạng nơ-ron, lượng tử hóa và huấn luyện MLP

Sau khi khối DCT tạo ra 13 hệ số MFCC cho từng frame, hệ thống cần một bộ phân loại để ánh xạ chuỗi đặc trưng âm học thành một trong 20 từ khóa/nhãn. Trong đề tài này, bộ phân loại được chọn là mạng nơ-ron truyền thẳng nhiều lớp (Multi-Layer Perceptron - MLP). Việc sử dụng MLP nhằm cân bằng giữa độ chính xác nhận diện, độ phức tạp phần cứng, khả năng lượng tử hóa fixed-point và khả năng kiểm chứng trên FPGA [8], [19].

Về phía thuật toán, đầu vào của MLP không phải một frame MFCC đơn lẻ mà là vector ngữ cảnh gồm 98 frame liên tiếp, tương ứng $98 \times 13 = 1274$ hệ số. Cách tổ chức này giúp mạng truyền thẳng vẫn quan sát được diễn biến thời gian của toàn bộ từ khóa trong khoảng xấp xỉ một giây. Phần thông tin theo thời gian được đưa vào mô hình thông qua cách ghép đặc trưng ở đầu vào, thay vì thông qua trạng thái hồi tiếp như trong RNN/LSTM. Đây là lựa chọn phù hợp với bài toán KWS hữu hạn 20 nhãn, trong đó mục tiêu là xác định một từ khóa đã biết xuất hiện trong một đoạn âm thanh ngắn.

Nếu sử dụng mạng nơ-ron tích chập (Convolutional Neural Network - CNN) hoặc DS-CNN, mô hình có thể khai thác tốt cấu trúc cục bộ trên miền thời gian-tần số [9], [13]. Tuy nhiên, khi hiện thực trên FPGA, các lớp tích chập thường cần thêm bộ đệm feature map, cơ chế quét cửa sổ, nhiều địa chỉ đọc đồng thời và logic điều phối phức tạp hơn. Nếu sử dụng mạng hồi tiếp hoặc LSTM, mô hình có khả năng biểu diễn phụ thuộc thời gian rõ hơn [10], nhưng cần lưu và cập nhật trạng thái ẩn qua từng bước thời gian, làm tăng độ phức tạp của FSM, bộ nhớ trạng thái và đường dữ liệu. Những kiến trúc này phù hợp với hệ thống nhận dạng lớn, không phải lựa chọn tối ưu cho mục tiêu của khóa luận là thiết kế một IP nhỏ gọn, dễ kiểm chứng và chạy ổn định trên FPGA.

MLP 1274-32-20 có cấu trúc tuyến tính rõ ràng, tầng đầu tiên biến đổi vector 1274 đặc trưng thành 32 giá trị ẩn, tầng thứ hai biến 32 giá trị ẩn thành 20 logit. Nhờ đó, toàn bộ phép suy luận có thể tổ chức bằng các phép nhân-cộng tích lũy (MAC), ROM trọng số, ROM bias, một bộ nhớ đặc trưng và FSM điều khiển tuần tự. Kiến trúc này cho phép tái sử dụng MAC theo thời gian thay vì mở rộng toàn bộ phép nhân song song, từ đó giảm áp lực lên tài nguyên DSP và giúp đường dữ liệu dễ đóng timing hơn.

Một lý do quan trọng khác là MLP phù hợp với quy trình fixed-point của đề tài. Trọng số có thể lượng tử hóa về Q4.12, đặc trưng MFCC dùng Q16.16, bias được đưa về Q28 và accumulator có thể thiết kế đủ rộng để kiểm soát overflow [19]. Các phép toán ReLU và ArgMax cũng thuận lợi cho phần cứng, ReLU chỉ cần kiểm tra dấu, còn ArgMax chỉ cần so sánh 20 logit. Vì vậy, thiết kế phần cứng không cần hiện thực softmax, phép mũ hoặc phép chia, giúp giảm độ trễ và giảm rủi ro sai khác giữa Python và thiết kế RTL.

Do đó, đề tài không khẳng định MLP luôn tốt hơn CNN hoặc RNN trong mọi bài toán nhận dạng tiếng nói. Đóng góp nằm ở việc lựa chọn và hiện thực một kiến trúc phân loại đủ gọn cho FPGA, phù hợp với dataset tiếng Việt của đề tài, đồng thời có thể đối sánh qua các mức Python floating-point, Python fixed-point, mô phỏng RTL và board-level KV260.

2.3.1. Cấu trúc mô hình MLP 1274-32-20

Đầu vào của MLP là vector đặc trưng gồm 1274 phần tử, tương ứng 98 frame và 13 hệ số MFCC trên mỗi frame. Vector này được đọc từ Feature RAM theo thứ tự tuyến tính, các hệ số của frame 0 đứng trước, tiếp theo là frame 1, cho đến frame 97. Thứ tự này phải thống nhất giữa script Python, module mfcc_feature_buffer và module mlp_core, nếu thứ tự flatten bị đảo hoặc lệch một frame, mạng vẫn chạy đúng về mặt số học nhưng kết quả phân loại có thể sai.

Mạng MLP gồm một tầng ẩn 32 nơ-ron và một tầng đầu ra 20 nơ-ron. Với vector đầu vào $x = [x_0, x_1, \dots, x_{1273}]$, giá trị tiền kích hoạt của nơ-ron ẩn thứ j được tính như sau:

$$a_j = \sum_{i=0}^{1273} x_i W_1[i, j] + b_1[j], \quad 0 \leq j < 32 \quad (2.23)$$

Tầng ẩn sử dụng hàm ReLU. Về mặt phần cứng, ReLU chỉ yêu cầu kiểm tra bit dấu và chọn giữa giá trị tính được hoặc 0, do đó không cần phép mũ, phép chia hoặc bảng tra phức tạp.

$$h_j = \max(0, a_j) \quad (2.24)$$

Tầng đầu ra nhận 32 giá trị h_j và sinh ra 20 logit, kết quả nhận dạng được xác định bằng phép ArgMax trực tiếp trên 20 logit, tương ứng với một nhãn từ khóa.

$$y_k = \sum_{j=0}^{31} h_j W_2[j, k] + b_2[k], \quad 0 \leq k < 20 \quad (2.25)$$

Trong khóa luận này, giá trị y_k ở tầng đầu ra được gọi là logit. Logit là điểm số chưa chuẩn hóa của từng lớp trước khi chuyển sang xác suất. Đối với bài toán phân loại 20 từ khóa/nhãn, MLP tạo ra 20 logit tương ứng với 20 class ID từ 0 đến 19. Giá trị logit càng lớn thì mô hình càng ưu tiên lớp đó so với các lớp còn lại [8].

Phần cứng không hiện thực softmax vì softmax chỉ biến đổi các logit thành xác suất tương đối nhưng không làm thay đổi vị trí phần tử lớn nhất. Do đó, điều kiện quyết định của IP là thứ tự tương đối giữa các logit, không phải dấu âm hoặc dương tuyệt đối của từng logit. Một logit âm không có nghĩa là lỗi, nó chỉ cho biết điểm số tuyến tính của lớp đó thấp hơn mốc 0 trong miền tính toán của tầng output.

Vì vậy, khi so sánh Python và thiết kế RTL ở Chương 5, tiêu chí quan trọng nhất là class có logit lớn nhất có giữ nguyên hay không. Sai số số học giữa hai miền có thể xuất hiện do fixed-point, nhưng nếu ArgMax không đổi thì result_id cuối cùng vẫn khớp mô hình tham chiếu.

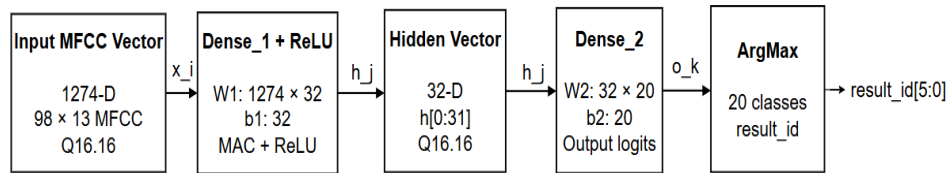
$$\text{result_id} = \text{argmax}_{0 \leq k < 20} (y_k) \quad (2.26)$$

Bảng 2.2. Cấu trúc và số lượng tham số của MLP.

Tầng	Kích thước	Số trọng số	Số bias	Vai trò
Input → Hidden	1274 → 32	40.768	32	Trích xuất tổ hợp phi tuyến từ chuỗi MFCC 98 frame
Hidden → Output	32 → 20	640	20	Tạo logit cho 20 từ khóa/nhân
Tổng	1274-32-20	41.408	52	41.460 tham số, gồm 41.408 trọng số và 52 bias cần lưu trong ROM

Tổng số tham số của MLP được tính theo công thức $1274 \times 32 + 32 + 32 \times 20 + 20 = 41.460$ tham số, gồm 41.408 trọng số và 52 bias. Trong phần hiện thực RTL, W1, b1, W2 và b2 được xem là bốn nhóm tham số/ROM, không phải bốn trọng số đơn lẻ.

Khối phân loại MLP



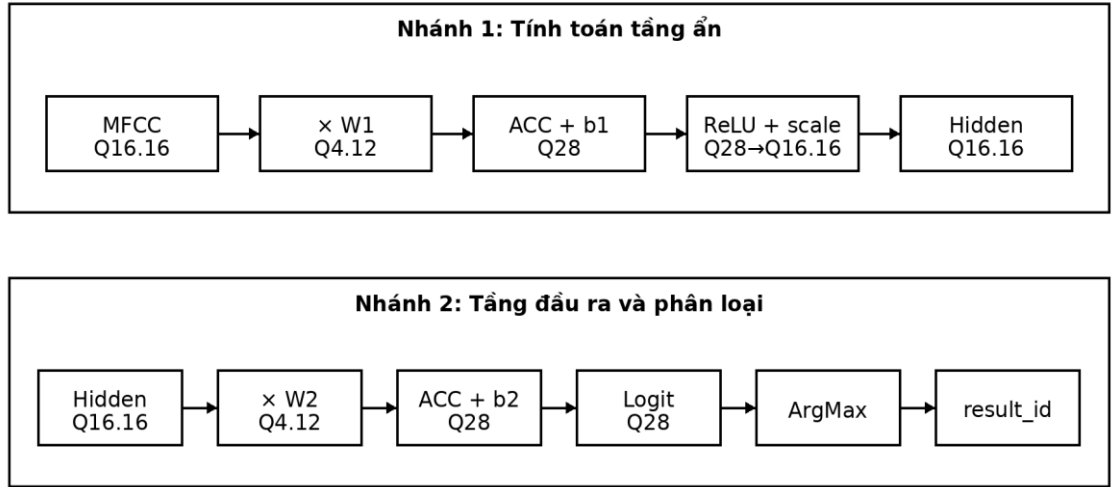
Hình 2.9. Kiến trúc mô hình MLP 1274-32-20 và luồng dữ liệu đặc trưng.

Trong Hình 2.9, khối đầu vào biểu diễn vector 1274 phần tử đọc từ Feature RAM. Luồng dữ liệu đi từ trái sang phải, vector MFCC ngữ cảnh được nhân với W1 để tạo 32 giá trị tiền kích hoạt, sau đó qua ReLU và được lưu tạm thành hidden vector. Hidden vector tiếp tục được nhân với W2 để tạo 20 logit. Mỗi logit là điểm số của một lớp, còn ArgMax ở cuối chỉ so sánh 20 điểm số này để xuất result_id. Vì ArgMax chỉ phụ thuộc vào logit lớn nhất, thiết kế phần cứng không cần hiện thực softmax.

2.3.2. Lượng tử hóa mô hình MLP và đồng bộ thang đo

Mô hình MLP sau khi huấn luyện trên Python sử dụng số thực, trong khi thiết kế RTL sử dụng số nguyên có dấu và số học fixed-point. Do đó, trọng số, bias và đặc trưng đầu vào phải được lượng tử hóa nhất quán để hạn chế sai số khi chuyển từ mô hình thuật toán sang phần cứng [19].

Hình 2.10 mô tả hai nhánh xử lý của mô hình MLP trong thiết kế phần cứng. Đầu vào MLP là MFCC ở định dạng Q16.16, trọng số được chọn ở dạng Q4.12 để cân bằng giữa dải động và sai số lượng tử.



Hình 2.10. Quy trình lượng tử hóa mô hình MLP cho RTL.

Khi nhân Q16.16 với Q4.12, tích trung gian có 28 bit phần thập phân. Vì vậy, bias phải được đưa về cùng thang Q28 trước khi cộng vào accumulator.

$$w_q = \text{round}(w_{\text{real}} \cdot 2^{12}) \quad (2.27)$$

$$b_q = \text{round}(b_{\text{real}} \cdot 2^{28}) \quad (2.28)$$

Phép tính tầng ẩn trong miền số nguyên có thể mô tả ở mức khái niệm như sau:

$$\text{ACC}_j = \sum_{i=0}^{1273} x_{i,q16} \cdot W_{1,i,j,q12} + b_{1,j,q28} \quad (2.29)$$

Kết quả ACC có cùng thang Q28. Trong thiết kế phần cứng, giá trị này được dịch phải để đưa về thang dùng cho tầng kế tiếp rồi áp dụng ReLU. Điểm quan trọng là bias phải được cộng trước khi dịch thang, nếu bias được cộng ở sai miền, giá trị tiền kích hoạt bị lệch có hệ thống và có thể làm thay đổi lớp ArgMax.

$$h_{j,q16} = \max(0, \text{ACC}_j \ggg 12) \quad (2.30)$$

Bảng 2.3. Quy ước lượng tử hóa và xuất tham số MLP.

Hệ số	Định dạng	Python	RTL
MFCC input	Q16.16	<code>round(mfcc_real · 2¹⁶)</code>	Thứ tự flatten phải giống <code>mfcc_feature_buffer</code>
W1, W2	Q4.12	<code>round(weight_real · 2¹²)</code>	Lưu two's complement trong ROM

Hệ số	Định dạng	Python	RTL
B1, B2	Q28	<code>round(bias_real · 2²⁸)</code>	Cộng trực tiếp vào accumulator MAC
Product X×W	Q28 phần lẻ	Mô phỏng bit-accurate trong Python	Dùng accumulator đủ rộng để tránh overflow
Hidden activation	Dịch RTL	Áp dụng ReLU sau khi scale	Giữ signed arithmetic shift
Output logit	Số nguyên	Không cần softmax	ArgMax trên 20 logit

Bảng 2.3 cho thấy lượng tử hóa không chỉ là nhân mọi giá trị với một hệ số cố định. Các đối tượng khác nhau nằm ở các miền scale khác nhau, đặc biệt là bias. Trong các thử nghiệm ban đầu, chỉ cần bias sai thang là toàn bộ logit tăng ra có thể bị lệch đáng kể mặc dù phép nhân trọng số vẫn đúng. Vì vậy, đồng bộ bias Q28 là điều kiện bắt buộc để kết quả RTL bám sát mô hình fixed-point trên Python.

2.3.3. Xuất tham số và yêu cầu nhất quán Python-fixed-point-RTL

Sau khi chọn checkpoint tốt nhất, các ma trận W1, W2 và các vector bias B1, B2 được chuyển từ miền số thực sang số nguyên fixed-point và xuất thành các file hệ số dùng cho RTL. Đây là bước liên kết trực tiếp giữa mô hình huấn luyện trên Python và phần cứng, nên mọi sai lệch về thứ tự lưu trữ, độ rộng bit hoặc quy tắc biểu diễn signed đều có thể làm kết quả phần cứng khác với mô hình tham chiếu.

Các yêu cầu nhất quán bắt buộc gồm, thứ tự flatten của ma trận đặc trưng 98 x 13 trong Python phải giống thứ tự địa chỉ của `mfcc_feature_buffer`, thứ tự trọng số trong W1 và W2 phải giống công thức tính địa chỉ trong `mlp_core`, giá trị âm phải được ghi theo bù hai với cùng độ rộng bit, và bias phải được đưa về đúng miền Q28 trước khi cộng vào accumulator. Nếu một trong các điều kiện này bị sai, thiết kế phần cứng có thể vẫn tạo `result_valid` đúng thời điểm nhưng `result_id` không đáng tin cậy.

2.4. Công trình nghiên cứu liên quan

2.4.1. Nghiên cứu KWS quốc tế

Chen và cộng sự [8] đề xuất DNN small-footprint cho KWS, đặt nền tảng cho hướng giảm chi phí mô hình thay vì dùng mô hình ASR (Automatic Speech Recognition). Công trình cho thấy một mạng truyền thẳng nhỏ có thể nhận diện từ khóa với độ trễ và bộ nhớ phù hợp thiết bị nhúng, nhưng chưa tập trung vào hiện thực RTL FPGA end-to-end.

Sainath và Parada [9] sử dụng CNN để khai thác cấu trúc cục bộ của đặc trưng thời gian-tần số. So với DNN, CNN chia sẻ trọng số và tận dụng tương quan lân cận, từ đó cải thiện độ chính xác với số tham số hợp lý. Tuy nhiên, convolution làm tăng yêu cầu bộ đệm feature map và tổ chức datapath phức tạp hơn MLP.

Arik và cộng sự [10] kết hợp convolution và recurrent layer để mô hình hóa đồng thời đặc trưng cục bộ và phụ thuộc thời gian. Kiến trúc có khả năng xử lý chuỗi tốt hơn nhưng chi phí phần cứng lớn hơn do trạng thái hồi tiếp và nhiều phép MAC.

Hello Edge của Zhang và cộng sự [11] khảo sát nhiều kiến trúc DNN, CNN và DS-CNN trên Google Speech Commands. Công trình là mốc quan trọng cho TinyML, nhưng kết quả accuracy không thể so trực tiếp với đề tài vì khác ngôn ngữ, dataset, số lớp và nền tảng triển khai.

Ở hướng accelerator, He và cộng sự [14] đề xuất kiến trúc cấu hình được cho KWS dựa trên mạng temporal efficient. Bae và cộng sự [13] sử dụng mạng nhị phân/tam phân depthwise-separable để giảm mạnh chi phí nhân. Các công trình này chứng minh lợi ích của lượng tử hóa cực thấp, đồng thời cho thấy sự đánh đổi giữa độ chính xác, tính linh hoạt và phạm vi pipeline được hiện thực.

Bảng 2.4. Một số công trình KWS quốc tế liên quan.

Công trình	Dataset	Nền tảng	Mô hình	Vai trò đối chiếu
Chen et al. [8]	English KWS	Software/embedded	DNN small-footprint	Baseline cho mạng truyền thẳng nhỏ

Công trình	Dataset	Nền tảng	Mô hình	Vai trò đối chiếu
Sainath & Parada [9]	English KWS	Software/embedded	CNN	Khai thác cấu trúc cục bộ thời gian-tần số
Arik et al. [10]	Noisy KWS	Software	CRNN	Mô hình hóa phụ thuộc thời gian, chi phí cao hơn
Hello Edge [11]	Google Speech Commands	MCU/TinyML	DNN/CNN/DS-CNN	Mức trade-off accuracy-memory-operations
He et al. [14]	Google Speech Commands	Zynq accelerator	Modified TENet	Accelerator nhỏ, phạm vi khác full pipeline
Bae et al. [13]	GSC, WUW + commands	ZCU104 FPGA	DS-BTNN	Nhị phân/tam phân để giảm DSP và bộ nhớ
Đề tài	20 nhãn tiếng Việt	KV260 FPGA	MFCC + MLP 1274-32-20	Full PCM-MFCC-context-classifier theo block

Bảng 2.4 không được dùng để xếp hạng accuracy trực tiếp. Độ chính xác phụ thuộc vào dataset, số từ khóa/nhãn, cách chia train/test và điều kiện nhiễu. Các công trình quốc tế chủ yếu được dùng để đối chiếu lựa chọn mô hình, kiểu lượng tử hóa và mức tài nguyên.

2.4.2. Các nghiên cứu tiếng Việt trên phần cứng

Tác giả Phan Đình Duy và cộng sự [15] triển khai nhận dạng lệnh robot tiếng Việt trên FPGA kết hợp Nios II. Hệ thống chứng minh khả năng xử lý tiếng Việt trên nền tảng nhúng, nhưng bài toán lệnh robot, đặc trưng và classifier khác với KWS 20 từ khóa/nhãn của đề tài.

Tác giả Dao và cộng sự [16] hiện thực khối trích xuất MFCC trên FPGA. Công trình gần với front-end của đề tài và là nguồn đối chiếu về tổ chức các phép FFT, Mel và DCT, nhưng không bao gồm classifier end-to-end và không đánh giá miền từ nhạy cảm.

Tác giả Pham và cộng sự [17] sử dụng ANN cho nhận dạng từ rời rạc trên nền tảng ASIC/SOPC. Công trình có số lớp gần hơn với đề tài, song khác công nghệ chế tạo, dataset và kiến trúc front-end. Nguyen và cộng sự [18] tập trung vào lõi MFCC

ASIC hiệu năng cao, cho thấy front-end có thể được tăng tốc mạnh nhưng sử dụng biểu diễn và mục tiêu thiết kế khác.

Bảng 2.5. Một số công trình nhận dạng tiếng Việt trên phần cứng.

Công trình	Bài toán	Nền tảng	Mô hình	Khác biệt so với đề tài
Phan Đình Duy et al. [15]	Lệnh robot tiếng Việt	DE2 + Nios II	FFT/Mel/VQ	Hệ lệnh và classifier khác, không phải KV260
Dao et al. [16]	Front-end nhận dạng tiếng nói	FPGA	MFCC IP	Không gồm context buffer và classifier KWS
Pham et al. [17]	Từ rời rạc tiếng Việt	ASIC/SOP C	MFCC + ANN	Công nghệ và dataset khác
Nguyen et al. [18]	Trích xuất đặc trưng	ASIC 130 nm	MFCC	Tập trung hiệu năng front- end
Đề tài	20 từ khóa nhảy cảm tiếng Việt	KV260	MFCC + MLP	Pipeline PCM-to-result và kiểm thử bằng file PCM

2.4.3. Khoảng trống nghiên cứu và định vị đề tài

Từ các công trình liên quan, khoảng trống nghiên cứu của đề tài được xác định ở ba điểm chính. Khoảng trống thứ nhất nằm ở bài toán và dữ liệu tiếng Việt. Phần lớn các nghiên cứu KWS quốc tế sử dụng tiếng Anh hoặc Google Speech Commands, trong khi tiếng Việt có thanh điệu, cách phát âm ngắn, sự biến thiên cao độ và đặc điểm âm vị khác với tiếng Anh. Ngoài ra, miền ứng dụng của khóa luận là nhận diện các từ khóa/nhãn nhảy cảm trong môi trường lớp học, không phải tập lệnh điều khiển thông dụng. Vì vậy, kết quả trên các tập dữ liệu tiếng Anh không thể dùng trực tiếp để kết luận về bài toán tiếng Việt của đề tài.

Khoảng trống thứ hai nằm ở mức độ hoàn chỉnh của hệ thống phần cứng. Một số nghiên cứu tiếng Việt trên phần cứng mới tập trung vào khối trích xuất đặc trưng MFCC, hệ lệnh nhỏ hoặc nền tảng xử lý khác, trong khi khóa luận hướng tới một pipeline end-to-end trên FPGA. Thiết kế không dừng ở việc tạo đặc trưng MFCC hoặc mô phỏng bộ phân loại riêng lẻ, mà hiện thực toàn bộ luồng từ mẫu PCM đầu

vào, tiền xử lý, FFT/Power, Mel, Log, DCT/MFCC, Feature Buffer 98 frame, MLP Core, đến result_id và result_valid.

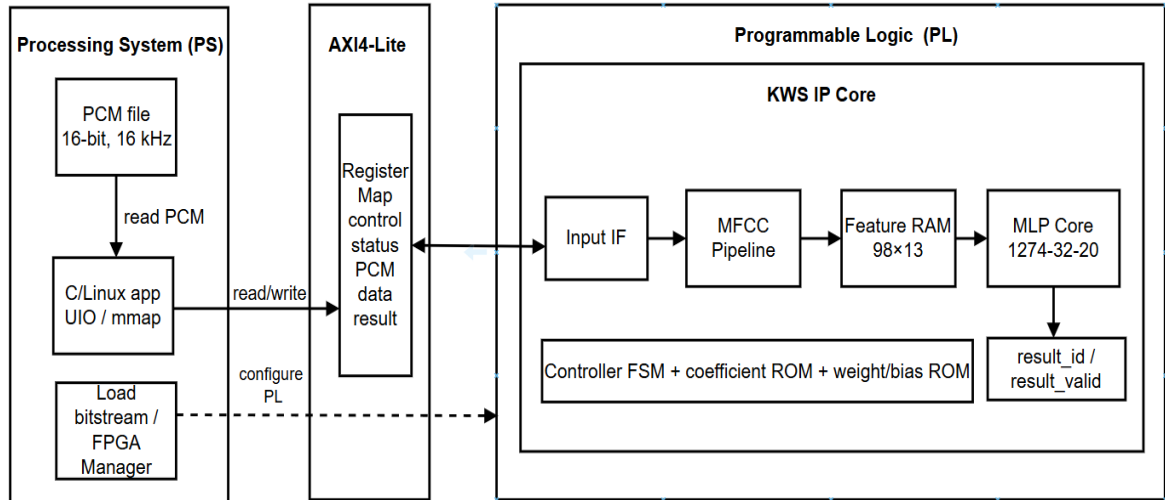
Khoảng trống thứ ba nằm ở việc chuyển đổi và hiện thực fixed-point cho toàn bộ chuỗi xử lý KWS. Một mô hình Python floating-point có thể đạt độ chính xác tốt, nhưng điều đó chưa chứng minh được mô hình có thể triển khai hiệu quả trên FPGA. Khi chuyển sang fixed-point/RTL, các yếu tố như sai số lượng tử hóa, cắt bit, làm tròn, thang đo bias, độ rộng accumulator, thứ tự lưu trữ tham số và thứ tự flatten vector đặc trưng đều có thể làm thay đổi logit đầu ra. Vì vậy, đề tài không chỉ dừng ở mô hình số thực mà tập trung chuyển đổi toàn bộ pipeline sang fixed-point, đồng bộ thang đo giữa Python và RTL, đồng thời bảo đảm kết quả phân cứng vẫn bám sát mô hình tham chiếu.

Từ ba khoảng trống trên, khóa luận định vị đóng góp chính ở ba hướng. Thứ nhất, đề tài xây dựng và tổ chức dataset 20 từ khóa/nhãn nhảy cảm tiếng Việt phục vụ bài toán KWS trong bối cảnh lớp học. Thứ hai, đề tài thiết kế pipeline end-to-end PCM-MFCC-Feature Buffer-MLP trên FPGA, có giao tiếp PS-PL và có thể kiểm thử bằng file PCM thật. Thứ ba, đề tài hiện thực toàn bộ hệ thống theo số học fixed-point, bao gồm lựa chọn Q-format, lượng tử hóa hệ số MFCC/MLP, đồng bộ scale bias, tổ chức ROM/RAM hệ số và triển khai datapath RTL phù hợp với tài nguyên FPGA.

Trong định vị này, lựa chọn MLP 1274-32-20 là một quyết định thiết kế nhằm phục vụ mục tiêu end-to-end fixed-point, không phải tuyên bố rằng MLP luôn vượt trội hơn CNN/RNN. MLP được chọn vì cấu trúc tính toán đơn giản, dễ lượng tử hóa, dễ lưu trữ tham số trong ROM, dễ triển khai bằng MAC tuần tự và phù hợp với cách ghép 98 frame MFCC thành vector ngữ cảnh. Nhờ đó, hệ thống giữ được khả năng quan sát diễn biến thời gian của từ khóa mà không phải đưa thêm lớp tích chập hoặc hồi tiếp phức tạp vào phân cứng.

CHƯƠNG 3. THIẾT KẾ VÀ HIỆN THỰC

3.1. Mô tả kiến trúc tổng quan



Hình 3.1. Kiến trúc tổng quan của hệ thống nhận diện từ khóa.

Hình 3.1 trình bày kiến trúc tổng quan của hệ thống nhận diện từ khóa khi triển khai trên nền tảng KV260. Dữ liệu PCM 16-bit, 16 kHz được chương trình C/Linux trên Processing System đọc từ file, sau đó truyền tới KWS IP trong Programmable Logic thông qua giao tiếp thanh ghi AXI4-Lite. Trong PL, KWS IP thực hiện chuỗi xử lý từ nhận mẫu PCM, trích xuất đặc trưng MFCC, tích lũy Feature RAM 98 frame $\times$ 13 hệ số, suy luận bằng MLP Core, đến xuất result_id và result_valid.

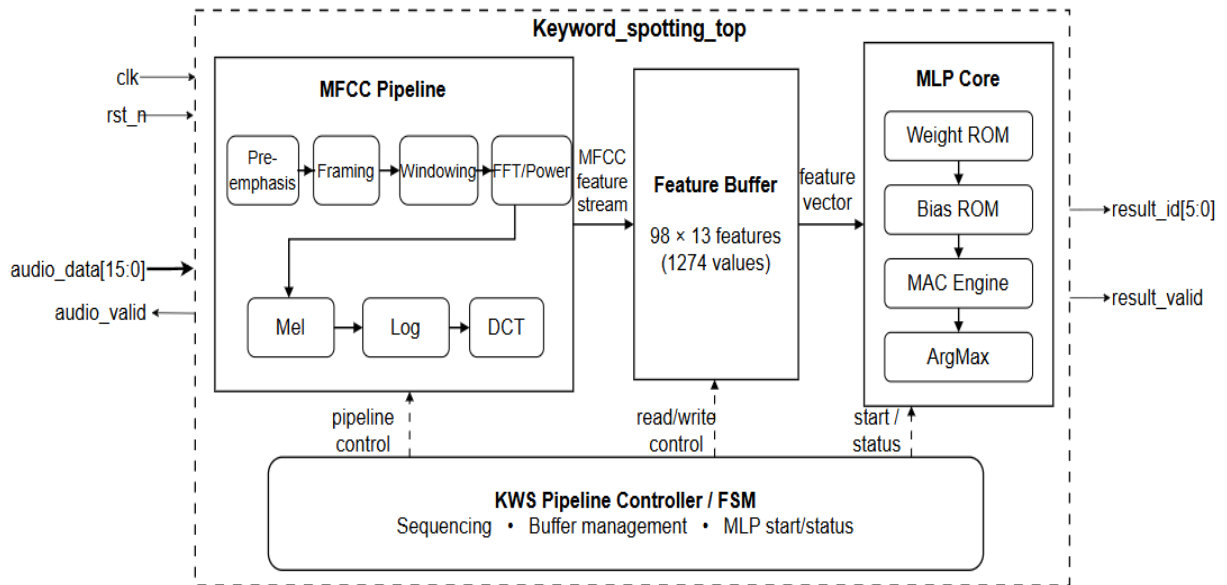
Trong kiến trúc này, PS đảm nhiệm các công việc điều khiển như nạp bitstream, khởi tạo UIO/mmap, gửi dữ liệu đầu vào và thu thập log kết quả. PL đảm nhiệm phần tính toán cố định của bộ tăng tốc. Cách phân chia này giúp phần mềm giữ vai trò điều phối linh hoạt, còn các phép xử lý lặp lại được chuyển thành datapath phần cứng có độ trễ và tài nguyên quan sát được.

Thiết kế phần cứng nhận diện từ khóa nhạy cảm tiếng Việt được tổ chức theo kiến trúc phân tầng. Chuỗi PCM 16-bit được chuyển thành đặc trưng MFCC, lưu thành vector ngữ cảnh và đưa vào MLP để phân loại. Ba nhóm khối chính gồm:

1. Khối tiền xử lý và trích xuất đặc trưng, MFCC Pipeline, biến đổi dữ liệu PCM 16-bit do PS/Linux hoặc testbench cung cấp thành 13 hệ số MFCC cho mỗi

- frame. Khối này chuyển tín hiệu từ miền thời gian sang biểu diễn phổ ngắn hạn, giảm chiều dữ liệu và giữ lại các đặc trưng ý nghĩa đối với nhận dạng tiếng nói.
2. Khối lưu trữ đệm đặc trưng, Feature RAM Buffer, tích lũy liên tiếp 98 frame MFCC. Do một từ khóa thường trải qua nhiều frame, bộ đệm này tạo vector $98 \times 13 = 1274$ phần tử để MLP có đủ ngữ cảnh thời gian khi suy luận.
 3. Khối phân loại mạng nơ-ron, MLP Core, thực hiện suy luận mạng 1274-32-20. Trọng số và bias sau huấn luyện được lưu trong ROM nội bộ, còn phép tính được tổ chức theo dạng nhân-cộng tích lũy, MAC, để tái sử dụng tài nguyên DSP và giảm mức độ song song không cần thiết.

Top-level cuối của hệ thống là module `keyword_spotting_top`. Bên trong module này gồm các khối chính: `mfcc_system_top_full`, `kws_pipeline_controller`, `mfcc_feature_buffer` và `mlp_core`. Pipeline MFCC chứa các module `pre_emphasis`, `framing`, `windowing`, `fft512_power_axis`, `mel_filterbank`, `log_block` và MFCC.



Hình 3.2. Kiến trúc top-level KWS IP Core.

Hình 3.2 mô tả cấu trúc top-level của KWS IP Core trong hệ thống FPGA. Dữ liệu PCM được đưa qua giao tiếp đầu vào, xử lý bởi pipeline MFCC, lưu vào Feature Buffer và phân loại bằng MLP Core. KWS Pipeline Controller điều phối trình tự thu dữ liệu, ghi bộ đệm, kích hoạt MLP và chốt kết quả. Các thanh ghi AXI4-Lite cho

phép phần mềm trên PS thực hiện soft reset, kiểm tra trạng thái ready/valid, gửi mẫu PCM và đọc result_id sau khi result_valid được kích hoạt.

Bảng 3.1. Thông số tổng quát của hệ thống.

Tham số	Ký hiệu	Giá trị	Đơn vị	Chi tiết
Tần số lấy mẫu	f _s	16.000	Hz	Chuẩn âm thanh mono 16 kHz
Độ dài khung	N	512	mẫu	Tương đương 32 ms tại f _s = 16 kHz
Bước nhảy	H	160	mẫu	Tương đương 10 ms, độ chồng lấn 352 mẫu
Kích thước FFT	N _{FFT}	512	điểm	Sử dụng 257 bin không âm, 0..256
Số bộ lọc Mel	N _{MELS}	40	dải	Dải tần từ 0 đến 8 kHz
Số hệ số MFCC	N _{CEPS}	13	hệ số	Giữ C0..C12
Số frame tích lũy	N _{FRAMES}	98	frame	Ngữ cảnh xấp xỉ 1 giây
Kích thước Input MLP	N _{IN}	1274	đặc trưng	98 × 13 = 1274
Định dạng đặc trưng	Q	Q16.16	fixed-point	32-bit signed cho MFCC/Log-Mel

3.2. Thiết kế chi tiết các khối chức năng

Mỗi khối trong mục này được trình bày theo vai trò trong pipeline, giao tiếp tín hiệu chính, cách tổ chức datapath và điểm cần kiểm chứng. Cách trình bày này giúp đối chiếu trực tiếp giữa mô hình Python, dữ liệu golden và thiết kế phần cứng.

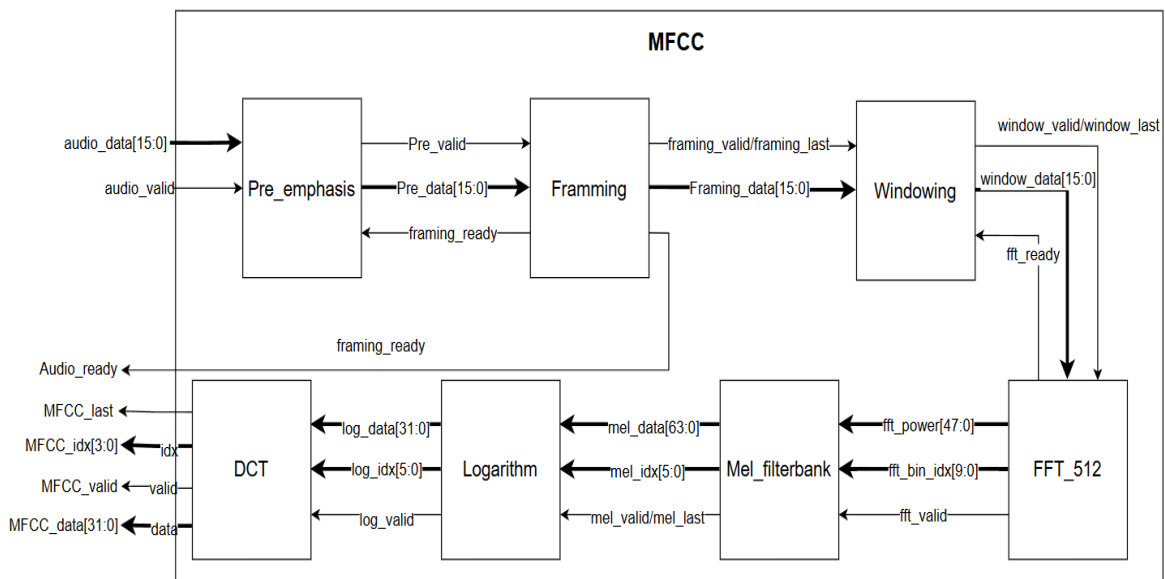
Bảng 3.2. Danh sách module RTL chính.

Module	Chức năng	Dữ liệu ra
pre_emphasis	Bộ lọc tiền nhấn bậc một	PCM sau tiền nhấn
Framing	Tạo frame 512 mẫu, hop 160 bằng RAM vòng	Stream frame 512 mẫu
windowing	Nhân hệ số Hamming Q1.15	Frame đã cửa sổ hóa
fft512_power_axis	FFT 512 điểm và phổ công suất	257 bin power
mel_filterbank	Tích lũy phổ thành 40 năng lượng Mel	40 giá trị Mel

log_block	Xấp xỉ log2 bằng LOD và LUT 256 phần tử	Log-Mel Q16.16
MFCC	DCT-II tạo hệ số MFCC	13 hệ số MFCC
mfcc_feature_buffer	Lưu 98 frame $\times$ 13 hệ số	Vector 1274
mlp_core	Suy luận MLP 1274-32-20	result_id

Bảng 3.2 cho thấy thiết kế được chia thành các module độc lập để dễ kiểm chứng và tích hợp. Để đáp ứng yêu cầu xử lý ổn định, pipeline MFCC được thiết kế theo mô hình xử lý dòng. Điểm mấu chốt là cơ chế bắt tay valid/ready giữa các module. Một dữ liệu chỉ được truyền khi khối nguồn báo valid và khối nhận báo ready, nhờ đó các khối có độ trễ khác nhau vẫn có thể phối hợp mà không làm mất mẫu.

- Tính song song mức nhiệm vụ: mỗi module trong pipeline, như Pre-emphasis, Windowing hoặc FFT, được thiết kế như một trạm xử lý độc lập. Khi một mẫu hoặc một frame rời khỏi khối trước, khối sau có thể tiếp nhận theo điều kiện handshake, không cần chờ toàn bộ file âm thanh hoàn thành.
- Quản lý thông lượng dữ liệu: một số khối như FFT, Mel Filterbank và DCT có độ trễ nội tại lớn hơn các khối xử lý từng mẫu. Vì vậy, các FSM nội bộ sử dụng valid, ready và last để xác định đúng ranh giới frame, bảo đảm dữ liệu không bị mất, lặp hoặc chồng lấn sai thứ tự.



Hình 3.3. Sơ đồ kết nối pipeline MFCC trong thiết kế RTL.

Thiết kế sử dụng clock mục tiêu 100 MHz, tương ứng chu kỳ 10 ns. Bảng 3.3 tổng hợp chu kỳ clock mục tiêu và độ trễ đo được của các khối chính trong pipeline. Các số chu kỳ này được lấy từ mô phỏng khi mẫu PCM được cấp theo handshake của testbench hoặc driver, do đó phản ánh thời gian xử lý của thiết kế phần cứng, không phải thời lượng phát âm vật lý của file âm thanh.

Bảng 3.3. Chu kỳ clock và độ trễ của các khối RTL.

Khối xử lý	Đơn vị	Số chu kỳ clock	Độ trễ tại 100 MHz	Ghi chú
Pre-emphasis	1 mẫu PCM	1	10 ns	Xử lý streaming, thông lượng 1 mẫu/chu kỳ
Framing	Frame đầu tiên	1024	10,24 us	512 chu kỳ nạp đủ frame + 512 chu kỳ xuất frame
Framing	Frame tiếp theo	672	6,72 us	160 chu kỳ nạp mẫu mới + 512 chu kỳ xuất frame
Windowing	1 frame 512 mẫu	512	5,12 us	Nhân 512 mẫu với hệ số Hamming
FFT/Power	1 frame	LFFT + 257	$(LFFT + 257) \times 10$ ns	LFFT là độ trễ nội bộ của FFT core, wrapper xuất 257 bin power
Mel Filterbank	257 bin $\rightarrow$ 40 Mel	297	2,97 us	257 chu kỳ tích lũy + 40 chu kỳ xuất
Log-Mel	40 dải Mel	40	0,40 us	1 giá trị Mel/chu kỳ, pipeline 1 stage
DCT/MFCC C	40 Log-Mel $\rightarrow$ 13 MFCC	560	5,60 us	40 chu kỳ nhận dữ liệu + 13×40 chu kỳ MAC
Feature Buffer	98 frame $\times$ 13 MFCC	1274 lần ghi hợp lệ	Phụ thuộc tốc độ MFCC đầu vào	mlp_start bật khi nhận hệ số MFCC cuối của frame thứ 98
MLP Core	Vector 1274 $\rightarrow$ result_id	122.945	1,229 ms	MAC tuần tự cho MLP 1274-32-20
Top-level	1 file PCM $\rightarrow$ result_valid	khoảng 208.340	khoảng 2,083 ms	Đo từ mô phỏng top-level/batch log

Các khối đầu pipeline như Pre-emphasis và Windowing có độ trễ nhỏ vì chỉ xử lý theo từng mẫu. Các khối theo frame như FFT/Power, Mel, Log và DCT cần thêm chu kỳ để nhận đủ frame, quét các bin tần số hoặc xuất tuần tự các hệ số. MLP Core có số chu kỳ lớn nhất vì phải thực hiện hai tầng MAC theo thời gian, đọc trọng số/bias từ ROM, cộng bias, áp dụng ReLU, tính 20 logit và chọn lớp bằng ArgMax.

Vì vậy, Bảng 3.3 không chỉ thể hiện độ trễ từng khối mà còn cho thấy nút chi phối latency của toàn hệ thống nằm ở tầng suy luận MLP và các khối xử lý theo frame.

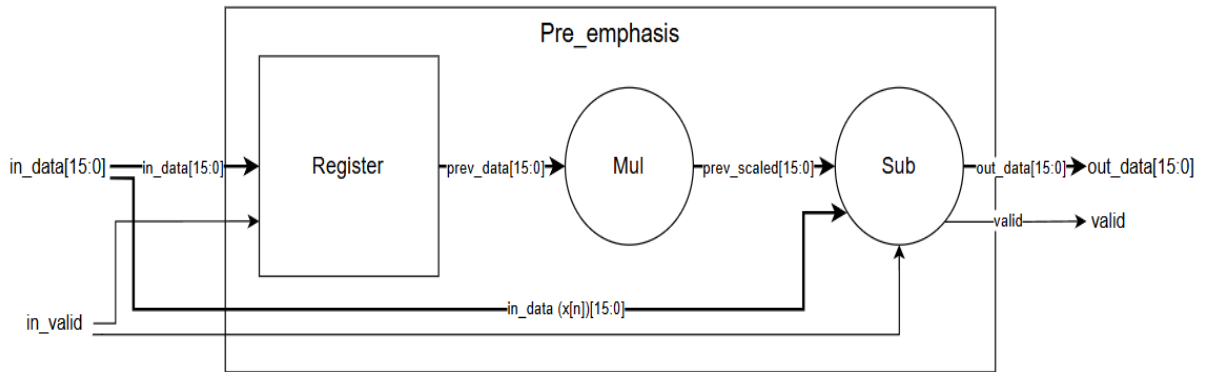
3.2.1. Khối Tiền nhấn (Pre-emphasis)

Khối Pre-emphasis là tầng xử lý đầu tiên của pipeline MFCC. Vai trò của khối là nhận từng mẫu PCM 16-bit, giữ lại mẫu trước đó và thực hiện bộ lọc thông cao bậc một để làm nổi bật tương đối các thành phần biến thiên nhanh trong tiếng nói. Đây là bước chuẩn bị trước khi dữ liệu được chia frame và đưa vào phân tích phổ.

Datapath của khối gồm thanh ghi lưu mẫu trước, bộ nhân hệ số α Q1.15, mạch làm tròn và bộ trừ. Mẫu đầu tiên sau reset sử dụng $x[-1] = 0$. Khi in_valid và in_ready cùng tích cực, mẫu hiện tại được xử lý ở đầu ra và đồng thời được chốt làm $prev_sample$ cho chu kỳ kế tiếp.

$$y[n] = x[n] - \alpha x[n-1], \quad \alpha = 0,97$$

Trong thiết kế phần cứng, hệ số α được lượng tử hóa theo định dạng Q1.15, tương ứng $COEFF = 31785$. Tích $\alpha x[n-1]$ tạo ra kết quả rộng hơn 16 bit, sau đó được làm tròn, dịch phải 15 bit và đưa về miền dữ liệu phù hợp trước khi thực hiện phép trừ với $x[n]$. Python bit-accurate phải dùng cùng quy tắc làm tròn và giới hạn bit để kết quả đối sánh được với thiết kế RTL.



Hình 3.4. Sơ đồ khối module *pre_emphasis*.

Để tối ưu tài nguyên trên KV260, khối chỉ cần một thanh ghi trễ, một bộ nhân và một datapath tổ hợp ngắn. Vì đầu ra vẫn là chuỗi mẫu theo thời gian, giao tiếp $valid/ready$ của khối được giữ đơn giản và có thể nối trực tiếp sang khối Framing.

Kết quả sau Pre-emphasis không nhằm tăng biên độ toàn bộ tín hiệu, mà thay đổi phân bố phổ trước khi thực hiện FFT. Điều này giúp các phụ âm, biên chuyển tiếp và thành phần cao tần có đóng góp rõ hơn trong các bước trích xuất đặc trưng tiếp theo.

Bảng 3.4. Giao diện tín hiệu module pre_emphasis.

Port name	Dir	Width	Fixed-point	Mô tả hành vi
clk	in	1	-	Clock hệ thống
rst_n	in	1	-	Reset active-low
in_valid	in	1	-	=1 → nhận mẫu vào, tính pre-emphasis
in_data	in	DATA_W (16)	signed int / (dùng hệ số Q15)	Mẫu âm thanh đầu vào (signed PCM)
in_ready	out	1	-	Luôn 1 (always ready)
out_valid	out	1	-	=1 khi có output hợp lệ (theo in_valid)
out_data	out	DATA_W (16)	signed int	$out = x[n] - \text{sat_round}(\text{prev} * \text{COEFF} \gg 15)$, gán về DATA_W bit

3.2.2. Khối Chia khung (Framing Control)

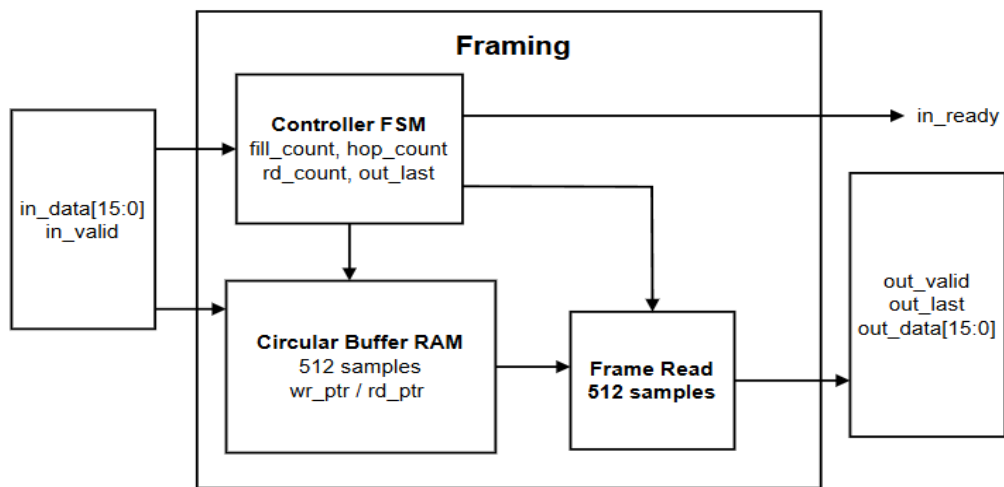
Khối Framing chuyển luồng mẫu PCM sau tiền nhân thành các frame 512 mẫu để phục vụ FFT. Vì hai frame liên tiếp cách nhau 160 mẫu, hệ thống phải tái sử dụng 352 mẫu chồng lấn. Nếu dùng thanh ghi dịch toàn bộ frame, chi phí phần cứng và hoạt động dịch dữ liệu sẽ lớn. Do đó, thiết kế sử dụng RAM vòng để lưu cục bộ 512 mẫu gần nhất.

Circular buffer có độ sâu 512. Trong pha nhận mẫu, wr_ptr tăng modulo 512 và ghi mẫu mới vào RAM khi in_valid hợp lệ. Khi đủ dữ liệu cho frame đầu tiên, hoặc khi đã nhận thêm 160 mẫu mới cho các frame tiếp theo, FSM chuyển sang pha xuất frame.

Trong pha xuất, rd_ptr được khởi tạo theo vị trí bắt đầu của frame cần đọc và quét tuần tự 512 vị trí trong RAM vòng. Dữ liệu được đưa ra out_data theo từng chu

kỳ, out_valid báo dữ liệu hợp lệ và out_last được kích hoạt tại mẫu cuối cùng của frame. Trong thời gian xuất frame, in_ready được hạ để tránh ghi đè khi module đang đọc dữ liệu.

Cách tổ chức này giữ đúng cơ chế overlap của thuật toán nhưng không cần sao chép hoặc dịch toàn bộ 512 mẫu. Đây là điểm quan trọng khi hiện thực trên FPGA vì bộ nhớ được tái sử dụng hiệu quả, còn ranh giới frame vẫn được xác định rõ bằng out_last trước khi chuyển sang Windowing.



Hình 3.5. Sơ đồ khối module framing.

Bảng 3.5. Giao diện tín hiệu module framing.

Port name	Dir	Width	Mô tả hành vi
clk	in	1	Xung nhịp hệ thống
rst_n	in	1	Tín hiệu Reset tích cực mức thấp
in_valid	in	1	=1: Báo hiệu có mẫu Audio hợp lệ từ khối Pre-emphasis
in_data	in	16	Dữ liệu mẫu Audio đầu vào (Signed PCM)
out_valid	out	1	=1: Báo hiệu dữ liệu khung đang được đẩy ra
out_data	out	16	Dữ liệu mẫu thuộc khung hiện tại
out_last	out	1	=1: Tại mẫu cuối cùng (thứ 512) của khung, dùng để đồng bộ FFT
in_ready	out	1	=1 khi module không bận xuất frame, dùng để backpressure input

3.2.3. Khối Cửa sổ hóa (Windowing)

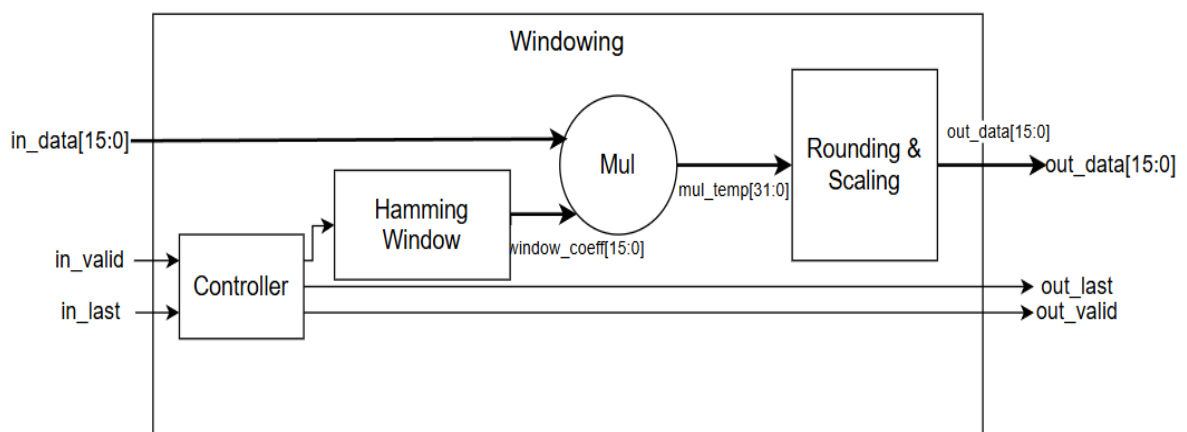
Khối Windowing thực hiện phép nhân từng mẫu của frame với hệ số cửa sổ Hamming tương ứng nhằm giảm rò rỉ phổ trước khi đưa dữ liệu vào FFT. Đây là phép nhân frame 512 mẫu và bảng hệ số cửa sổ đã được chuẩn bị trước.

512 hệ số Hamming được tính trước bằng Python, lượng tử hóa theo Q1.15 và lưu trong ROM. Khi một frame được truyền vào, bộ đếm `rom_addr` chạy từ 0 đến 511 để đọc đúng hệ số ứng với từng mẫu. Giá trị thực của cửa sổ Hamming không âm và có đỉnh xấp xỉ 1,0, tương ứng gần 16'h7FFF trong Q1.15.

Mỗi mẫu đầu vào signed 16-bit được nhân với hệ số Q1.15, tạo kết quả 32-bit. Kết quả này được cộng hằng làm tròn, sau đó dịch phải 15 bit để đưa về độ rộng 16 bit theo datapath của khối. Cách làm này giữ cùng thang dữ liệu cho đầu ra Windowing và đầu vào FFT.

Về mặt điều khiển, `rom_addr` được đồng bộ với `in_valid`, còn `out_valid` và `out_last` được truyền theo dòng dữ liệu sau khi nhân hệ số. Kết quả của khối là một frame đã được làm mềm ở hai biên, giúp phổ FFT tập trung hơn và giảm lan năng lượng sang các bin lân cận.

$$\text{out_data} = (\text{mult_temp} + 16384) \ggg 15$$



Hình 3.6. Sơ đồ khối module windowing.

Bảng 3.6. Giao diện tín hiệu module windowing.

Port name	Dir	Width	Fixed-point	Mô tả hành vi
clk	in	1	-	Clock

Port name	Dir	Width	Fixed-point	Mô tả hành vi
rst_n	in	1	-	Reset active-low
in_valid	in	1	-	=1 → nhân sample với hệ số window
in_data	in	DATA_W (16)	signed int	Sample thuộc frame
in_last	in	1	-	=1 tại sample cuối frame (để reset index window)
out_valid	out	1	-	=1 khi output hợp lệ (theo in_valid)
out_data	out	DATA_W (16)	signed, sau nhân Q1.15	$out = (in_data \times win_coeff_Q15 + 2^{14}) \ggg 15$
out_last	out	1	-	Pass-through từ in_last

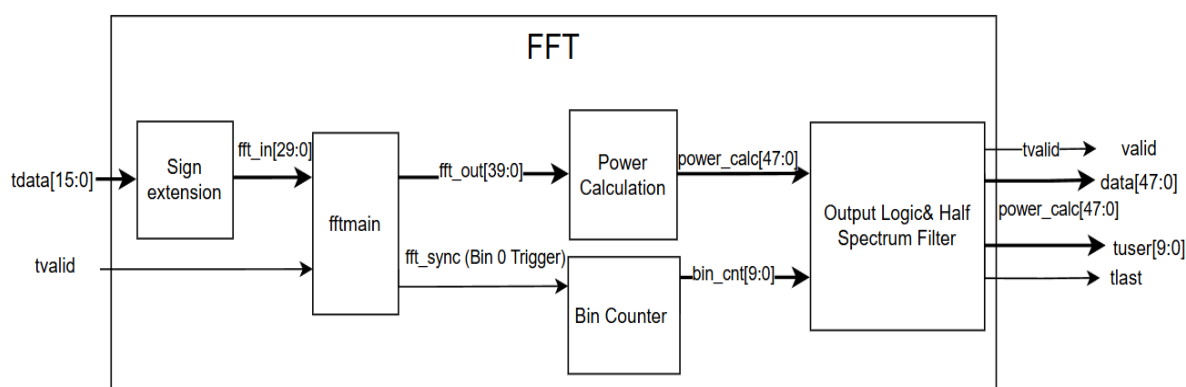
3.2.4. Khối Biến đổi Fourier và Phổ năng lượng (FFT & Power Spectrum)

Khối FFT/Power chuyển từng frame sau Windowing từ miền thời gian sang miền tần số. Đầu vào của khối là mẫu signed 16-bit sau cửa sổ hóa, được đưa vào phần thực của FFT, phần ảo được gán bằng 0. Kết quả FFT là số phức, sau đó được biến đổi thành phổ công suất để làm đầu vào cho Mel Filterbank.

Thiết kế tích hợp wrapper giao tiếp frame với lõi FFT Open source [33] và mạch tính power spectrum. Wrapper đóng gói dữ liệu theo AXI-Stream của FFT IP, theo dõi tín hiệu valid/last và chỉ giữ 257 bin đầu tiên từ $k = 0$ đến $k = 256$. Việc chỉ giữ nửa phổ không âm vì đầu vào là tín hiệu thực, nửa phổ còn lại đối xứng.

Lõi FFT sử dụng cấu hình 512 điểm. Đầu ra gồm phần thực và phần ảo 20-bit theo scale nội bộ của lõi fftmain. Với mỗi bin hợp lệ, khối Power Spectrum tính $P[k] = \text{Re}[k]^2 + \text{Im}[k]^2$. Khi FFT_W = 20, bình phương mỗi thành phần cần khoảng 40 bit, phép cộng cần thêm bit bảo vệ, vì vậy kết quả được mở rộng hoặc cắt về POW_W = 48 bit trước khi truyền sang Mel Filterbank.

Tín hiệu m_axis_tlast được kích hoạt tại bin 256 để báo kết thúc nửa phổ không âm. Nhờ vậy, khối Mel phía sau nhận chính xác 257 giá trị công suất cho mỗi frame và có thể khởi tạo lại accumulator cho frame kế tiếp.



Hình 3.7. Sơ đồ khối module *fft512_power_axis*.

Bảng 3.7. Giao diện tín hiệu module *fft512_power_axis*.

Port name	Dir	Width	Fixed-point	Mô tả hành vi
clk	in	1	-	Clock
rst	in	1	-	Reset active-high (lưu ý khác rst_n)
s_axis_tvalid	in	1	-	Valid input sample vào FFT
s_axis_tready	out	1	-	Luôn 1 (always ready)
s_axis_tdata	in	IN_W (16)	signed int	Sample time-domain sau window
s_axis_tlast	in	1	-	Marker sample cuối frame FFT (512)
m_axis_tvalid	out	1	-	=1 khi power-bin hợp lệ
m_axis_tdata	out	POW_W (48)	unsigned magnitude ²	power = re ² + im ²
m_axis_tuser	out	10	-	Bin index 0..256
m_axis_tlast	out	1	-	=1 tại bin cuối (256)

3.2.5. Khối Bộ lọc Mel (Mel Filterbank)

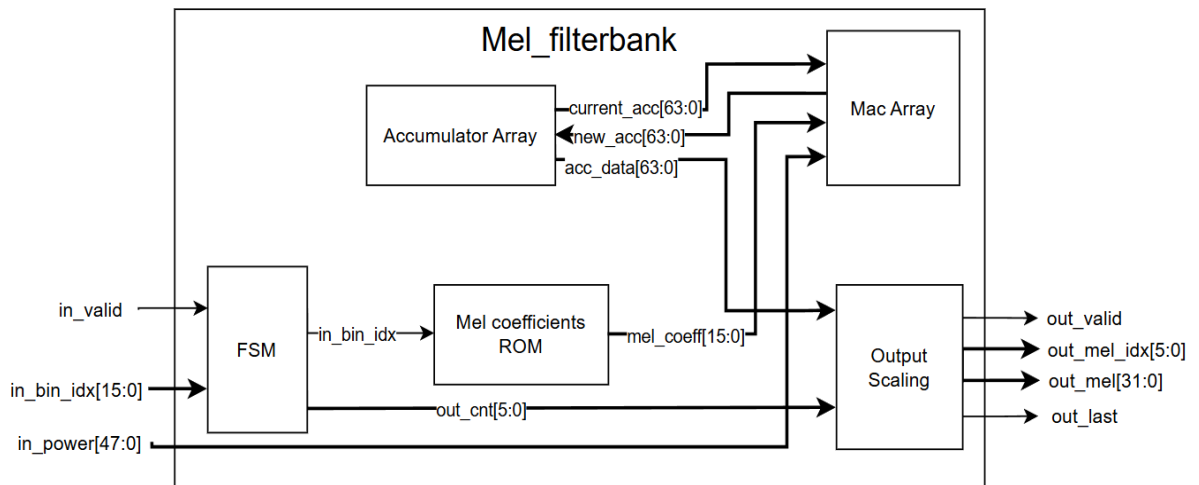
Khối Mel Filterbank ánh xạ phổ công suất 257 bin từ thang tần số tuyến tính sang 40 băng Mel. Mục tiêu của khối là giảm số chiều dữ liệu và gom năng lượng phổ theo cách gần hơn với cảm nhận thính giác, trước khi đưa sang phép logarit.

Hệ số của 40 bộ lọc tam giác được tính trước bằng Python, lượng tử hóa Q1.15 và ghi vào file *mel_coeff.hex*. Thiết kế phần cứng nạp toàn bộ ma trận này vào ROM để bảo đảm thứ tự hệ số khớp với mô hình golden. Địa chỉ tuyến tính của hệ số được tính theo $\text{mel_idx} \times \text{N_BINS} + \text{bin_idx}$.

Với mỗi bin FFT hợp lệ, phần cứng nhân in_power với hệ số Mel tương ứng và cộng vào 40 accumulator 64-bit. Thiết kế hiện tại cập nhật các băng Mel theo ma trận hệ số đầy đủ. Cách này đơn giản hóa điều khiển và giúp việc đối chiếu với Python dễ hơn, đổi lại số phép nhân theo mỗi bin tăng lên.

FSM của khối gồm ba pha chính. Ở S_IDLE, module chờ bin đầu tiên của frame và xóa các accumulator. Ở S_ACC, module cập nhật accumulator khi bin FFT hợp lệ. Ở S_OUTPUT, sau khi nhận đủ 257 bin, module xuất tuần tự 40 giá trị Mel và dịch phải 15 bit để bù scale Q1.15 của hệ số lọc. Đầu ra của khối là vector năng lượng Mel dùng cho Log-Mel.

$$acc[i] = acc[i] + in_power \times mel_coeff[i \times 257 + in_bin_idx]$$



Hình 3.8. Sơ đồ khối module *mel_filterbank*.

Bảng 3.8. Giao diện tín hiệu module *mel_filterbank*.

Port name	Dir	Width	Fixed-point	Mô tả hành vi
clk	in	1	-	Clock
rst_n	in	1	-	Reset active-low
in_valid	in	1	-	=1 khi bin power vào hợp lệ
in_bin_idx	in	16	-	Bin index 0..256
in_power	in	IN_W (48)	power magnitude^2	Power spectrum bin
out_valid	out	1	-	=1 khi mel output hợp lệ
out_mel	out	OUT_W (64 tại top-level)	signed, acc >>> 15	Mel energy tại out_mel_idx

Port name	Dir	Width	Fixed-point	Mô tả hành vi
out_mel_idx	out	6	-	Mel index 0..39
out_last	out	1	-	=1 tại mel cuối (39)

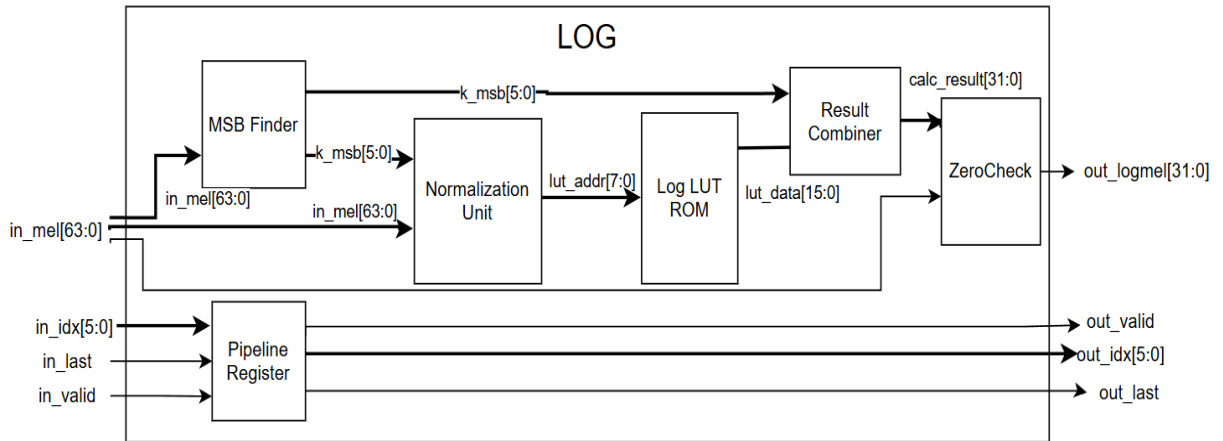
3.2.6. Khối Logarit năng lượng (Log-Mel)

Khối Log-Mel nén dải động của 40 năng lượng Mel trước khi đưa vào DCT. Do năng lượng Mel có thể chênh lệch nhiều bậc độ lớn, phép log giúp các băng yếu vẫn có ảnh hưởng trong vector đặc trưng, thay vì bị các băng năng lượng lớn chi phối.

Thiết kế phần cứng sử dụng log cơ số 2 để thuận tiện cho biểu diễn nhị phân. Với một số nguyên dương X , vị trí bit 1 cao nhất cho biết phần nguyên của $\log_2(X)$. Phần định trị sau khi chuẩn hóa được xấp xỉ bằng bảng tra LUT 256 phần tử.

Quy trình xử lý gồm ba bước. Đầu tiên, Leading-One Detector xác định chỉ số bit 1 cao nhất. Tiếp theo, thiết kế lấy 8 bit sau MSB làm địa chỉ để đọc phần thập phân từ ROM log_lut. Cuối cùng, phần nguyên và phần thập phân được ghép lại thành kết quả Q16.16 để truyền sang DCT/MFCC.

$$X = 2^k \cdot m, \quad 1 \leq m < 2$$



Hình 3.9. Sơ đồ khối module log_block.

Cách tổ chức này tránh phải dùng FPU hoặc IP log floating-point, phù hợp với mục tiêu fixed-point của đề tài. Python golden model phải dùng cùng cơ số log2, cùng quy tắc kẹp đầu vào và cùng thang Q16.16 để tránh sai khác khi đối sánh.

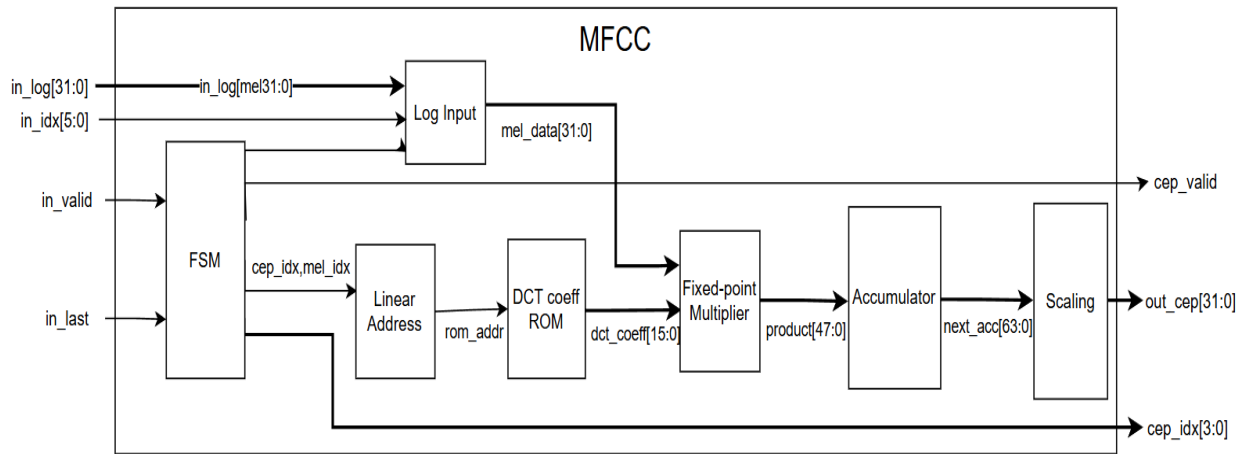
$$\log_2(X) = k + \log_2(m)$$

Bảng 3.9. Giao diện tín hiệu module log_block.

Port name	Dir	Width	Fixed-point	Mô tả hành vi
clk	in	1	-	Clock
rst_n	in	1	-	Reset active-low
in_valid	in	1	-	=1 $\rightarrow$ xử lý log(mel)
in_mel	in	IN_W (64)	signed Mel energy, chưa phải Q16.16	Mel energy input từ mel_filterbank
in_idx	in	6	-	Mel index
in_last	in	1	-	Marker mel cuối
out_valid	out	1	-	=1 khi logmel hợp lệ
out_logmel	out	OUT_W (32)	signed Q16.16	Log approximation (LUT)
out_idx	out	6	-	Pass-through index
out_last	out	1	-	Pass-through last

3.2.7. Khối Biến đổi Cosine rời rạc (DCT/MFCC)

Khối DCT/MFCC nhận 40 giá trị Log-Mel của mỗi frame và tạo 13 hệ số MFCC C0 đến C12. Vai trò của khối là nén thông tin đường bao phổ vào các hệ số bậc thấp, đồng thời giảm số chiều dữ liệu trước khi ghi vào Feature RAM.



Hình 3.10. Sơ đồ khối module DCT/MFCC.

Thiết kế sử dụng một MAC tuần tự để tính 13 tích vô hướng. Hệ số cosine được tính trước bằng Python, lượng tử hóa fixed-point và lưu trong ROM. FSM quản lý chỉ số cepstral p trong miền 0-12 và chỉ số băng Mel m trong miền 0-39. Với mỗi giá trị p, khối lần lượt đọc 40 giá trị Log-Mel, nhân với hệ số DCT tương ứng và cộng

dồn vào accumulator. Tín hiệu out_last chỉ được phát cùng hệ số C12 để báo rằng frame hiện tại đã tạo đủ 13 hệ số.

Bảng 3.10. Giao diện tín hiệu module MFCC.

Port name	Dir	Width	Fixed-point	Mô tả hành vi
clk	in	1	-	Clock
rst_n	in	1	-	Reset active-low
in_valid	in	1	-	=1 → ghi logmel vào RAM
in_logmel	in	IN_W (32)	Q16.16 signed	Log-mel input
in_idx	in	6	-	Index 0..39
in_last	in	1	-	=1 khi nhận mel cuối → bắt đầu DCT
out_cep_valid	out	1	-	=1 khi cepstrum output hợp lệ
out_cep	out	OUT_W (32)	Q16.16 signed	Cepstral coeff (DCT output)
out_cep_idx	out	4	-	Cep index 0..12

3.2.8. Khối Bộ đệm đặc trưng (Feature RAM Buffer)

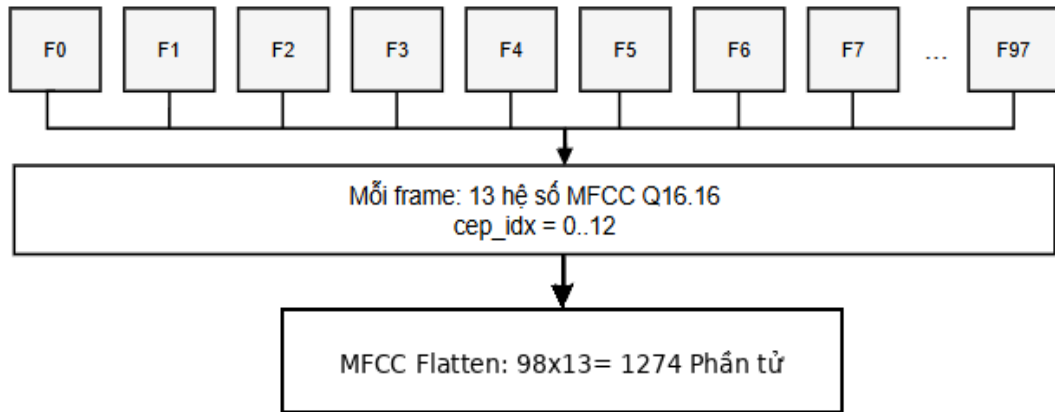
Khối Feature RAM Buffer nối pipeline MFCC với MLP Core. MFCC sinh dữ liệu theo từng frame, mỗi frame gồm 13 hệ số, trong khi MLP cần một vector đủ dài để quan sát toàn bộ diễn biến của từ khóa. Vì MLP là mô hình truyền thẳng và không có bộ nhớ hồi tiếp, bộ đệm 98 frame đóng vai trò gom chuỗi thời gian thành vector tĩnh 1274 phần tử.

Bộ nhớ có 1274 phần tử, mỗi phần tử rộng 32 bit, tương ứng 98 frame và 13 hệ số MFCC trên mỗi frame. Ở phía ghi, khi wr_en được kích hoạt, giá trị MFCC từ khối DCT/MFCC được ghi vào vị trí wr_addr. Địa chỉ ghi được controller tạo theo quy tắc $\text{frame_count} \times 13 + \text{cep_idx}$, trong đó frame_count nằm trong miền 0-97 và cep_idx nằm trong 0-12.

Ở phía đọc, MLP Core truy xuất dữ liệu thông qua cổng đọc đồng bộ của RAM. Khi rd_en được kích hoạt, dữ liệu tại địa chỉ rd_addr được chốt ra rd_data ở cạnh clock. Cách tổ chức này cho phép MLP đọc tuần tự 1274 đặc trưng trong quá trình tính toán tầng đầu vào.

Thiết kế sử dụng cơ chế block-based accumulation. Hệ thống ghi đủ một block gồm 98 frame $\times$ 13 hệ số trước khi kích hoạt MLP xử lý. Khi hoàn tất một lần suy luận, controller hoặc driver điều phối việc bắt đầu block dữ liệu tiếp theo, thay vì tự động dịch cửa sổ sau mỗi frame mới. Cách mô tả này cần được giữ nhất quán khi tính latency và khi xây dựng chương trình C gửi dữ liệu.

$$wr_{addr} = frame_{count} \times 13 + cep_{idx}$$



Hình 3.11. Kiến trúc bộ đệm đặc trưng theo block 98 frame.

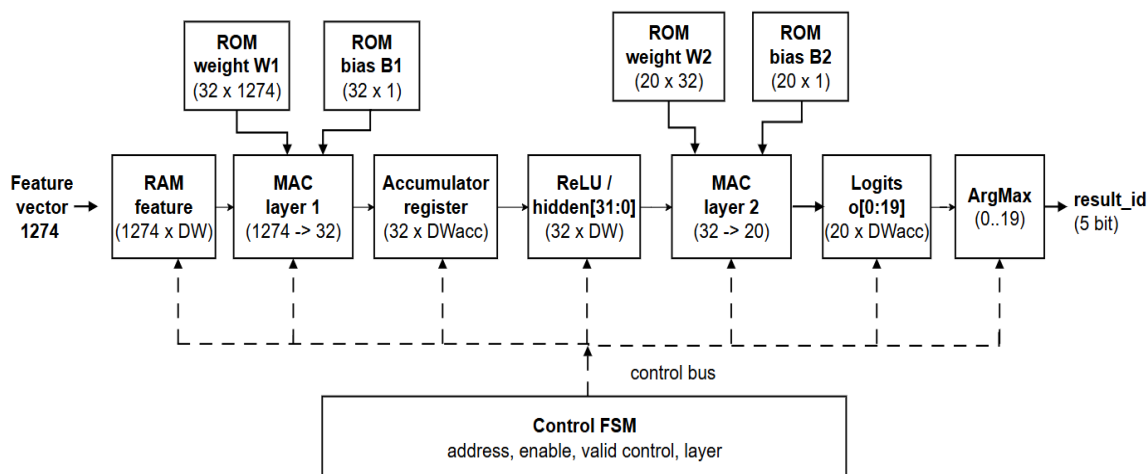
Hình 3.11 minh họa cấu trúc Feature RAM Buffer theo cơ chế block-based. Khối RAM nhận lần lượt dữ liệu MFCC và ghi vào địa chỉ do controller tạo theo quy tắc $frame_count \times 13 + cep_idx$. Controller chỉ phát mlp_start sau khi đã tích lũy đủ 98 frame hợp lệ. Thiết kế hiện tại không sử dụng sliding window, sau mỗi lần suy luận, controller hoặc driver điều phối block dữ liệu kế tiếp để đảm bảo dữ liệu của từng file PCM độc lập với nhau.

Bảng 3.11. Giao diện tín hiệu module mfcc_feature_buffer.

Port name	Dir	Width	Mô tả hành vi
clk	in	1	Clock hệ thống
wr_en	in	1	Tín hiệu cho phép ghi từ khối DCT
wr_data	in	32	Dữ liệu MFCC (Q16.16)
wr_addr	in	11	Địa chỉ ghi Feature RAM, 0-1273
rd_addr	in	11	Địa chỉ đọc Feature Buffer (0..1273)

Port name	Dir	Width	Mô tả hành vi
rd_data	out	32	Dữ liệu đặc trưng gửi sang MLP sau một chu kỳ đọc đồng bộ
rd_en	in	1	Cho phép đọc dữ liệu đặc trưng từ RAM

3.2.9. Khối Phân loại mạng nơ-ron (MLP Core)



Hình 3.12. Datapath xử lý của MLP Core.

Hình 3.12 phân tách datapath và control của MLP Core. MAC Engine được tái sử dụng cho tầng ẩn và tầng đầu ra, Weight ROM và Bias ROM được chọn theo trạng thái điều khiển. Sau khi cộng bias, kết quả tầng ẩn được đưa về thang Q16.16 và áp dụng ReLU trước khi lưu vào Hidden RAM.

MLP Core là khối suy luận trung tâm của hệ thống, nhận vector đặc trưng 1274 phần tử và tạo 20 logit đầu ra. Mỗi logit là điểm số chưa chuẩn hóa của một lớp. Thiết kế phần cứng không cần tính Softmax vì việc phân loại chỉ yêu cầu chọn lớp có logit lớn nhất bằng ArgMax.

Để giảm tài nguyên, thiết kế không triển khai toàn bộ các phép nhân song song. Thay vào đó, một MAC Engine được tái sử dụng theo thời gian. Cách tổ chức này làm tăng số chu kỳ xử lý so với triển khai song song hoàn toàn, nhưng phù hợp với mục tiêu tiết kiệm DSP và dễ kiểm chứng theo từng bước tính toán.

a. Tổ chức lưu trữ trọng số

Toàn bộ tham số mô hình sau huấn luyện được lượng tử hóa và lưu trong các ROM nội bộ. W1 có kích thước $1274 \times 32 = 40.768$ trọng số, B1 gồm 32 bias của tầng ẩn, W2 có kích thước $32 \times 20 = 640$ trọng số, B2 gồm 20 bias của tầng đầu ra. Tùy kích thước và chỉ thị tổng hợp, Vivado có thể ánh xạ các ROM này sang BRAM hoặc distributed ROM.

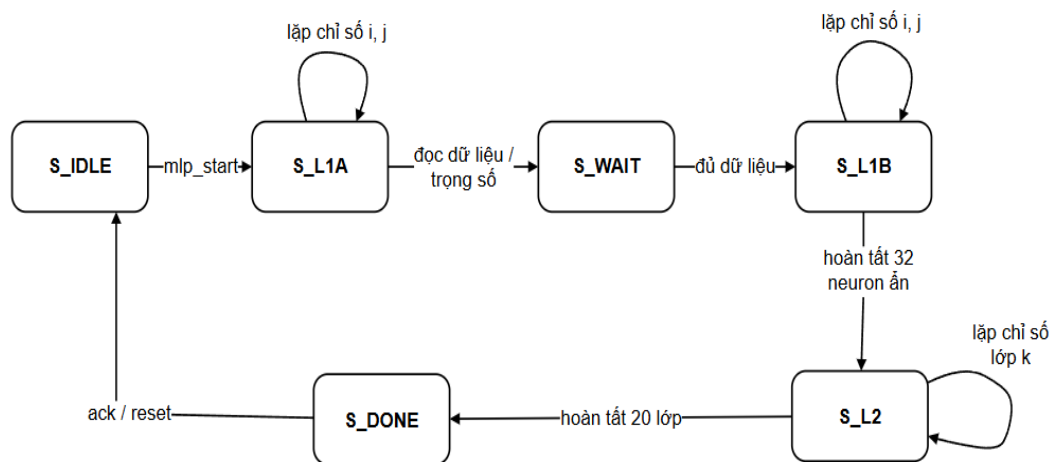
Trọng số được lưu theo thứ tự tuyến tính giống file HEX do Python sinh ra. Điều này đặc biệt quan trọng vì chỉ cần đảo sai thứ tự flatten của W1 hoặc W2, các logit đầu ra sẽ thay đổi dù giá trị trọng số không sai. Vì vậy, quy tắc địa chỉ của ROM phải được đối chiếu trực tiếp với mô hình Python fixed-point.

b. Máy trạng thái điều khiển FSM

Hình 3.13 mô tả quá trình suy luận bằng FSM ở mức chức năng. Ở S_IDLE, khởi chờ mlp_start từ Feature RAM Buffer. Ở S_L1A và S_WAIT, FSM tạo địa chỉ đọc đặc trưng và trọng số cho tầng ẩn, đồng thời chờ dữ liệu hợp lệ từ RAM/ROM.

Ở S_L1B, MAC Engine quét 1274 đầu vào cho từng nơ-ron ẩn, cộng bias B1, áp dụng ReLU và lưu 32 giá trị hidden.

Ở S_L2, FSM đọc 32 giá trị hidden, nhân với W2, cộng B2 và tạo lần lượt 20 logit đầu ra. Trong quá trình này, bộ so sánh ArgMax cập nhật giá trị logit lớn nhất và chỉ số lớp tương ứng. Khi hoàn tất 20 lớp, FSM chuyển sang S_DONE để phát mlp_done và giữ result_id ổn định cho controller/top-level chốt kết quả.



Hình 3.13. FSM điều khiển MLP Core.

c. Đầu ra phân loại và cơ chế chốt kết quả

Đầu ra trực tiếp của MLP Core là `result_id[5:0]` và tín hiệu `mlp_done`. `result_id` nằm trong miền 0-19, tương ứng 20 từ khóa/nhãn của dataset. `result_valid` không được xem là tín hiệu tính toán bên trong MLP, mà là tín hiệu hợp lệ do controller hoặc top-level phát sau khi nhận `mlp_done` và chốt `result_id`.

Cách tách `mlp_done` và `result_valid` giúp hệ thống kiểm soát rõ thời điểm MLP hoàn tất và thời điểm kết quả được công bố ra ngoài IP. Khi kiểm chứng, testbench cần theo dõi cả hai tín hiệu này để phân biệt lỗi tính toán MLP với lỗi điều phối ở mức top-level.

Bảng 3.12. Giao diện tín hiệu module `mlp_core`.

Port name	Dir	Width	Mô tả hành vi
<code>clk, rst_n</code>	in	1	Clock và Reset
<code>start</code>	in	1	Tín hiệu bắt đầu suy luận
<code>ram_addr</code>	out	11	Địa chỉ đọc Feature Buffer (0..1273)
<code>ram_data</code>	in	32	Dữ liệu đặc trưng đầu vào
<code>done</code>	out	1	=1 khi hoàn tất suy luận
<code>result_id</code>	out	6	ID từ khóa dự đoán trong dải 0-19

3.3. Kiến trúc SoC và vai trò PS-PL

3.3.1. Nền tảng KV260 và thành phần PS/PL

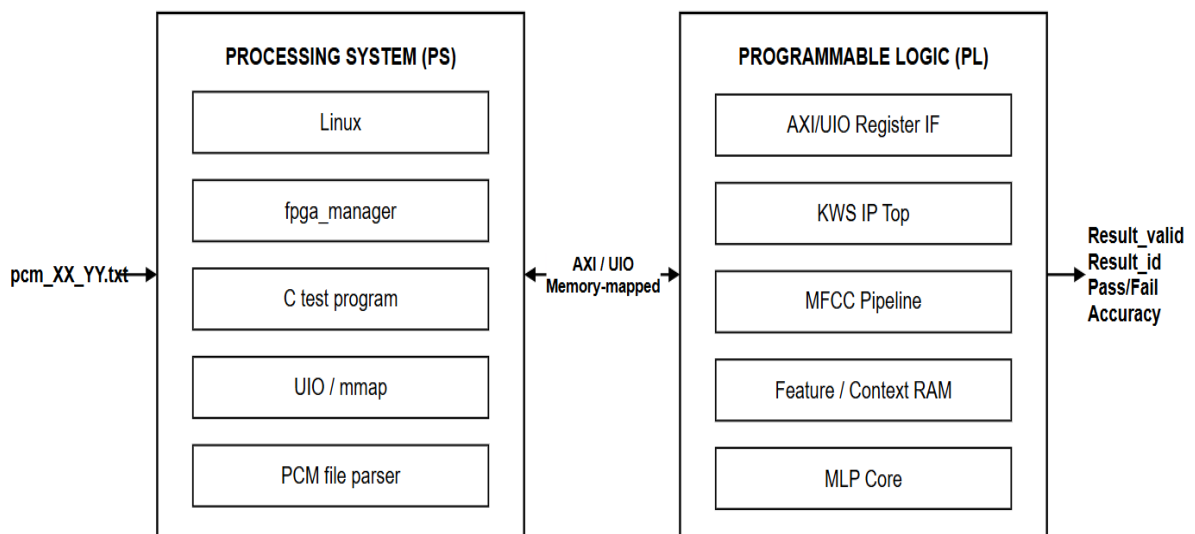
Kria KV260 sử dụng K26 SOM dựa trên Zynq UltraScale+ MPSoC. Processing System, PS, chứa các lõi ARM, bộ điều khiển bộ nhớ và ngoại vi. Programmable Logic, PL, chứa tài nguyên lập trình gồm LUT, FF, BRAM/UltraRAM và DSP48E2. Trong hệ thống đề xuất, PS chịu trách nhiệm quản lý file, nạp bitstream, điều khiển thanh ghi và thu thập kết quả, trong khi PL thực hiện datapath KWS.

3.3.2. Luồng nạp, điều khiển và trao đổi dữ liệu

Bitstream sau implementation được sao chép vào `/lib/firmware` và nạp bằng Linux FPGA Manager. Sau khi PL được cấu hình, device tree và UIO cung cấp vùng ánh xạ cho KWS IP. Chương trình C mở `/dev/uioX`, gọi `mmap`, kiểm tra thanh ghi

trạng thái, phát soft reset, sau đó truyền từng mẫu PCM theo handshake phần mềm-phần cứng.

Cách thiết kế PS-PL cho phép phần mềm xử lý các tác vụ không cố định như duyệt file, tách expected_id, quản lý timeout và ghi log. Phần cứng chỉ tập trung vào chuỗi tính toán lặp lại, gồm MFCC, Feature Buffer và MLP. Nhờ vậy, cùng một bộ file PCM có thể được dùng cho mô phỏng, regression và kiểm thử board-level.



Hình 3.14. Kiến trúc SoC PS-PL khi tích hợp KWS IP.

Quan sát Hình 3.14, dữ liệu PCM không được nạp trực tiếp từ testbench khi chạy trên board. Chương trình C trên PS mở thiết bị UIO, ánh xạ vùng thanh ghi và chỉ ghi mẫu khi trạng thái ready cho phép. Kết quả result_id chỉ được đọc khi result_valid được kích hoạt.

Kiến trúc này tách rõ đường điều khiển và đường tính toán, đồng thời cho phép cùng một bộ dữ liệu pcm_XX_YY.txt được sử dụng cho cả simulation và kiểm thử board.

Bảng 3.13. Vai trò các thành phần của nền tảng KV260 trong đề tài.

Thành phần	Vai trò
PS/Linux	Nạp bitstream, đọc file PCM, điều khiển batch test, ghi log
PL/FPGA	MFCC pipeline, context buffer, MLP và thanh ghi kết quả

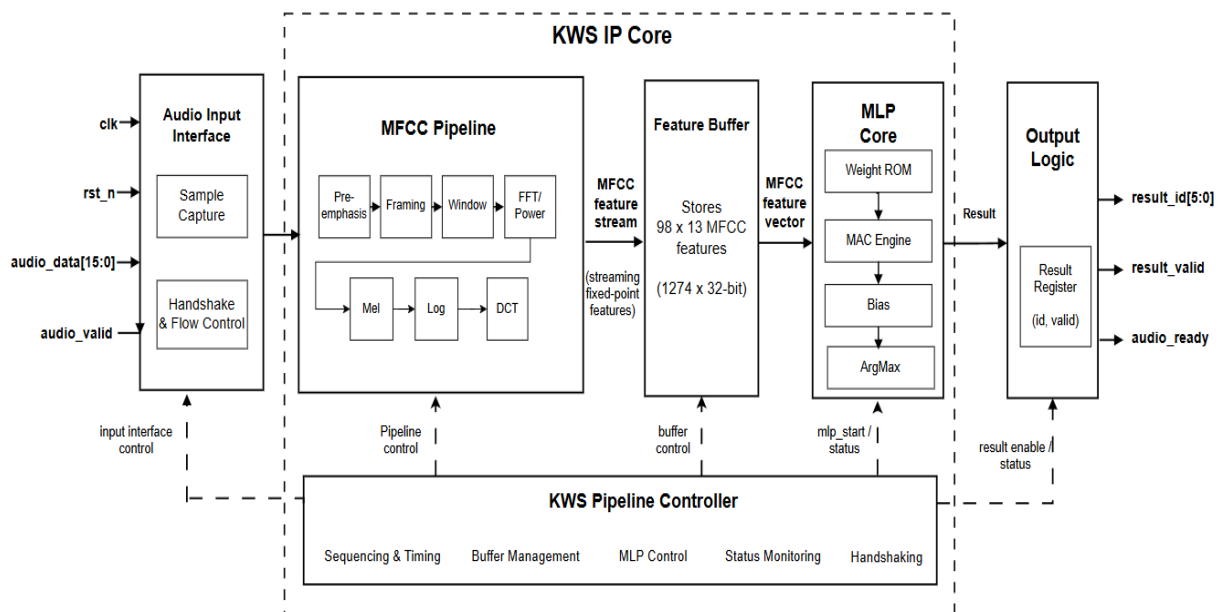
Thành phần	Vai trò
DSP48E2	Phép nhân và MAC
BRAM/ROM	Hamming, Mel, DCT, weight, bias và feature buffer
UIO/mmap	Ánh xạ thanh ghi IP sang user space
fpga_manager	Cấu hình PL bằng bitstream

3.4. Kiến trúc phần cứng và phần mềm

3.4.1. Kiến trúc phần cứng

Phần cứng được thiết kế theo kiến trúc streaming. Các module MFCC xử lý dữ liệu theo từng mẫu hoặc từng frame, các hệ số cố định được lưu trong ROM, còn accumulator của Mel, DCT và MLP được mở rộng độ rộng bit để hạn chế overflow. Feature Buffer và MLP dùng cơ chế đọc địa chỉ tuần tự để tái sử dụng MAC thay vì triển khai toàn bộ phép nhân song song.

Ở top-level, giao tiếp audio_valid/audio_ready được dùng như cơ chế truyền mẫu PCM từ lớp thanh ghi hoặc driver vào KWS IP. Mỗi mẫu chỉ được nhận khi hai tín hiệu cùng tích cực. result_valid được phát khi kết quả phân loại đã hợp lệ và result_id giữ giá trị nhẵn trong dải 0-19.



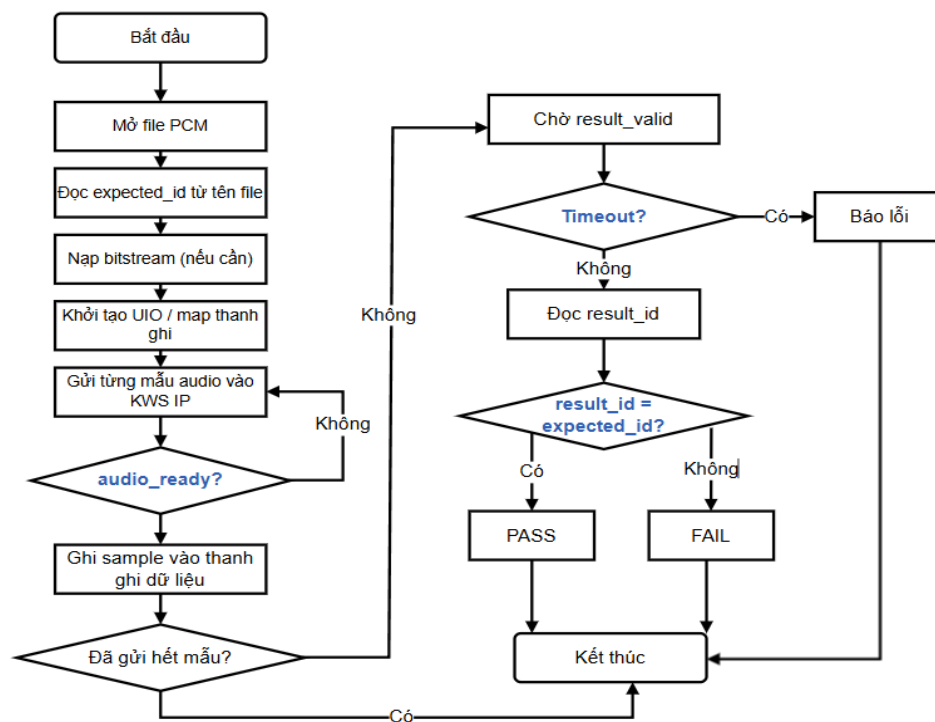
Hình 3.15. Kiến trúc phần cứng của KWS IP trong PL.

Hình 3.15 mô tả cấu trúc phần cứng bên trong KWS IP khi triển khai trong Programmable Logic. Dữ liệu âm thanh đầu vào đi qua Audio Input Interface để đồng bộ bộ audio_valid, audio_ready và audio_data. Sau đó, dữ liệu được đưa vào MFCC Pipeline, gồm các khối Pre-emphasis, Framing, Windowing, FFT/Power, Mel Filterbank, Log và DCT để tạo chuỗi đặc trưng MFCC.

Các hệ số MFCC được ghi vào Feature Buffer để tích lũy đủ 98 frame, tương ứng $98 \times 13 = 1274$ đặc trưng. Khi bộ đệm sẵn sàng, MLP Core được kích hoạt để đọc vector đặc trưng, thực hiện suy luận mạng 1274-32-20 và tạo kết quả phân loại. Output Logic chốt result_id, phát result_valid và cập nhật trạng thái audio_ready.

KWS Pipeline Controller điều phối trình tự xử lý của toàn bộ IP, bao gồm quản lý timing, điều khiển bộ đệm, phát mlp_start, theo dõi trạng thái MLP và bắt tay dữ liệu giữa các khối. Nhờ đó, đường dữ liệu PCM-MFCC-Feature Buffer-MLP-result được tách rõ với đường điều khiển, giúp thiết kế dễ kiểm chứng ở mức module, top-level và board-level.

3.4.2. Kiến trúc phần mềm



Hình 3.16. Luồng chương trình C kiểm thử trên PS/Linux.

Hình 3.16 mô tả luồng chương trình C kiểm thử chạy trên PS/Linux. Chương trình bắt đầu bằng mở file PCM, tách expected_id từ tên file hoặc tên thư mục, nạp bitstream nếu cần và khởi tạo vùng thanh ghi KWS IP thông qua UIO/mmap. Trong quá trình gửi dữ liệu, phần mềm chỉ ghi mẫu PCM vào thanh ghi dữ liệu khi trạng thái audio_ready cho phép, nhằm bảo đảm đúng thứ tự bắt tay phần mềm-phần cứng.

Phần mềm kiểm thử được tổ chức thành ba lớp: nạp bitstream, driver UIO và ứng dụng batch test. Lớp nạp bitstream cấu hình PL thông qua FPGA Manager, lớp UIO/mmap cho phép chương trình C truy cập vùng thanh ghi của KWS IP, còn lớp batch test duyệt file PCM, tách expected_id, gửi dữ liệu, xử lý timeout và thống kê PASS/FAIL.

Sau khi gửi hết mẫu âm thanh, chương trình chờ result_valid. Nếu quá thời gian chờ, testcase được ghi nhận là timeout. Khi result_valid được kích hoạt, phần mềm đọc result_id và so sánh với expected_id để xác định PASS hoặc FAIL. Do dataset được đánh số thư mục từ 01 đến 20, trong khi MLP xuất ID từ 0 đến 19, chương trình sử dụng quy tắc $\text{expected_id} = \text{folder_id} - 1$.

3.5. Tích hợp top-level và giao tiếp PS-PL

3.5.1. Giao tiếp top-level

Top-level nhận audio_valid, audio_data và trả audio_ready. Mẫu chỉ được chấp nhận khi audio_valid và audio_ready cùng bằng 1 tại cạnh lên clock. Các module truyền ra frame bằng tín hiệu last hoặc chỉ số tương đương để tránh sai lệch số mẫu. Kết quả phân loại chỉ được xem là hợp lệ khi result_valid được kích hoạt.

Bên trong top-level, dữ liệu đi tuần tự qua pipeline MFCC, Feature Buffer và MLP Core. audio_data đi vào pipeline MFCC, 13 hệ số MFCC mỗi frame được ghi vào Feature Buffer, sau đó MLP Core được kích hoạt khi đủ 1274 đặc trưng. Các tín hiệu result_id và result_valid được đưa ra ngoài IP để lớp AXI4-Lite register wrapper cho phần mềm đọc.

Bảng 3.14. Giao tiếp top-level của KWS IP.

Tín hiệu	Hướng	Độ rộng	Ý nghĩa
clk	Input	1	Clock hệ thống

Tín hiệu	Hướng	Độ rộng	Ý nghĩa
rst_n	Input	1	Reset active-low
audio_valid	Input	1	Mẫu PCM hợp lệ
audio_data	Input	16	PCM signed 16-bit
audio_ready	Output	1	IP sẵn sàng nhận mẫu
result_valid	Output	1	Kết quả hợp lệ
result_id	Output	6	ID lớp 0-19
sample_count	Debug	32	Số mẫu đã nhận

3.5.2. Ánh xạ thanh ghi phần mềm-phần cứng

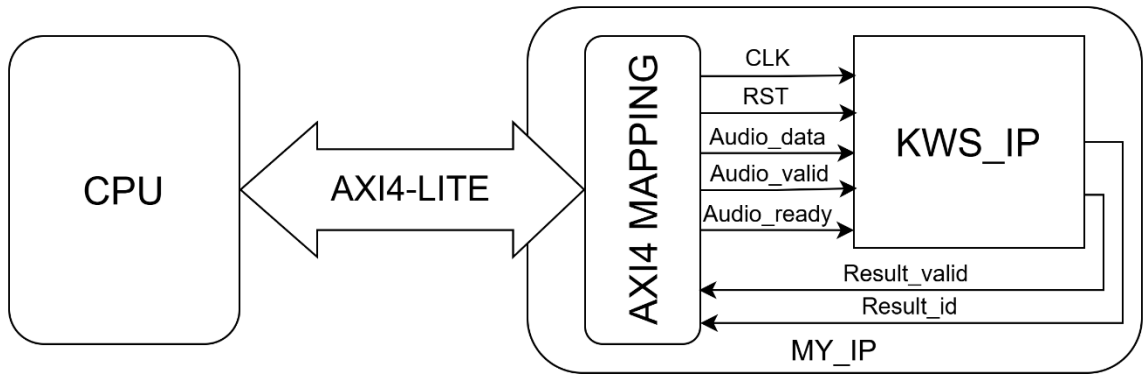
Bảng 3.15. Ánh xạ thanh ghi phần mềm-phần cứng của KWS IP.

Offset	Tên thanh ghi	Hướng	Chức năng
0x00	CTRL	W	Reset/start/enable
0x04	STATUS	R	ready, busy, result_valid
0x08	AUDIO_DATA	W	Ghi mẫu PCM 16-bit
0x0C	RESULT	R	Đọc result_id
0x10	SAMPLE_COUNT	R	Debug số mẫu
0x14	LAST_SAMPLE	R	Debug mẫu cuối

Các thanh ghi được tổ chức theo hướng tối giản phục vụ kiểm thử board-level. Nhóm control dùng để soft reset hoặc phát lệnh điều khiển, nhóm status phản ánh ready, pending và result_valid, thanh ghi PCM data nhận từng mẫu 16-bit từ phần mềm, còn thanh ghi result chứa result_id[5:0]. Giúp chương trình C chỉ cần thao tác read/write register mà không phụ thuộc vào chi tiết pipeline bên trong PL.

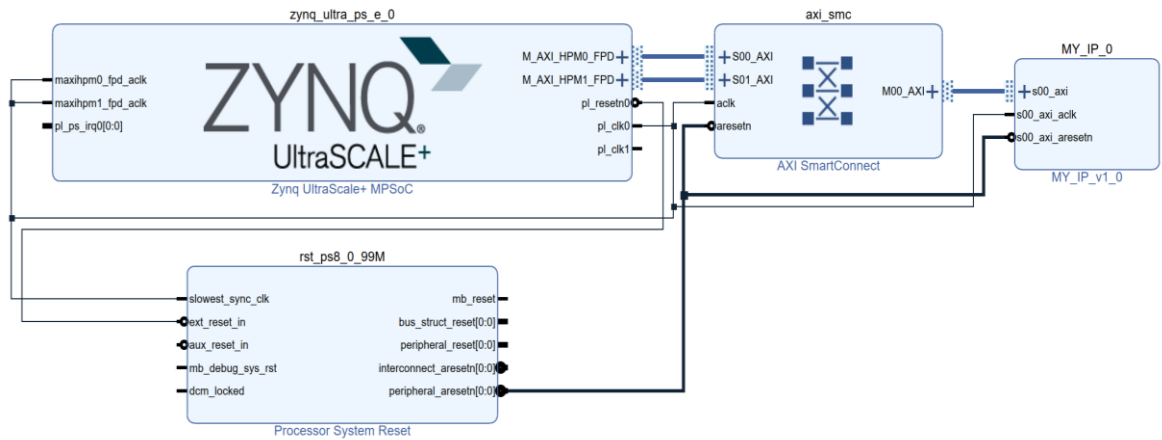
3.5.3. Trình tự xử lý phần mềm

Phần mềm xử lý trình tự: mở UIO, gọi mmap, phát soft reset, chờ IP ready, ghi từng mẫu PCM, theo dõi sample_count, chờ result_valid, timeout, đọc result_id và so sánh với expected_id. Trong quá trình gửi mẫu, phần mềm luôn kiểm tra trạng thái trước khi ghi audio_data. Nếu IP chưa sẵn sàng, chương trình chờ hoặc thử lại để tránh mất mẫu. Các trường hợp timeout ghi vào log và tính vào thống kê accuracy, giúp kết quả board-level có thể đối chiếu với cùng filelist của mô phỏng RTL.



Hình 3.17. Sơ đồ ánh xạ giao tiếp AXI4-Lite giữa CPU và KWS IP.

Trong Hình 3.17, CPU trong PS truy cập KWS IP thông qua giao tiếp AXI4-Lite và khối ánh xạ thanh ghi. Các tín hiệu audio_data, audio_valid, audio_ready, result_valid và result_id được đóng gói thành thanh ghi đọc/ghi để chương trình C truyền mẫu PCM và đọc kết quả. PS phụ trách đọc file, ghi dữ liệu và kiểm tra trạng thái, còn PL thực hiện pipeline MFCC và MLP.



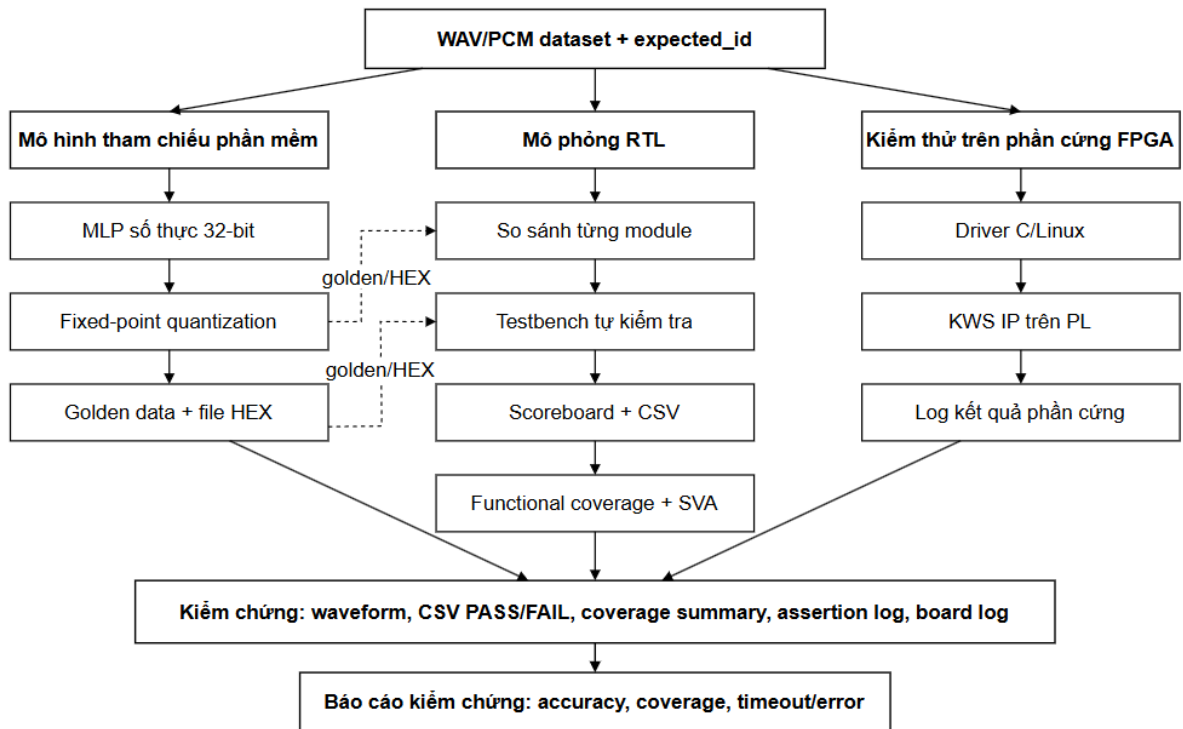
Hình 3.18. Vivado Block Design và kết nối PS-PL của hệ thống.

Hình 3.18 minh họa block design của hệ thống trên Vivado. Trong sơ đồ này, PS được kết nối với KWS IP trong PL thông qua giao tiếp AXI4-Lite. Clock, reset và vùng ánh xạ địa chỉ được cấu hình để chương trình Linux có thể truy cập trực tiếp các thanh ghi điều khiển của accelerator.

CHƯƠNG 4. MÔ HÌNH KIỂM CHỨNG

4.1. Mô hình kiểm chứng tổng quan

Hình 4.1 mô tả khung kiểm chứng tổng quan theo ba nhánh: mô hình tham chiếu phần mềm, mô phỏng RTL và kiểm thử trên phần cứng FPGA. Nhánh phần mềm tạo dữ liệu tham chiếu, huấn luyện/lượng tử hóa mô hình và xuất file hệ số. Nhánh RTL nhận cùng dữ liệu PCM, chạy qua testbench trước và sau synthesis để kiểm tra chức năng phần cứng. Nhánh phần cứng FPGA dùng bitstream sau implementation để xác minh hoạt động của IP trong môi trường PS-PL thực tế.



Hình 4.1. Mô hình kiểm chứng phần mềm, RTL và FPGA.

Ở mức module, comparator kiểm tra số phần tử, thứ tự index, sai số tuyệt đối, sai số trung bình và vị trí khác biệt đầu tiên giữa mô hình tham chiếu phần mềm và RTL. Ở mức top-level, testbench ghi lại file đầu vào, số mẫu đã gửi, expected_id, result_id, timeout/error và trạng thái PASS/FAIL. Ở mức phần cứng FPGA, chương trình C/Linux ghi log theo cấu trúc tương tự để đối chiếu với kết quả RTL.

Testbench RTL gồm ba phần chính. Driver đọc file PCM và phát mẫu theo giao thức audio_valid/audio_ready. Monitor bắt result_valid, result_id và các trạng thái cần thiết. Scoreboard đối chiếu result_id với expected_id trong filelist rồi ghi CSV/summary. Nhờ đó, mỗi lần chạy batch đều để lại log có thể kiểm tra lại, thay vì chỉ dựa vào ảnh chụp waveform.

4.2. Mô hình tham chiếu phần mềm và dữ liệu kiểm thử

Mô hình tham chiếu phần mềm được triển khai bằng Python và được tổ chức thành hai mức. Mức floating-point sử dụng số thực 32-bit để huấn luyện và đánh giá mô hình MLP trong điều kiện thuật toán lý tưởng. Mức fixed-point mô phỏng đúng các thang đo Q1.15, Q16.16, Q4.12 và Q28, đồng thời sử dụng cùng quy tắc làm tròn, dịch bit và bão hòa như RTL. Vì vậy, mọi kết luận về sai số phần cứng đều được đối chiếu với mô hình fixed-point thay vì chỉ so với floating-point.

Dữ liệu kiểm thử được chuẩn hóa về PCM signed 16-bit, mono, tần số lấy mẫu 16 kHz và độ dài xử lý 16032 mẫu, tương ứng 98 frame với frame length 512 và hop 160. Đối với batch RTL, mỗi file PCM được cộng thêm phần padding để flush pipeline, tạo tổng số mẫu 17032. Filelist kiểm thử chứa đường dẫn PCM và expected_id 0-based, bảo đảm cùng một batch có thể dùng cho Python fixed-point, RTL simulation và board-level validation.

Các file hệ số hamming_coeff.hex, mel_coeff.hex, dct_coeff.hex cùng các file trọng số/bias mlp_w1.hex, mlp_b1.hex, mlp_w2.hex và mlp_b2.hex đều được sinh từ Python. Sau khi export, script Python fixed-point được chạy lại trực tiếp trên các hệ số đã lượng tử hóa để kiểm tra số dòng, thứ tự lưu trữ, biểu diễn bù hai và miền giá trị. Bước này giúp loại trừ lỗi do file hệ số trước khi đưa sang Vivado.

4.3. Mục tiêu, phạm vi và tiêu chí đánh giá

Để tránh đánh giá cảm tính, quá trình kiểm chứng sử dụng các chỉ số định lượng thay vì chỉ quan sát waveform hoặc một vài log đơn lẻ. Mỗi testcase được xem là đạt khi DUT xử lý xong trong thời gian timeout cho phép, result_id nằm trong miền

hợp lệ 0-19 và result_id khớp expected_id của file PCM. Các testcase bị timeout, lỗi giao thức hoặc result_id ngoài miền hợp lệ đều được tính là không đạt.

$$\text{Accuracy} = \frac{N_{\text{correct}}}{N_{\text{total}}} \times 100\%$$

Trong đó N_{correct} là số file PCM có result_id trùng với expected_id, còn N_{total} là tổng số file được đưa vào batch kiểm thử. Ví dụ, kết quả 295/300 ở mức RTL validation nghĩa là 295 file được phân loại đúng trên tổng 300 file, tương ứng 98,33%. Khi đánh giá toàn bộ dataset 2.000 mẫu, cùng công thức này được áp dụng để thu được $1.996/2.000 = 99,80\%$.

$$\text{Timeout Rate} = \frac{N_{\text{timeout}}}{N_{\text{total}}} \times 100\%$$

$$\text{Error Rate} = \frac{N_{\text{error}}}{N_{\text{total}}} \times 100\%$$

Timeout rate phản ánh tỷ lệ testcase không nhận được result_valid trong khoảng thời gian chờ đã định. Error rate phản ánh các lỗi không thuộc sai phân loại thông thường, chẳng hạn result_id ngoài miền 0-19, lỗi đọc dữ liệu hoặc lỗi giao thức. Trong các batch RTL và board-level của đề tài, timeout/error bằng 0 cho thấy hệ thống hoàn tất xử lý ổn định, các mẫu không đạt còn lại là lỗi phân loại, không phải lỗi treo pipeline.

$$\text{Class Coverage} = \frac{N_{\text{class_hit}}}{20} \times 100\%$$

Class coverage cho biết tập kiểm thử đã chạm đủ 20 nhãn hay chưa. Chỉ số này cần thiết vì một batch có thể đạt accuracy cao nhưng chỉ kiểm tra một số lớp dễ. Với hệ thống KWS 20 lớp, batch dùng cho regression phải bảo đảm toàn bộ class ID 0-19 đều xuất hiện ít nhất một lần, khi đó coverage theo lớp đạt $20/20 = 100\%$.

Đối với validation và đánh giá toàn bộ dataset, kích thước batch cũng là một tiêu chí quan trọng. Kết quả trên một vài mẫu đơn lẻ chỉ phù hợp để debug luồng điều khiển, trong khi kết quả dùng để kết luận accuracy phải dựa trên tập đủ lớn. Vì vậy, báo cáo sử dụng batch validation 300 file và batch toàn bộ 2.000 file ở mức RTL và board-level để đánh giá độ chính xác và tính ổn định của IP trên KV260.

Quy trình kiểm chứng được thực hiện theo ba mức chính: mức phần mềm, mức RTL và mức triển khai trên phần cứng FPGA KV260. Quy trình này kiểm chứng theo trình tự: mô hình phần mềm được dùng để xác nhận thuật toán và sinh dữ liệu tham chiếu, RTL được kiểm chứng bằng mô phỏng trước và sau synthesis, cuối cùng bitstream được nạp lên FPGA để đánh giá hoạt động thực tế của IP trong hệ thống PS-PL.

Ở cả ba mức, mục tiêu kiểm chứng không chỉ là quan sát result_id cuối cùng mà còn kiểm tra tính nhất quán dữ liệu, giao thức điều khiển và khả năng hoàn tất xử lý không timeout. Dữ liệu trung gian được đối chiếu với golden model, giao thức được kiểm tra qua valid/ready, last, sample_count, reset và result_valid, kết quả phân loại được đánh giá bằng expected_id, result_id và accuracy trên từng batch kiểm thử.

Bảng 4.1. Kế hoạch kiểm chứng.

Mức kiểm chứng	Đối tượng	Phương pháp	Mục tiêu
Phần mềm	Mô hình phần mềm floating-point 32-bit và fixed-point	Trích xuất MFCC, huấn luyện/đánh giá MLP, mô phỏng Q-format, sinh golden data và file HEX	Xác nhận thuật toán và mô hình lượng tử hóa trước khi chuyển sang RTL
Phần cứng RTL	Các module RTL, top-level keyword_spotting_top, pre-synthesis và post-synthesis simulation	Vivado XSim, self-checking testbench, scoreboard, waveform, batch regression, functional coverage và SVA	Xác nhận chức năng phần cứng, giao thức điều khiển, dữ liệu trung gian và kết quả phân loại
Phần cứng FPGA KV260	Bitstream chạy trên PL, chương trình C/Linux trên PS, giao tiếp UIO/mmap	Nạp bitstream bằng fpga_manager, chạy single test và batch test nhiều file PCM	Xác nhận IP hoạt động trên phần cứng thật, không timeout/error và kết quả gần với mô phỏng RTL

4.4. Cơ chế đối sánh dữ liệu Python-RTL và chỉ số sai số

Đối với các giá trị trung gian và logit đầu ra, kiểm chứng không chỉ dùng trạng thái PASS/FAIL mà còn ghi lại sai số số học giữa Python và RTL. Với một vector gồm N phần tử, sai số tại phần tử i được định nghĩa là:

$$e_i = y_i^{\text{RTL}} - y_i^{\text{Python}}$$

$$\text{MAE} = \frac{1}{N} \sum_{i=0}^{N-1} |e_i|$$

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=0}^{N-1} e_i^2}$$

$$\text{Max Error} = \max_i |e_i|$$

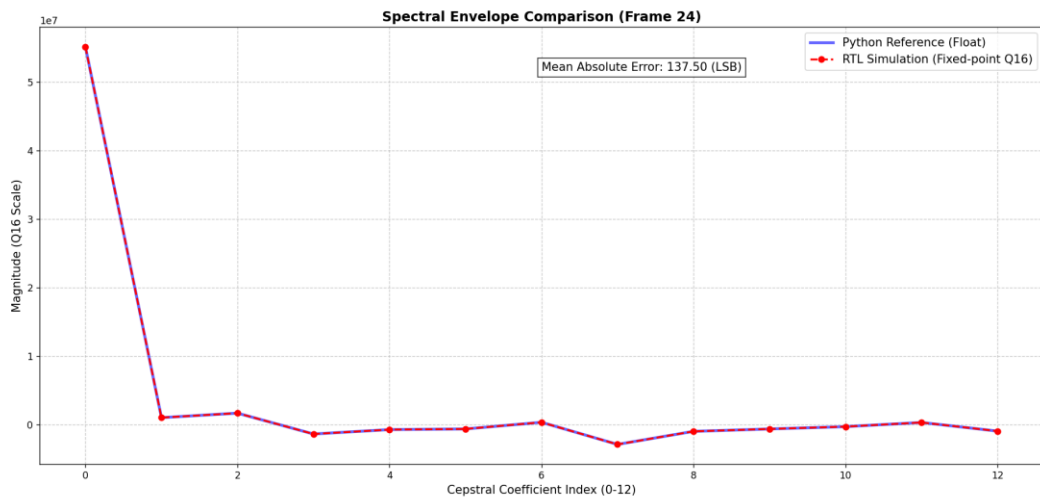
MAE (Mean Absolute Error) là sai số tuyệt đối trung bình, phản ánh mức lệch trung bình giữa hai miền tính toán. RMSE (Root Mean Square Error) là căn bậc hai của sai số bình phương trung bình, nhạy hơn với các sai số lớn cục bộ. Max Error là sai số tuyệt đối lớn nhất, dùng để khoanh vùng phần tử hoặc class có độ lệch lớn nhất. Các chỉ số này giúp phân biệt sai số số học ở từng tầng với sai số phân loại cuối cùng.

Đối với logit của MLP, dấu của sai số cũng có ý nghĩa: $e_i > 0$ cho biết RTL tính logit của class i lớn hơn Python, còn $e_i < 0$ cho biết RTL tính nhỏ hơn Python. Tuy nhiên, sai số này chỉ làm thay đổi result_id nếu nó làm đổi vị trí ArgMax giữa các class. Vì vậy, khi phân tích Hình 5.24, báo cáo vừa xét độ lớn sai số vừa xét khoảng cách giữa logit lớn nhất và các logit còn lại.

Cơ chế đối sánh được kiểm chứng theo từng mức. Đối với tín hiệu điều khiển, các chỉ số sample_idx, frame_idx, bin_idx, mel_idx, cep_idx và tín hiệu last phải khớp chính xác với số lượng phần tử kỳ vọng. Đối với dữ liệu số học, comparator ghi lại số phần tử được so sánh, sai số tuyệt đối lớn nhất, sai số trung bình tuyệt đối, RMSE và vị trí sai khác đầu tiên. Đối với classifier, testcase đạt khi result_id khớp expected_id, result_id nằm trong miền 0-19 và không xuất hiện timeout.

Hình 4.2 thể hiện ví dụ đối sánh MFCC giữa Python Golden Model và RTL Simulation trên cùng vector đặc trưng. Hai đường kết quả được đưa về cùng thang đo và cùng thứ tự hệ số C0-C12 để đánh giá sai số fixed-point sau hiện thực phần cứng.

Trong Hình 4.2, trục ngang biểu diễn chỉ số hệ số MFCC, còn trục dọc biểu diễn giá trị hệ số sau khi quy đổi về cùng miền số. Sai khác giữa hai đường chủ yếu đến từ làm tròn fixed-point, xấp xỉ logarit bằng LOD/LUT và cách cắt bit sau các phép MAC. Nếu sai số vượt ngưỡng đã định nghĩa, testcase được truy vết ngược về frame, mel bin hoặc hệ số cepstrum tương ứng để khoanh vùng lỗi.



Hình 4.2. Đồ thị so sánh giá trị hệ số MFCC giữa Python và RTL.

```

=====
KWS FUNCTIONAL COVERAGE + SVA SUMMARY
=====
[FC] total_files           = 20
[FC] pass_count            = 20
[FC] fail_count            = 0
[FC] timeout_count         = 0
[FC] error_count           = 0
[FC] no_result_count       = 0
[FC] result_valid_count    = 20
[FC] expected_class_coverage = 20/20
[FC] result_id_class_hit    = 20/20
[FC] sample_count_min      = 17032
[FC] sample_count_max      = 17032
[SVA] assertion_fail_count = 0
[FC PASS] Expected class coverage reached 20/20.
[FC PASS] timeout/error/no_result = 0.
[SVA PASS] 0 assertion fail.
=====

```

Hình 4.3. Kết quả tổng hợp functional coverage và assertion kiểm chứng RTL.

Hình 4.3 trình bày log tổng hợp functional coverage và SVA của tập kiểm thử RTL. Log cho thấy toàn bộ 20 testcase đều pass, timeout/error/no_result bằng 0, expected class coverage và result_id class coverage đạt 20/20, đồng thời assertion_fail_count bằng 0. Kết quả này minh họa cách testbench theo dõi độ phủ chức năng và kiểm tra các điều kiện đúng sai của giao tiếp kết quả.

4.5. Kịch bản kiểm thử, coverage và assertion

Kịch bản kiểm thử được chia thành bốn nhóm. Nhóm thứ nhất là module-level test, dùng để kiểm tra từng khối xử lý riêng biệt bằng golden data và waveform. Nhóm thứ hai là top-level batch regression, dùng để đưa nhiều file PCM qua toàn bộ pipeline PCM-MFCC-Feature Buffer-MLP. Nhóm thứ ba là post-synthesis functional simulation, dùng để xác nhận netlist sau synthesis vẫn giữ đúng hành vi chức năng trên mẫu đại diện. Nhóm thứ tư là board-level test trên KV260, dùng để kiểm tra luồng PS-PL, UIO/mmap, nạp bitstream, truyền mẫu PCM và đọc result_id trên phần cứng thật.

Functional coverage trong khóa luận được thực hiện để kiểm tra mức độ phủ kiểm thử. Các coverpoint chính gồm expected class ID, result_id hợp lệ, sample_count, result_valid, timeout/error, các chỉ số biên frame/bin/mel/cep và trạng thái reset. Assertion được dùng để bắt các lỗi giao thức khó thấy khi chỉ quan sát waveform, ví dụ result_id bị X/Z, result_id vượt miền 0-19, result_valid phát sai điều kiện hoặc dữ liệu thay đổi khi valid đang chờ ready.

Code coverage được sử dụng như một lớp kiểm tra bổ sung nhằm xác định các nhánh FSM, điều kiện reset và đường xử lý RTL đã được kích hoạt trong regression. Khác với functional coverage, code coverage không chứng minh hệ thống đúng về mặt chức năng, nhưng giúp phát hiện những nhánh FSM, điều kiện reset hoặc đường xử lý chưa từng được chạy trong mô phỏng.

Bảng 4.2 trình bày các phương pháp kiểm chứng chi tiết tương ứng với ba mức ở Bảng 4.1. Bảng này tập trung vào mục tiêu kiểm chứng, cách thực hiện, tiêu chí đánh giá và kết quả đạt được. Các kết quả waveform, log Vivado, log nạp bitstream,

ảnh setup kit và bảng thống kê accuracy được trình bày ở Chương 5 để đối chiếu kết quả từng hạng mục.

Bảng 4.2. Kế hoạch kiểm chứng, tiêu chí đánh giá và kết quả đạt được.

Hạng mục	Mức	Phương pháp	Tiêu chí đạt	Kết quả
Phần mềm FP32	Phần mềm	Huấn luyện và đánh giá mô hình MFCC-MLP số thực 32-bit, lưu loss, accuracy và confusion matrix.	Accuracy validation $\geq$ 95%, accuracy toàn bộ dataset $\geq$ 95%, có checkpoint tốt nhất, có báo cáo per-class và confusion matrix.	300/300 validation = 100,00%, 2000/2000 full dataset = 100,00%. Đạt.
Phần mềm fixed-point	Phần mềm	Mô phỏng Q-format: MFCC Q16.16, weight Q4.12, bias Q28, kiểm tra file HEX.	Accuracy validation $\geq$ 90%, độ suy giảm so với floating-point không vượt 5 điểm phần trăm, 4/4 file HEX đúng số dòng, không overflow bias.	299/300 validation = 99,67%, 1999/2000 full dataset = 99,95%, 4/4 file HEX đúng số dòng. Đạt.
Golden/data correctness	Phần mềm $\rightarrow$ RTL	Python sinh golden data cho từng khối xử lý, testbench so sánh output RTL theo index và sai số.	Đúng số phần tử, đúng thứ tự index, dữ liệu không lệch frame/bin/mel/cep, sai số fixed-point nằm trong ngưỡng cho phép của từng khối hoặc không làm thay đổi ArgMax ở tầng phân loại.	Golden data được dùng cho kiểm chứng từng khối, waveform và log đối chiếu trình bày ở Chương 5. Đạt.
Module-level RTL	RTL pre-synthesis	Mô phỏng pre-emphasis, framing, windowing, FFT/Power, Mel, Log, DCT/MFCC, Feature Buffer và MLP Core.	100% module chính có testbench, không assertion fail, valid/ready/last đúng vị trí, index và địa chỉ rd/wr đúng miền, không xuất hiện X/Z ở tín hiệu kết quả.	100% module có waveform/log, các tín hiệu chính khớp hành vi kỳ vọng. Đạt.

Top-level RTL batch	RTL pre-synthesis	Chạy filelist PCM qua keyword_spotting_top bằng self-checking testbench và scoreboard.	Accuracy validation $\geq 90\%$, full dataset accuracy $\geq 95\%$, timeout rate = 0%, error rate = 0%, sample_count đúng, result_id nằm trong 0-19.	295/300 validation = 98,33%, 1996/2000 full dataset = 99,80%, timeout/error = 0. Đạt.
Functional coverage	RTL pre-synthesis	Theo dõi class hit, số file xử lý, sample_count, result_valid, pass/fail, timeout/error và các biên index.	Độ phủ expected class = 20/20 lớp, file coverage = 100%, timeout/error = 0, các coverpoint chính về result_valid, sample_count và result_id hợp lệ đều được hit.	Expected-class coverage 20/20 = 100%, file coverage 2000/2000 = 100%, timeout/error = 0. Đạt.
Assertion checking	RTL pre-synthesis	Dùng assertion để kiểm tra result_id, result_valid, valid/ready, reset và điều kiện timeout.	0 assertion fail, result_id không vượt miền 0-19, result_id không X/Z khi result_valid = 1, không phát result_valid trước khi pipeline hoàn tất.	Không ghi nhận assertion fail trong các testcase đã chạy. Đạt.
Post-synthesis functional simulation	Netlist sau synthesis	Chạy mô phỏng chức năng sau synthesis trên testcase đại diện.	Testcase đại diện PASS, result_id khớp mô phỏng pre-synthesis, không X/Z ở output, không lỗi reset hoặc treo mô phỏng.	Post-synthesis functional simulation PASS trên mẫu đại diện. Đạt.
Board-level KV260	FPGA board	Nạp bitstream, chạy chương trình qua C/Linux qua UIO/mmap, truyền file PCM và đọc result_id.	Accuracy batch board-level $\geq 90\%$, timeout rate = 0%, error rate = 0%, IP không treo khi chạy nhiều file liên tiếp, result_id nằm trong 0-19.	294/300 validation = 98,00%, 1.995/2.000 full dataset = 99,75%, timeout/error = 0. Đạt.
Implementation/timing	Post-implementation	Tổng hợp, place-and-route và kiểm tra timing trên Vivado.	Thiết kế fit tài nguyên KV260, WNS ≥ 0 ns, TNS = 0 ns, Fmax ≥ 100 MHz	LUT = 26.053, FF = 21.879, BRAM = 74, DSP = 131, WNS = 0,225 ns,

			và $WNS \geq 0$ ns tại clock mục tiêu 100 MHz.	$F_{max} \approx 102,3$ MHz. Đạt.
Power estimate	Post-implementation	Dùng Vivado power estimate để ước lượng công suất on-chip.	Có báo cáo công suất sau implementation, công suất on-chip nằm trong mức phù hợp với triển khai edge FPGA, kết quả được dùng như số liệu ước lượng, không thay cho đo công suất board-level.	Tổng công suất on-chip ước lượng 0,595 W. Đạt ở mức power estimate.

Các tiêu chí trong Bảng 4.2 được thiết lập để phân biệt rõ giữa kết quả phân loại, độ ổn định giao thức và khả năng triển khai phần cứng. Accuracy được dùng để đánh giá chất lượng nhận diện của mô hình và IP trên các batch dữ liệu. Timeout rate và error rate được đặt bằng 0 vì hệ thống không chỉ cần dự đoán đúng mà còn phải hoàn tất xử lý ổn định, không treo và không phát sinh lỗi đọc/ghi dữ liệu. Đối với mô phỏng RTL, tiêu chí đạt không chỉ dựa vào result_id cuối cùng mà còn bao gồm valid/ready, last, index, địa chỉ bộ nhớ và assertion. Đối với implementation, thiết kế được xem là đạt khi ánh xạ được lên tài nguyên KV260 và đáp ứng timing tại clock mục tiêu 100 MHz. Nhờ đó, kết quả ở Chương 5 có cơ sở đánh giá rõ ràng thay vì chỉ trình bày các log hoặc waveform riêng lẻ.

4.6. Kế hoạch kiểm chứng và báo cáo kết quả kiểm chứng

Cơ chế đánh giá kết quả kiểm chứng được chia thành bốn nhóm. Nhóm giao thức yêu cầu mỗi mẫu chỉ được nhận khi audio_valid và audio_ready cùng tích cực, các tín hiệu last và index xuất hiện đúng vị trí, sample_count khớp số mẫu đã gửi, result_valid chỉ xuất hiện sau khi pipeline hoàn tất. Nhóm đối chiếu kết quả yêu cầu result_id nằm trong dải 0-19 và được so sánh với expected_id lấy từ filelist.

Testbench dùng mô hình driver-monitor-scoreboard để giảm thao tác kiểm tra thủ công. Driver phát mẫu PCM vào DUT, monitor bắt result_valid/result_id, còn scoreboard ghi trạng thái đạt/không đạt, timeout hoặc error cho từng file. Sau khi

chạy xong, testbench sinh summary và CSV để thống kê accuracy theo toàn bộ batch và theo từng nhân.

Functional coverage được dùng để xem tập kiểm thử đã chạm đến các tình huống quan trọng hay chưa. Các bộ đếm chính gồm số nhân được hit, số file đã xử lý, sample_count hợp lệ, result_valid xuất hiện, timeout/error và result_id ngoài miền hợp lệ. Assertion kiểm tra các lỗi khó thấy trên waveform như result_id bị X/Z, result_id vượt miền 0-19 hoặc dữ liệu đầu vào thay đổi khi valid đang chờ ready.

Bên cạnh batch validation, các corner tests được bổ sung để đánh giá hành vi hệ thống trong các điều kiện biên như silence input, biên độ cực đại, biên độ cực tiểu. Các testcase này không dùng để tính accuracy mô hình, nhưng có vai trò phát hiện lỗi overflow, lỗi reset, lỗi timeout và lỗi giao thức khi dữ liệu đầu vào không giống mẫu tiếng nói thông thường.

Toàn bộ kết quả kiểm chứng được lưu theo định dạng để có thể đối chiếu với kết quả kỳ vọng. Mỗi dòng kết quả được định dạng gồm index, loại testcase, đường dẫn file PCM, expected_id, trạng thái, predicted_id, sample_count và timeout_cycles. Summary cuối regression ghi tổng số mẫu, số mẫu đúng, số mẫu sai, timeout, error và accuracy tổng.

CHƯƠNG 5. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1. Môi trường thực nghiệm và dữ liệu đánh giá

Môi trường thực nghiệm được tổ chức theo ba mức đánh giá tương ứng với verification plan ở Chương 4. Ở mức phần mềm, mô hình được huấn luyện và đánh giá bằng PyTorch/Python với dữ liệu MFCC floating-point và fixed-point. Ở mức mô phỏng RTL, thiết kế được kiểm chứng bằng Vivado/XSim thông qua module-level testbench, top-level full-pipeline, batch regression và post-synthesis functional simulation. Ở mức triển khai trên phần cứng FPGA, bitstream được nạp lên KV260 và chương trình C/Linux sử dụng UIO/mmap để truyền dữ liệu PCM, điều khiển IP và đọc kết quả nhận diện.

Dữ liệu đánh giá được tổ chức theo quy tắc pcm_XX_YY.txt, trong đó XX biểu diễn ID thư mục dữ liệu và YY là số thứ tự mẫu. Vì nhãn dữ liệu bắt đầu từ 01 trong khi result_id của MLP nằm trong miền 0-19, chương trình kiểm thử ánh xạ expected_id = XX - 1 trước khi so sánh với kết quả phần cứng. Timeout được ghi nhận như một trạng thái lỗi riêng trong đánh giá end-to-end.

5.2. Kết quả kiểm chứng mức phần mềm

Ở mức floating-point, mô hình được huấn luyện và đánh giá bằng PyTorch với đặc trưng MFCC trích xuất từ Python. Ở mức fixed-point, cùng cấu hình MFCC được mô phỏng theo Q-format của RTL, sau đó xuất trọng số và bias thành các file HEX cho phần cứng. Kết quả phần mềm là mốc tham chiếu cho mô phỏng RTL và kiểm thử trên phần cứng FPGA.

Hình 5.1 ghi nhận quá trình huấn luyện mô hình floating-point. Sau bước early stopping, checkpoint được khôi phục tại epoch có validation accuracy tốt nhất. Kết quả cuối cùng trên tập validation đạt 300/300 mẫu đúng, tương ứng 100,00%, không ghi nhận mẫu sai ở mức phần mềm floating-point.

```

--- ĐANG TRÍCH XUẤT FLOAT FEATURE VALIDATION ---
-> Train samples sau augmentation: 6800
-> Validation samples: 300
Saved feature normalization: D:\DA2\Training data\float_feature_norm.npz
--- ĐANG TRAIN FLOATING-POINT MLP 20 LỚP ---
Epoch 0 | Train Loss: 0.9158 | Train Acc: 83.81% (5699/6800) | Val Loss: 0.8290 | Val Acc: 87.33% (262/300)
Epoch 100 | Train Loss: 0.0002 | Train Acc: 100.00% (6800/6800) | Val Loss: 0.0166 | Val Acc: 99.33% (298/300)
Epoch 200 | Train Loss: 0.0001 | Train Acc: 100.00% (6800/6800) | Val Loss: 0.0136 | Val Acc: 99.67% (299/300)
Epoch 300 | Train Loss: 0.0000 | Train Acc: 100.00% (6800/6800) | Val Loss: 0.0177 | Val Acc: 99.33% (298/300)
Epoch 400 | Train Loss: 0.0000 | Train Acc: 100.00% (6800/6800) | Val Loss: 0.0031 | Val Acc: 100.00% (300/300)
Epoch 500 | Train Loss: 0.0000 | Train Acc: 100.00% (6800/6800) | Val Loss: 0.0217 | Val Acc: 99.33% (298/300)
Epoch 600 | Train Loss: 0.0000 | Train Acc: 100.00% (6800/6800) | Val Loss: 0.0318 | Val Acc: 99.00% (297/300)
Epoch 700 | Train Loss: 0.0000 | Train Acc: 100.00% (6800/6800) | Val Loss: 0.0474 | Val Acc: 98.67% (296/300)
Epoch 800 | Train Loss: 0.0000 | Train Acc: 100.00% (6800/6800) | Val Loss: 0.0277 | Val Acc: 99.33% (298/300)
[EARLY STOP] Không cải thiện theo val_acc sau 500 epoch.
[BEST VAL LOSS] Epoch 394, Val Loss = 0.0030
[BEST VAL ACC] Epoch 384, Val Acc = 100.00%
[RESTORE] Float model selected by val_acc: epoch 384
[FINAL TRAIN] Loss = 0.0001, Acc = 100.00% (6800/6800), Wrong = 0
[FINAL VAL] Loss = 0.0037, Acc = 100.00% (300/300), Wrong = 0

```

Hình 5.1. Log huấn luyện và đánh giá Python floating-point trên tập validation.

Hình 5.2 ghi nhận quá trình warm-up và fixed-point. Mô hình được chọn ở chế độ std_only với fixed validation accuracy đạt 299/300 mẫu, tương ứng 99,67%. Kết quả này cho thấy sai khác do lượng tử hóa là nhỏ và mô hình fixed-point đủ ổn định để xuất hệ số sang RTL.

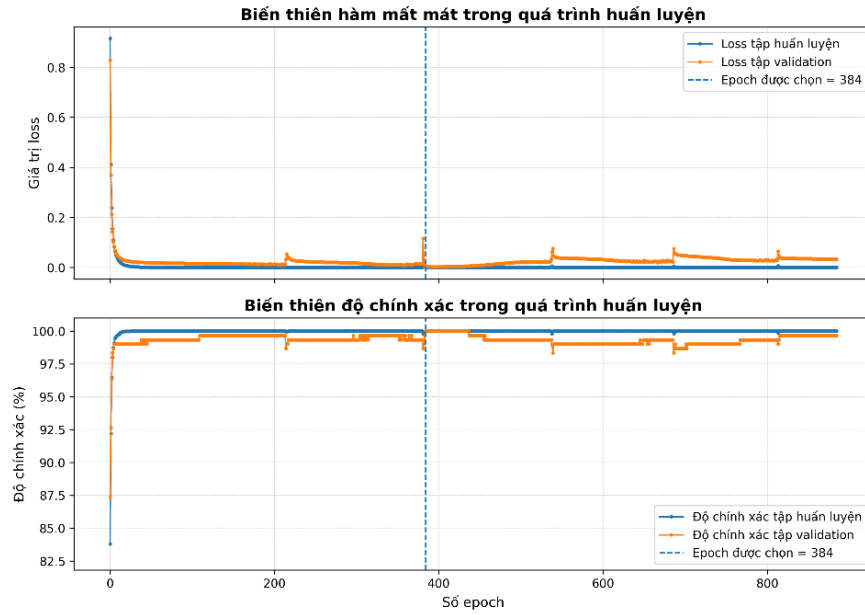
```

[NORM CHECK] std min=4.23397, std max=448.567
[WARMUP] Epoch 0 | Train Acc: 88.19% (5997/6800) | Val Acc: 89.33% (268/300) | Val Loss: 0.4951
[WARMUP] Epoch 50 | Train Acc: 100.00% (6800/6800) | Val Acc: 99.33% (298/300) | Val Loss: 0.0160
[WARMUP] Epoch 100 | Train Acc: 100.00% (6800/6800) | Val Acc: 99.33% (298/300) | Val Loss: 0.0148
[WARMUP] Epoch 150 | Train Acc: 100.00% (6800/6800) | Val Acc: 99.33% (298/300) | Val Loss: 0.0136
[WARMUP] Epoch 200 | Train Acc: 100.00% (6800/6800) | Val Acc: 99.33% (298/300) | Val Loss: 0.0139
[WARMUP] Epoch 250 | Train Acc: 100.00% (6800/6800) | Val Acc: 99.67% (299/300) | Val Loss: 0.0107
[QAT] Epoch 0 | QAT Val Acc: 99.33% (298/300) | FIXED Val Acc: 99.33% (298/300) | FIXED Train Acc: 100.00% (6800/6800) | Val Loss: 0.0258
[QAT] Epoch 50 | QAT Val Acc: 99.67% (299/300) | FIXED Val Acc: 99.67% (299/300) | FIXED Train Acc: 100.00% (6800/6800) | Val Loss: 0.0106
[QAT] Epoch 100 | QAT Val Acc: 99.67% (299/300) | FIXED Val Acc: 99.67% (299/300) | FIXED Train Acc: 100.00% (6800/6800) | Val Loss: 0.0096
[QAT] Epoch 150 | QAT Val Acc: 99.67% (299/300) | FIXED Val Acc: 99.67% (299/300) | FIXED Train Acc: 100.00% (6800/6800) | Val Loss: 0.0096
[QAT] Epoch 200 | QAT Val Acc: 99.00% (297/300) | FIXED Val Acc: 99.00% (297/300) | FIXED Train Acc: 100.00% (6800/6800) | Val Loss: 0.0223
[QAT] Epoch 250 | QAT Val Acc: 99.67% (299/300) | FIXED Val Acc: 99.67% (299/300) | FIXED Train Acc: 100.00% (6800/6800) | Val Loss: 0.0205
[QAT] Epoch 300 | QAT Val Acc: 99.67% (299/300) | FIXED Val Acc: 99.67% (299/300) | FIXED Train Acc: 100.00% (6800/6800) | Val Loss: 0.0194
[QAT] Epoch 350 | QAT Val Acc: 99.67% (299/300) | FIXED Val Acc: 99.67% (299/300) | FIXED Train Acc: 100.00% (6800/6800) | Val Loss: 0.0157
[QAT] Epoch 400 | QAT Val Acc: 99.33% (298/300) | FIXED Val Acc: 99.33% (298/300) | FIXED Train Acc: 100.00% (6800/6800) | Val Loss: 0.0304
[QAT] Epoch 450 | QAT Val Acc: 99.67% (299/300) | FIXED Val Acc: 99.67% (299/300) | FIXED Train Acc: 100.00% (6800/6800) | Val Loss: 0.0248
[QAT] Epoch 500 | QAT Val Acc: 99.67% (299/300) | FIXED Val Acc: 99.67% (299/300) | FIXED Train Acc: 100.00% (6800/6800) | Val Loss: 0.0190
[QAT] Epoch 550 | QAT Val Acc: 99.67% (299/300) | FIXED Val Acc: 99.67% (299/300) | FIXED Train Acc: 100.00% (6800/6800) | Val Loss: 0.0168
[QAT] Epoch 600 | QAT Val Acc: 99.67% (299/300) | FIXED Val Acc: 99.67% (299/300) | FIXED Train Acc: 100.00% (6800/6800) | Val Loss: 0.0191
[EARLY STOP] norm_mode=std_only, fixed-val không cải thiện sau 450 epoch.
[CANDIDATE DONE] norm_mode=std_only | best_epoch=179 | Fixed Val Acc=99.67% (299/300) | QAT Val Acc=99.67% (299/300)

=====
KẾT QUẢ CHỌN MODEL QAT TỐT NHẤT
=====
norm=std_only | fixed-val= 99.67% (299/300) | qat-val= 99.67% (299/300)
[SELECTED] norm_mode=std_only, fixed-val=99.67% (299/300), epoch=179

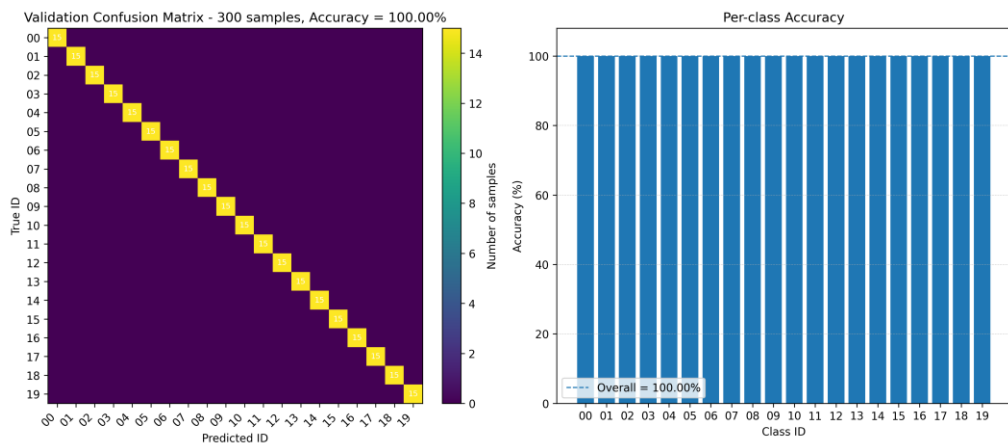
```

Hình 5.2. Log huấn luyện fixed-point trên tập validation.



Hình 5.3. Đường cong train/validation loss và accuracy Python floating-point.

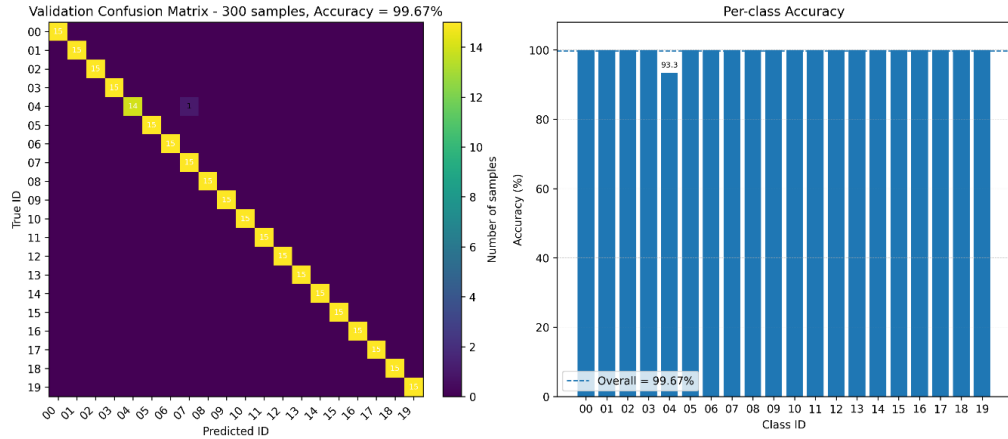
Hình 5.3 trình bày quá trình huấn luyện của mô hình MLP floating-point trên đặc trưng MFCC. Đường loss huấn luyện giảm nhanh về gần 0, trong khi accuracy huấn luyện đạt 100% sau một số epoch đầu. Accuracy trên tập validation dao động trong vùng cao, chủ yếu quanh 99-100%, và đạt 100% tại checkpoint được chọn ở epoch 384. Checkpoint này được chọn theo tiêu chí validation accuracy nhằm ưu tiên khả năng phân loại đúng trên tập kiểm chứng.



Hình 5.4. Ma trận nhầm lẫn của mô hình Python floating-point validation 300 mẫu.

Hình 5.4 trình bày ma trận nhầm lẫn của mô hình Python floating-point trên tập validation gồm 300 mẫu. Mỗi lớp có 15 mẫu kiểm chứng. Toàn bộ giá trị khác 0

đều nằm trên đường chéo chính của ma trận, cho thấy mô hình phân loại đúng toàn bộ 300/300 mẫu, tương ứng accuracy 100%. Kết quả này được sử dụng làm baseline phần mềm trước khi chuyển sang mô hình fixed-point và RTL.



Hình 5.5. Ma trận nhầm lẫn của mô hình Python fixed-point validation 300 mẫu.

Hình 5.5 trình bày ma trận nhầm lẫn của mô hình Python fixed-point bám RTL trên cùng tập validation 300 mẫu. Kết quả đạt 299/300 mẫu đúng, tương ứng accuracy 99,67%. Sai lệch duy nhất xuất hiện ở lớp 04, trong đó một mẫu bị dự đoán sang lớp 07, các lớp còn lại đều đạt 15/15 mẫu đúng. Kết quả này cho thấy quá trình lượng tử hóa và mô phỏng fixed-point gần như bảo toàn độ chính xác so với mô hình floating-point, đồng thời phù hợp để xuất trọng số HEX và triển khai sang RTL.

```
Total : 300
Pass : 295
Fail : 5
Timeout : 0
Errors : 0
Accuracy: 98.33%

-----
Per-class accuracy
ID | Total | Correct | Wrong | Accuracy
0 | 15 | 15 | 0 | 100.00%
1 | 15 | 15 | 0 | 100.00%
2 | 15 | 15 | 0 | 100.00%
3 | 15 | 15 | 0 | 100.00%
4 | 15 | 14 | 1 | 93.33%
5 | 15 | 15 | 0 | 100.00%
6 | 15 | 15 | 0 | 100.00%
7 | 15 | 14 | 1 | 93.33%
8 | 15 | 15 | 0 | 100.00%
9 | 15 | 14 | 1 | 93.33%
10 | 15 | 15 | 0 | 100.00%
11 | 15 | 15 | 0 | 100.00%
12 | 15 | 15 | 0 | 100.00%
13 | 15 | 15 | 0 | 100.00%
14 | 15 | 15 | 0 | 100.00%
15 | 15 | 15 | 0 | 100.00%
16 | 15 | 15 | 0 | 100.00%
17 | 15 | 15 | 0 | 100.00%
18 | 15 | 14 | 1 | 93.33%
19 | 15 | 14 | 1 | 93.33%

=====
Summary saved: D:/DA2/rtl_pcm_val_pyfixed_pad/rtl_accuracy_summary.txt
CSV saved : D:/DA2/rtl_pcm_val_pyfixed_pad/rtl_accuracy_result.csv
=====
$finish called at time : 625020195 ns : File "D:/DA2/tb_keyword_spotting_batch.sv"
run: Time (s): cpu = 00:02:26 ; elapsed = 01:16:39 . Memory (MB): peak = 1759.809 ;
```

Hình 5.6. Log batch validation 300 mẫu ở mức RTL pre-synthesis simulation.

Hình 5.6 thể hiện kết quả batch regression trên tập validation 300 file trong mô phỏng RTL behavioral. Kết quả đạt 295/300 mẫu đúng, 5 mẫu sai, timeout = 0 và error = 0, tương ứng accuracy 98,33%.

```
Total      : 2000
Pass       : 1996
Fail       : 4
Timeout    : 0
Errors     : 0
Accuracy   : 99.80%

-----
Per-class accuracy
ID | Total | Correct | Wrong | Accuracy
0 | 100 | 100 | 0 | 100.00%
1 | 100 | 100 | 0 | 100.00%
2 | 100 | 100 | 0 | 100.00%
3 | 100 | 100 | 0 | 100.00%
4 | 100 | 99 | 1 | 99.00%
5 | 100 | 100 | 0 | 100.00%
6 | 100 | 100 | 0 | 100.00%
7 | 100 | 99 | 1 | 99.00%
8 | 100 | 100 | 0 | 100.00%
9 | 100 | 100 | 0 | 100.00%
10 | 100 | 100 | 0 | 100.00%
11 | 100 | 100 | 0 | 100.00%
12 | 100 | 100 | 0 | 100.00%
13 | 100 | 100 | 0 | 100.00%
14 | 100 | 100 | 0 | 100.00%
15 | 100 | 100 | 0 | 100.00%
16 | 100 | 100 | 0 | 100.00%
17 | 100 | 100 | 0 | 100.00%
18 | 100 | 99 | 1 | 99.00%
19 | 100 | 99 | 1 | 99.00%
=====
Summary saved: D:/DA2/rtl_pcm_all_pyfixed_pad/rtl_accuracy_summary.txt
CSV saved    : D:/DA2/rtl_pcm_all_pyfixed_pad/rtl_accuracy_result.csv
=====
$finish called at time : 4166800195 ns : File "D:/DA2/tb_keyword_spotting_batch.sv"
run: Time (s): cpu = 00:28:48 ; elapsed = 08:26:34 . Memory (MB): peak = 2476.871 ;
```

Hình 5.7. Log batch toàn bộ 2.000 mẫu ở mức RTL pre-synthesis simulation.

Hình 5.7 ghi nhận kết quả chạy toàn bộ dataset 2.000 file ở mức RTL pre-synthesis. RTL đạt 1.996/2.000 mẫu đúng, 4 mẫu sai, không có timeout/error, tương ứng accuracy 99,80%.

```
[TB] Da nap tong cong 17000 samples
[TB] Dang cho ket qua MLP...

-----
DEBUG: Class 0 | Logit: -8189175955
DEBUG: Class 1 | Logit: -6977685763
DEBUG: Class 2 | Logit: -11983788295
DEBUG: Class 3 | Logit: -2122961140
DEBUG: Class 4 | Logit: -533953897
DEBUG: Class 5 | Logit: -2822314024
DEBUG: Class 6 | Logit: 1157311050
DEBUG: Class 7 | Logit: 2088574692
DEBUG: Class 8 | Logit: 697175894
DEBUG: Class 9 | Logit: -3977162212
DEBUG: Class 10 | Logit: -5252645572
DEBUG: Class 11 | Logit: -6557779941
DEBUG: Class 12 | Logit: -1249151756
DEBUG: Class 13 | Logit: -2463920086
DEBUG: Class 14 | Logit: -3072782909
DEBUG: Class 15 | Logit: -1873180643
DEBUG: Class 16 | Logit: -3049812560
DEBUG: Class 17 | Logit: 7195260924
DEBUG: Class 18 | Logit: 2286993777
DEBUG: Class 19 | Logit: -9952244
[TB MONITOR] result_valid at time=2086255000, result_id=17

=====
KET QUA NHAN DIEN FPGA
=====
-> Detected ID: 17
-> LABEL: [ 17_NGU VAI LON ]
```

Hình 5.8. Log kiểm chứng top-level RTL pre-synthesis cho một testcase đại diện.

Hình 5.8 trình bày log của testbench top-level ở mức pre-synthesis. Trên testcase đại diện, testbench phát đủ số mẫu PCM, hệ thống hoàn tất xử lý không timeout và result_id khớp expected_id, qua đó xác nhận luồng điều khiển và chức năng phân loại hoạt động đúng ở mức behavioral RTL.

```
[TB] Da gui 10000 samples
[TB] Da gui 11000 samples
[TB] Da gui 12000 samples
[TB] Da gui 13000 samples
[TB] Da gui 14000 samples
[TB] Da gui 15000 samples
[TB] Da gui 16000 samples
[TB] Da gui 17000 samples

-----

[TB] Da nap tong cong 17000 samples
[TB] Dang cho ket qua MLP...

-----

[TB MONITOR] result_valid at time=2086255000, result_id=17

=====
                        KET QUA NHAN DIEN FPGA
=====
-> Detected ID: 17
-> LABEL: [ 17_NGU VAI LON ]
=====
$finish called at time : 2086365 ns : File "C:/Users/Admin/Downloads/dblclckfft-mast
run: Time (s): cpu = 00:10:01 ; elapsed = 00:47:09 . Memory (MB): peak = 6680.773 ; c
```

Hình 5.9. Log post-synthesis functional simulation.

Hình 5.9 ghi nhận kết quả mô phỏng chức năng sau synthesis trên cùng testcase đại diện. Kết quả cho thấy sau khi ánh xạ netlist, thiết kế vẫn bảo toàn hành vi chức năng: result_valid xuất hiện đúng thời điểm và result_id khớp nhãn kỳ vọng.

```
SUMMARY
Total: 300, Pass: 294, Fail: 6, Accuracy: 98.00%

Per-class accuracy:
ID 00: 15/15 = 100.00%
ID 01: 15/15 = 100.00%
ID 02: 15/15 = 100.00%
ID 03: 14/15 = 93.33%
ID 04: 14/15 = 93.33%
ID 05: 15/15 = 100.00%
ID 06: 15/15 = 100.00%
ID 07: 14/15 = 93.33%
ID 08: 15/15 = 100.00%
ID 09: 14/15 = 93.33%
ID 10: 15/15 = 100.00%
ID 11: 15/15 = 100.00%
ID 12: 15/15 = 100.00%
ID 13: 14/15 = 93.33%
ID 14: 15/15 = 100.00%
ID 15: 15/15 = 100.00%
ID 16: 15/15 = 100.00%
ID 17: 15/15 = 100.00%
ID 18: 14/15 = 93.33%
ID 19: 15/15 = 100.00%
```

Hình 5.10. Log batch validation 300 mẫu trên phần cứng FPGA KV260.

Hình 5.10 ghi nhận kết quả trên phần cứng FPGA KV260 với 300 file PCM: 294 mẫu đúng, 6 mẫu sai, timeout = 0 và error = 0, tương ứng accuracy 98,00%. Kết quả này được đối chiếu với mô phỏng RTL validation ở Hình 5.6.

5.3. Kết quả kiểm chứng RTL theo từng module

Mục này trình bày các kết quả mô phỏng RTL đã ghi nhận theo từng giai đoạn xử lý. Các waveform được trình bày theo luồng xử lý pipeline, bao gồm thời điểm nhận mẫu, phát frame, nhân Hamming, xuất FFT/power, tạo Mel/Log-Mel/MFCC, ghi Feature RAM và kích hoạt MLP.

Bảng 5.1. Tổng hợp kết quả kiểm chứng RTL theo từng khối.

STT	Khối	Tín hiệu	Kết quả
1	Input/reset	rst_n, audio_valid, audio_ready, audio_data	Mẫu PCM chỉ được chấp nhận khi handshake hợp lệ.
2	Pre-emphasis	prev_sample, product, out_data	Pre-emphasis bám công thức fixed-point và giữ đúng latency.
3	Framing	sample_count, rd_cnt, out_valid, out_last	Framing tạo frame 512 mẫu với hop 160 và overlap 352 mẫu.
4	Windowing	win_idx, coeff, product, out_data	Windowing đọc đúng ROM Hamming và truyền last tại mẫu cuối frame.
5	FFT/Power	tvalid, tuser/bin_idx, tlast, Re/Im, power	FFT/Power xuất 257 bin không âm, tlast tại bin 256.
6	Mel	bin_idx, mel_idx, acc, out_mel	Mel tạo đủ 40 năng lượng băng tần, accumulator không tràn trong mẫu kiểm thử.
7	Log	msb_pos, lut_addr, out_logmel	Log-Mel tạo log2 Q16.16 bằng LOD/LUT và giữ đúng index.
8	DCT	mel_idx, cep_idx, acc, out_cep	DCT/MFCC xuất 13 hệ số C0-C12 theo đúng thứ tự.

STT	Khối	Tín hiệu	Kết quả
9	Feature RAM	frame_count, wr_addr, rd_addr, mlp_start	Feature RAM chỉ phát mlp_start sau khi đủ 98 frame.
10	MLP	state, feature_addr, hidden_idx, output_idx, result_id	MLP chạy đủ tầng 1274-32-20 và xuất result_id trong miền 0-19.
11	Top-level	sample_count, result_valid, result_id	Top-level hoàn tất luồng PCM-to-result với cơ chế timeout kiểm soát.

Bảng 5.1 cho thấy kiểm chứng RTL không chỉ tập trung vào result_id cuối cùng mà bao phủ toàn bộ chuỗi PCM-to-result. Ba nhóm điều kiện được kiểm tra xuyên suốt là: đúng giao thức điều khiển, đúng số lượng phần tử ở từng tầng và đúng điều kiện hoàn tất xử lý. Nhóm giao thức gồm reset, valid/ready, last và các chỉ số index, nhóm số lượng phần tử gồm 512 mẫu/frame, 257 bin FFT, 40 băng Mel, 13 hệ số MFCC và 1274 đặc trưng, nhóm kết thúc gồm mlp_start, done, result_valid và result_id hợp lệ.

Ý nghĩa của bảng là kiểm tra lỗi theo tầng. Nếu result_id cuối cùng sai, log kiểm chứng cho phép kiểm tra ngược xem sai lệch xuất hiện từ dữ liệu PCM, frame, phổ FFT, Mel/Log/DCT, Feature RAM hay MLP. Cách tổ chức này tránh tình trạng chỉ biết testcase fail ở cuối nhưng không xác định được nguyên nhân kỹ thuật.

Các kết quả trong Bảng 5.1 liên hệ trực tiếp với Bảng 3.3 về chu kỳ xử lý. Mỗi waveform ở các mục 5.3.1 đến 5.3.10 không chỉ xác nhận tín hiệu đúng mà còn xác nhận số chu kỳ thực tế khớp với chu kỳ thiết kế dự kiến. Vì vậy, phần phân tích từng khối dưới đây được sử dụng đối chiếu cả hành vi chức năng và thời gian xử lý.

5.3.1. Kiểm chứng khối Pre-emphasis

Waveform ở Hình 5.11 ghi nhận kết quả mô phỏng của khối pre-emphasis. Các tín hiệu quan trọng gồm in_valid, in_ready, prev_sample, tích $\alpha \times \text{prev_sample}$, out_data và out_valid, nhờ đó có thể kiểm tra trực tiếp phép sai phân fixed-point theo từng mẫu.



Hình 5.11. Waveform kiểm chứng khối Pre-emphasis.

Trong Hình 5.11, in_valid và in_ready xác nhận thời điểm khối pre-emphasis nhận mẫu PCM hợp lệ. out_valid xuất hiện tương ứng với dữ liệu đầu ra sau khi phép tính fixed-point hoàn tất. Việc quan sát đồng thời prev_sample, tích $\alpha \times \text{prev_sample}$ và out_data cho phép kiểm tra trực tiếp quan hệ $y[n] = x[n] - \alpha \times x[n-1]$, đồng thời xác nhận khối không tạo backpressure vì in_ready luôn ở mức sẵn sàng.

Hình 5.11 cho thấy mỗi mẫu PCM chỉ được xử lý khi điều kiện bắt tay hợp lệ. Giá trị prev_sample được cập nhật theo mẫu đã nhận ở chu kỳ trước, tích $\alpha \times \text{prev_sample}$ được làm tròn và dịch thang trước khi trừ khỏi mẫu hiện tại. Dữ liệu đầu ra vì vậy bám công thức tiền nhân cố định và không làm thay đổi thứ tự mẫu trong luồng vào pipeline.

Về chu kỳ xử lý, khối pre-emphasis có latency 1 chu kỳ cho mỗi mẫu hợp lệ và in_ready luôn ở mức sẵn sàng. Điều này phù hợp với Bảng 3.3: thông lượng của khối là 1 mẫu/chu kỳ nên khối tiền nhân không tạo nút thắt cổ chai cho pipeline. Sai số nếu có chỉ đến từ phép nhân hệ số Q1.15, làm tròn và gán về độ rộng 16 bit, không đến từ việc thay đổi thứ tự mẫu.

5.3.2. Kiểm chứng khối Framing

Waveform ở Hình 5.12 ghi nhận hoạt động của khối framing sau pre-emphasis. Các tín hiệu trọng tâm gồm bộ đếm mẫu, bộ đếm đọc frame, out_valid, out_last và trạng thái phát frame 512 mẫu.



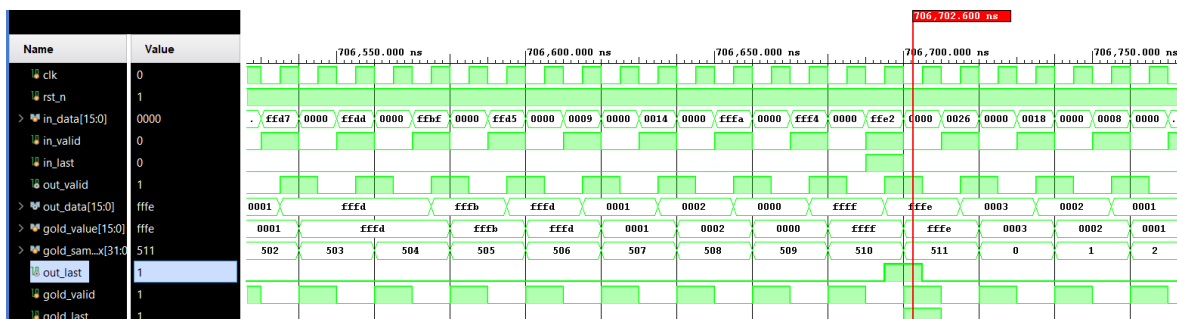
Hình 5.12. Waveform kiểm chứng khối Framing.

Hình 5.12 cho thấy out_valid của khối framing xuất hiện theo từng cụm 512 mẫu. Tín hiệu out_last chỉ được kích hoạt tại mẫu cuối cùng của frame, còn khoảng cách giữa hai lần phát frame phản ánh hop 160 mẫu và overlap 352 mẫu. Kết quả này xác nhận khối framing bảo toàn ranh giới frame trước khi đưa dữ liệu sang windowing.

Về chu kỳ xử lý, frame đầu tiên cần 1024 chu kỳ vì module phải nhận đủ 512 mẫu đầu tiên trước khi xuất 512 mẫu của frame. Các frame tiếp theo cần 672 chu kỳ, gồm 160 chu kỳ nhận mẫu mới theo hop size và 512 chu kỳ xuất frame. Số chu kỳ này xác nhận cơ chế overlap 352 mẫu hoạt động đúng: module không dịch toàn bộ 512 mẫu mà tái sử dụng dữ liệu trong RAM vòng.

5.3.3. Kiểm chứng khối Windowing

Waveform ở Hình 5.13 ghi nhận hoạt động của khối windowing. Nhóm tín hiệu được quan sát gồm win_idx, hệ số Hamming đọc từ ROM, tích nhân mẫu $\times$ hệ số, out_data sau khi làm tròn/dịch phải và out_last tại mẫu cuối frame.



Hình 5.13. Waveform kiểm chứng khối Windowing.

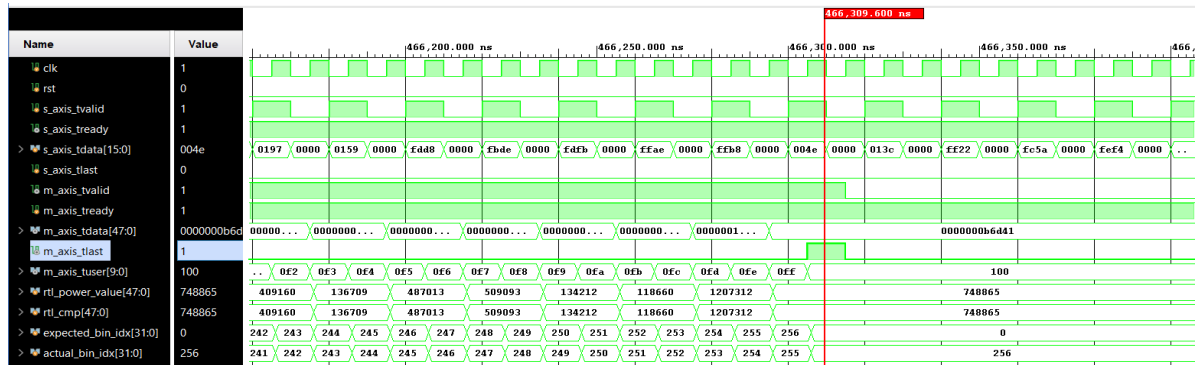
Hình 5.13 cho thấy win_idx tăng tuần tự từ đầu đến cuối frame, hệ số Hamming được truy xuất đúng địa chỉ và tín hiệu out_last được truyền cùng mẫu thứ 512. Giá trị out_data thể hiện kết quả nhân của số trong miền fixed-point, bảo đảm dữ liệu sau windowing vẫn giữ đúng thứ tự mẫu trước khi cấp vào FFT.

Về chu kỳ xử lý, windowing cần 512 chu kỳ cho một frame vì mỗi chu kỳ nhân một mẫu với một hệ số Hamming tương ứng. win_idx tăng tuần tự từ 0 đến 511 và out_last đi cùng mẫu cuối frame, cho thấy địa chỉ hệ số và ranh giới frame được đồng bộ với luồng dữ liệu.

5.3.4. Kiểm chứng khối FFT/Power Spectrum

Waveform ở Hình 5.14 ghi nhận hoạt động của khối fft512_power_axis sau khi nhận đủ một frame đã windowing. Các tín hiệu cần quan sát gồm s_axis_tvalid/s_axis_tlast ở đầu vào FFT, m_axis_tvalid, m_axis_tuser, m_axis_tdata và m_axis_tlast ở đầu ra power spectrum.

Hình 5.14 cho thấy sau độ trễ nội tại của lõi FFT, các bin phổ công suất được phát ra theo thứ tự từ 0 đến 256. Tín hiệu m_axis_tlast xuất hiện tại bin 256, xác nhận thiết kế chỉ giữ nửa phổ không âm của tín hiệu thực. Giá trị power luôn không âm và được truyền sang Mel Filterbank với đúng chỉ số bin.

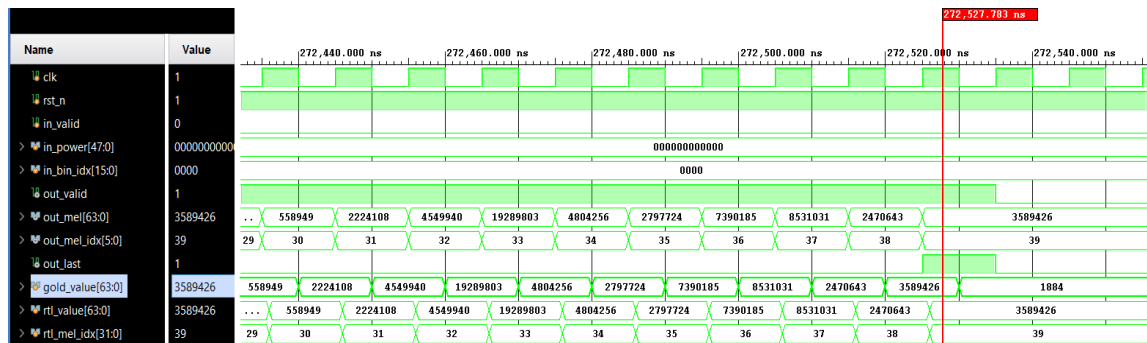


Hình 5.14. Waveform kiểm chứng khối FFT/Power Spectrum.

Về chu kỳ xử lý, độ trễ của khối FFT/Power gồm độ trễ nội bộ LFFT của FFT core cộng 257 chu kỳ để wrapper phát nửa phổ không âm. m_axis_tuser chạy từ 0 đến 256 và m_axis_tlast tại bin 256 xác nhận thiết kế giữ đúng 257 bin cần cho Mel Filterbank. Giá trị power không âm vì được tính theo $\text{Re}^2 + \text{Im}^2$.

5.3.5. Kiểm chứng khối Mel Filterbank

Waveform ở Hình 5.15 ghi nhận quá trình Mel Filterbank nhận 257 bin power và tích lũy năng lượng cho 40 dải Mel. Các tín hiệu cần quan sát gồm bin_idx, mel_idx, accumulator, out_mel, out_valid và out_last.



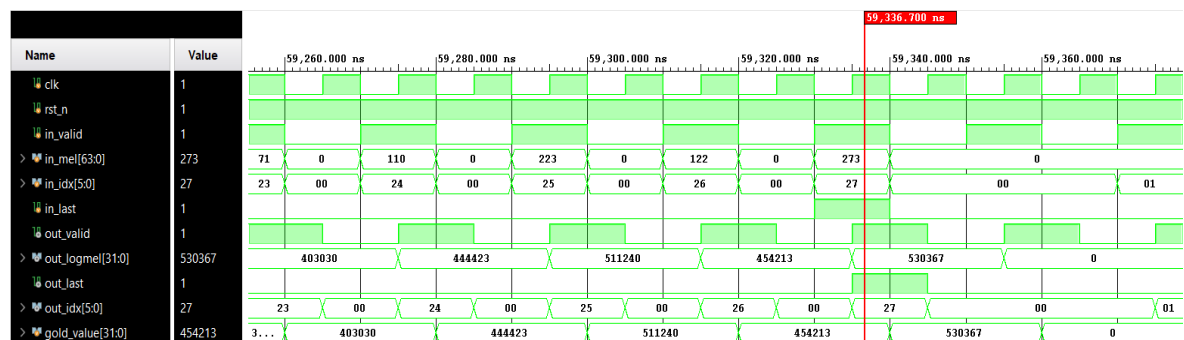
Hình 5.15. Waveform kiểm chứng khối Mel Filterbank.

Hình 5.15 cho thấy các giá trị power được tích lũy vào 40 accumulator Mel theo chỉ số bin. `out_mel_idx` chạy trong dải 0-39 và `out_last` được kích hoạt tại giá trị Mel cuối cùng, xác nhận module xuất đủ 40 năng lượng Mel cho mỗi frame.

Về chu kỳ xử lý, Mel Filterbank cần 297 chu kỳ cho mỗi frame, gồm 257 chu kỳ tích lũy các bin phổ công suất và 40 chu kỳ xuất vector năng lượng Mel. Kết quả này khớp với tổ chức FSM gồm pha tích lũy và pha output. Việc `out_mel_idx` chạy từ 0 đến 39 chứng minh khối Mel xuất đủ 40 dải năng lượng cho bước Log-Mel.

5.3.6. Kiểm chứng khối Log-Mel

Waveform ở Hình 5.16 ghi nhận hoạt động của khối Log-Mel sau khi nhận các giá trị năng lượng Mel. Các tín hiệu trọng tâm gồm `in_mel`, `in_idx`, `msb_pos`, `lut_addr`, `out_logmel`, `out_idx` và `out_last`.



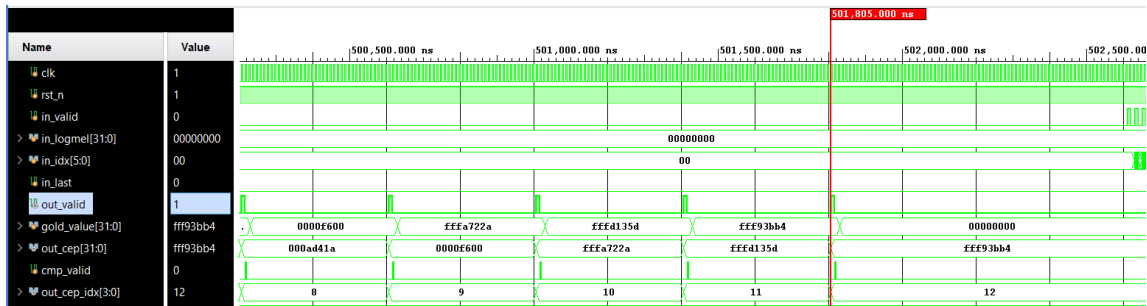
Hình 5.16. Waveform kiểm chứng khối Log-Mel.

Hình 5.16 cho thấy Leading-One Detector xác định đúng vị trí MSB của năng lượng Mel, sau đó các bit định trị được dùng để tạo địa chỉ LUT. out_logmel được phát cùng out_idx tương ứng, bảo toàn thứ tự 40 băng Mel trong khi chuyển dữ liệu sang miền log2 Q16.16.

Về chu kỳ xử lý, Log-Mel xử lý 40 giá trị Mel trong 40 chu kỳ, tương ứng một giá trị mỗi chu kỳ. LOD xác định vị trí bit 1 cao nhất để tạo phần nguyên log2, còn LUT tạo phần thập phân. Sai số của khối này là nguồn sai số có hệ thống so với log floating-point, nhưng được kiểm soát bằng cùng mô hình Python fixed-point.

5.3.7. Kiểm chứng khối DCT/MFCC

Waveform ở Hình 5.17 ghi nhận hoạt động của khối DCT sau khi nhận đủ 40 giá trị Log-Mel. Các tín hiệu cần quan sát gồm mel_idx, cep_idx, accumulator, out_cep_valid, out_cep và out_cep_idx.



Hình 5.17. Waveform kiểm chứng khối DCT/MFCC.

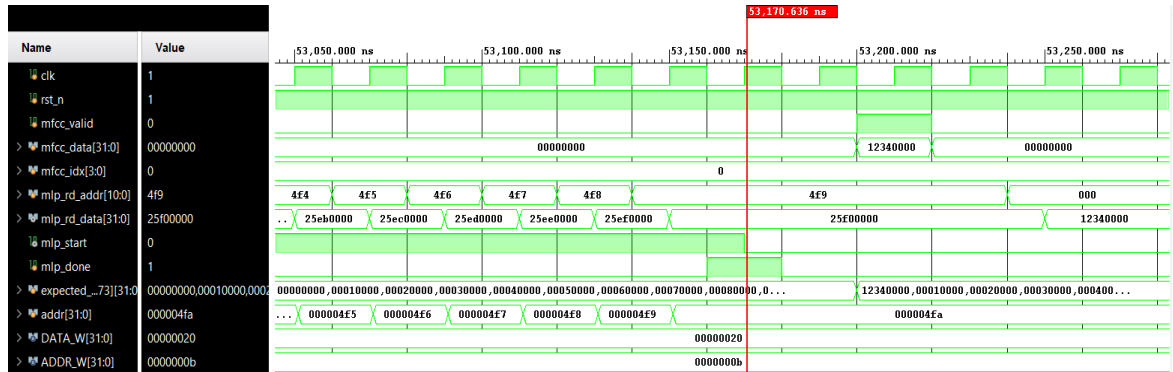
Hình 5.17 cho thấy DCT chỉ bắt đầu sau khi vector 40 Log-Mel đã hoàn tất. cep_idx chạy tuần tự trong dải 0-12 và out_cep_valid xuất hiện tương ứng với từng hệ số MFCC. Thứ tự C0-C12 được bảo toàn để ghi vào Feature RAM, hạn chế sai lệch vị trí đặc trưng khi ghép vector MLP.

Về chu kỳ xử lý, DCT cần 560 chu kỳ cho một frame: 40 chu kỳ nhận vector Log-Mel và 13×40 chu kỳ MAC để tạo 13 hệ số C0-C12. Việc cep_idx tăng đúng từ 0 đến 12 bảo đảm thứ tự hệ số MFCC được giữ nguyên khi ghi sang Feature RAM.

5.3.8. Kiểm chứng Feature RAM Buffer

Waveform ở Hình 5.18 ghi nhận hoạt động của Feature RAM Buffer trong quá trình ghi các hệ số MFCC.

Nhóm tín hiệu được quan sát gồm feature_count, wr_addr, wr_en, mlp_rd_addr và mlp_start.



Hình 5.18. Waveform kiểm chứng Feature RAM Buffer 98 frame.

Hình 5.18 cho thấy controller ghi đủ 1274 hệ số MFCC vào Feature RAM trước khi kích hoạt MLP. Địa chỉ ghi nằm trong miền 0-1273 và thứ tự ghi tương ứng với quy tắc flatten 98×13 .

Về chu kỳ xử lý, Feature RAM phát mlp_start sau 1274 lần ghi hợp lệ, bằng $98 \text{ frame} \times 13$ hệ số. Đây là điều kiện quan trọng vì MLP chỉ có thể suy luận khi toàn bộ context 98 frame đã sẵn sàng. Nếu thiếu hoặc thừa một hệ số, thứ tự flatten sẽ lệch và kết quả phân loại có thể sai dù các khối trước vẫn xuất dữ liệu hợp lệ.

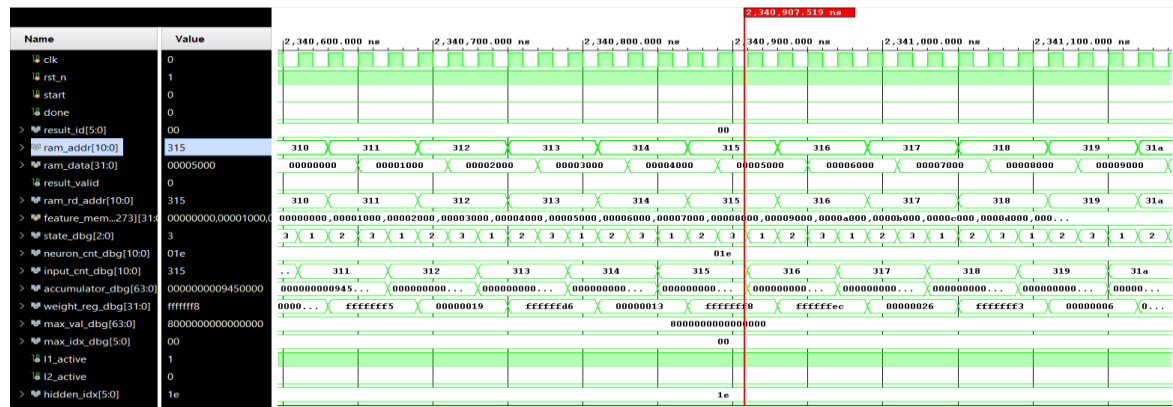
5.3.9. Kiểm chứng MLP Core

Waveform ở Hình 5.19 ghi nhận chu trình suy luận của MLP Core. Các tín hiệu quan trọng gồm state, ram_addr, hidden_idx, output_idx, accumulator, done và result_id.

Hình 5.19 cho thấy MLP Core đọc đủ 1274 đặc trưng, tính 32 nơ-ron tầng ẩn, áp dụng ReLU, tạo 20 logit đầu ra và thực hiện ArgMax. Tín hiệu done/result_valid chỉ xuất hiện sau khi quá trình suy luận hoàn tất.

Về chu kỳ xử lý, MLP Core cần khoảng 122.945 chu kỳ để tính mạng 1274-32-20 theo kiến trúc MAC tuần tự. Con số này phản ánh lựa chọn trade-off của thiết kế: tái sử dụng datapath MAC để giảm DSP và tài nguyên, đổi lại thời gian suy luận dài hơn so với kiến trúc unroll song song toàn bộ ma trận. Với clock 100 MHz, độ trễ

này xấp xỉ 1,229 ms, vẫn nhỏ so với thời lượng danh định của một mẫu âm thanh 1 giây.

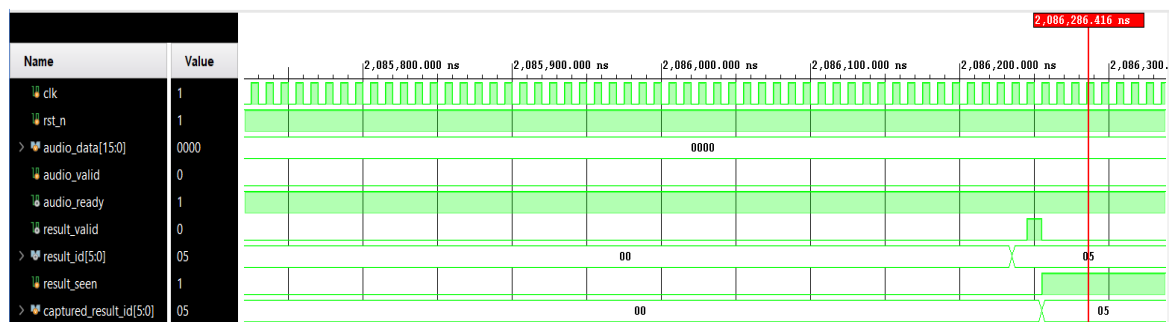


Hình 5.19. Waveform kiểm chứng MLP Core cấu hình 1274-32-20.

5.3.10. Kiểm chứng top-level RTL full-pipeline

Top-level RTL được kiểm chứng bằng testbench tb_keyword_spotting trên Vivado XSim. Mục tiêu của bước kiểm chứng này là quan sát trực tiếp luồng xử lý từ lúc testbench gửi dữ liệu PCM vào DUT đến khi IP phát sinh kết quả nhận diện ở ngõ ra. Với mỗi testcase đại diện cho một từ khóa, testbench đọc file PCM tương ứng, đưa từng mẫu âm thanh vào DUT theo giao thức valid/ready, sau đó chờ tín hiệu result_valid để ghi nhận result_id.

Hình 5.20 minh họa waveform kiểm chứng top-level cho một testcase đại diện. Trên waveform, tín hiệu audio_valid cho biết thời điểm testbench đang gửi PCM, audio_ready cho biết IP có thể nhận mẫu, result_valid và result_id xác nhận kết quả phân loại cuối cùng.



Hình 5.20. Waveform kiểm chứng top-level cho một testcase từ khóa đại diện.

Bước kiểm chứng bằng waveform được thực hiện cho các testcase đại diện của từng từ khóa nhằm bảo đảm rằng top-level không chỉ cho kết quả đúng ở mức thống kê batch, mà còn có luồng điều khiển hợp lệ ở mức tín hiệu. Các tín hiệu cần quan sát gồm clk, rst_n, audio_valid, audio_ready, audio_data, sample_count, result_valid, result_id và expected_id. Kết quả đánh giá định lượng trên tập validation và toàn bộ dataset được trình bày riêng trong bảng tổng hợp kết quả ở phần sau.

Bên cạnh mô phỏng RTL behavioral ở mức pre-synthesis, thiết kế còn được chạy post-synthesis functional simulation trên synthesized netlist. Bước này dùng cùng testbench tb_keyword_spotting nhưng đối tượng mô phỏng là netlist sau synthesis, nhằm xác nhận quá trình tổng hợp không làm thay đổi hành vi chức năng của pipeline.

Kết quả post-synthesis cho thấy netlist vẫn phát result_valid, result_id khớp expected_id trên mẫu kiểm thử đại diện và không ghi nhận timeout/error. Do mô phỏng netlist có thời gian chạy lớn hơn behavioral simulation, kiểm chứng post-synthesis được dùng như bước xác nhận chức năng sau tổng hợp, trong khi batch regression lớn được thực hiện ở mức RTL pre-synthesis.

Về chu kỳ xử lý toàn hệ thống, top-level cần khoảng 208.340 chu kỳ, tương đương khoảng 2,083 ms tại clock 100 MHz trong điều kiện testbench cấp PCM theo handshake. Con số này là thời gian xử lý phần cứng trong mô phỏng, không phải thời lượng vật lý của file âm thanh. Kết quả cho thấy sau khi nhận đủ dữ liệu và flush pipeline, IP có thể phát result_valid trong thời gian hữu hạn, không bị treo hoặc mất trạng thái điều khiển.

Tổng hợp các waveform trong mục 5.3 cho thấy dữ liệu được bảo toàn qua từng ranh giới xử lý: 512 mẫu tạo một frame, 257 bin FFT tạo 40 năng lượng Mel, 40 Log-Mel tạo 13 hệ số MFCC và 98 frame tạo vector 1274 phần tử cho MLP. Nhờ cách phân tích theo tầng này, kết quả cuối result_id không bị trình bày như một giá trị rời rạc, mà được giải thích bởi toàn bộ chuỗi điều khiển và dữ liệu trước đó.

5.4. Kết quả triển khai trên phần cứng FPGA KV260

Sau khi hoàn tất mô phỏng RTL và implementation, thiết kế được triển khai thực nghiệm trên nền tảng phần cứng FPGA Xilinx Kria KV260 để kiểm tra khả năng

hoạt động của IP trong môi trường phần cứng thực tế. Bitstream SoC_wrapper_v4.bit được nạp xuống FPGA thông qua cơ chế fpga_manager trên Linux. Dữ liệu đầu vào là các file PCM đã được chuẩn hóa giống với tập dữ liệu trong mô phỏng RTL, bảo đảm quá trình đối chiếu giữa mô phỏng và board-level được thực hiện trên cùng định dạng dữ liệu.

Hình 5.21 trình bày setup thực nghiệm trên phần cứng FPGA KV260. Hệ thống gồm board KV260, nguồn cấp, kết nối Ethernet/USB-UART và máy tính điều khiển. Máy tính host được dùng để truyền file kiểm thử, nạp bitstream, chạy chương trình C/Linux điều khiển IP và thu thập log kết quả từ terminal.



Hình 5.21. Setup thực nghiệm trên phần cứng FPGA KV260.

Quá trình nạp bitstream được thực hiện trước khi chạy kiểm thử. Log nạp bitstream ghi nhận lệnh copy bitstream vào /lib/firmware và nạp file SoC_wrapper_v4.bit thông qua /sys/class/fpga_manager/fpga0/firmware. Sau khi nạp thành công, chương trình kiểm thử trên Linux truy cập IP qua /dev/uio4, gửi mẫu PCM và chờ kết quả phân loại.

```
root@BeagleBone:/home/debian# echo 0 > /sys/class/fpga_manager/fpga0/flags
root@BeagleBone:/home/debian# cp SoC_wrapper_v3.bit /lib/firmware/
root@BeagleBone:/home/debian# echo SoC_wrapper_v3.bit > /sys/class/fpga_manager/fpga0/firmware
[ 1632.497117] fpga_manager fpga0: writing SoC_wrapper_v3.bit to Xilinx ZynqMP FPGA Manager
```

Hình 5.22. Log nạp bitstream trên KV260.

Với kiểm thử một file PCM đơn lẻ, chương trình điều khiển ghi lần lượt các mẫu âm thanh vào IP, đồng thời theo dõi expected_id, result_id, số mẫu đã gửi và bộ đếm phần cứng hw_count. Khi result_valid được kích hoạt, result_id được đọc về từ thanh ghi kết quả và so sánh với expected_id của file đầu vào. Log trong Hình 5.23 minh họa các testcase đơn lẻ trên board, trong đó mỗi file được gửi đủ 17032 mẫu và kết quả được ghi rõ PASS hoặc FAIL. Kết quả batch test board-level đạt 294/300 mẫu đúng, 6 mẫu sai, không ghi nhận timeout/error, tương ứng accuracy 98,00%. Kết quả này gần với mô phỏng RTL validation 295/300 mẫu đúng và xác nhận IP hoạt động ổn định sau khi được nạp lên KV260.

```

root@beaglebone:/home/debian/Embedded_Software# sudo ./kws_test_debug --pcn-txt ../pcn_full_v2/pcn_09_15.txt
Opened device: /dev/tty04 (HY_IP)
[initial_before_reset] sw_ready=1 result_valid=1 result_id=0 sample_count=17000 last_sample=0x0000 raw=0x00000021 {kws_ready=1 valid=0 pending=0 raw_result_valid=0 soft_reset=0}
Soft reset KWS/bridge...
[after_reset] sw_ready=1 result_valid=0 result_id=0 sample_count=0 last_sample=0x0000 raw=0x00000001 {kws_ready=1 valid=0 pending=0 raw_result_valid=0 soft_reset=0}
Sending samples from ../pcn_full_v2/pcn_09_15.txt ...
sent 1000 samples
sent 2000 samples
sent 3000 samples
sent 4000 samples
sent 5000 samples
[during_send] sw_ready=1 result_valid=0 result_id=0 sample_count=5000 last_sample=0x219f raw=0x00000001 {kws_ready=1 valid=0 pending=0 raw_result_valid=0 soft_reset=0}
sent 6000 samples
sent 7000 samples
sent 8000 samples
sent 9000 samples
sent 10000 samples
[during_send] sw_ready=1 result_valid=0 result_id=0 sample_count=10000 last_sample=0x0000 raw=0x00000001 {kws_ready=1 valid=0 pending=0 raw_result_valid=0 soft_reset=0}
sent 11000 samples
sent 12000 samples
sent 13000 samples
sent 14000 samples
sent 15000 samples
[during_send] sw_ready=1 result_valid=0 result_id=0 sample_count=15000 last_sample=0x0000 raw=0x00000001 {kws_ready=1 valid=0 pending=0 raw_result_valid=0 soft_reset=0}
sent 16000 samples
sent 17000 samples
Done. Total sent samples = 17000
[after_send] sw_ready=1 result_valid=0 result_id=0 sample_count=17000 last_sample=0x0000 raw=0x00000001 {kws_ready=1 valid=0 pending=0 raw_result_valid=0 soft_reset=0}
[final] sw_ready=1 result_valid=1 result_id=9 sample_count=17000 last_sample=0x0000 raw=0x00000021 {kws_ready=1 valid=0 pending=0 raw_result_valid=0 soft_reset=0}
result_valid = 1, result_id = 9

```

Hình 5.23. Log kiểm thử một file PCM trên KV260.

Ngoài kiểm thử đơn lẻ, hệ thống được chạy batch test trên KV260 với 300 file PCM. Mục tiêu của batch test là đánh giá tính ổn định của luồng PS-PL, khả năng đọc/ghi thanh ghi điều khiển, quá trình truyền dữ liệu PCM và độ chính xác result_id trên nhiều mẫu liên tiếp. Log batch được trình bày ở Hình 5.10 và được tổng hợp lại trong Bảng 5.2.

Bảng 5.2. Tổng hợp kết quả đánh giá theo các mức thực nghiệm.

Chỉ số	Phần mềm FP32	Phần mềm fixed-point	RTL pre-synthesis	RTL post-synthesis	FPGA KV260
Miền đánh giá	MLP PyTorch số thực 32-bit	Mô hình fixed-point/fixed-	Mô phỏng RTL behavioral trên XSim	Mô phỏng synthesized netlist	Bitstream chạy trên phần cứng

		point tám RTL			FPGA KV260
Tập dữ liệu	Dataset 20 từ khóa/nhãn, 100 mẫu/nhãn	Cùng dataset	PCM TXT có padding	Một số mẫu đại diện	Batch validation 300 file PCM; full dataset 2.000 file PCM
Số mẫu đánh giá	300	300	300	20	300
Số mẫu đúng trên validation	300	299	295	20	294
Số mẫu sai trên validation	0	1	5	0	6
Timeout/error	Không áp dụng	Không áp dụng	0 timeout, 0 error	0 timeout, 0 error	0 timeout, 0 error
Accuracy trên validation	100,00%	99,67%	98,33%	100%	98,00%
Kết quả toàn bộ dataset gốc	2000/2000 = 100,00%	1999/2000 = 99,95%	1996/2000 = 99,80%	20/20 mẫu đại diện=100%	1995/2000 = 99,75%
Diễn giải	Mốc tham chiếu thuật toán.	Sai khác do lượng tử hóa được kiểm soát trước khi xuất HEX.	RTL chạy full pipeline PCM- MFCC- MLP và tự ghi CSV/sum mary.	Xác nhận synthesis không làm thay đổi chức năng trên mẫu kiểm thử.	Xác nhận PS- PL, UIO/mmap, bitstream và IP trên board hoạt động ổn định.

Bảng 5.2 là bảng kết quả quan trọng nhất của quá trình đánh giá end-to-end vì nó đặt cùng một bài toán lên năm miền kiểm chứng: Python floating-point, Python fixed-point, RTL pre-synthesis, RTL post-synthesis và KV260 board-level. Các miền này không có cùng mục đích. Python floating-point là mốc thuật toán lý tưởng, Python fixed-point kiểm tra tác động lượng tử hóa, RTL pre-synthesis xác nhận pipeline phần cứng đầy đủ, post-synthesis xác nhận netlist sau tổng hợp không đổi chức năng trên mẫu đại diện, board-level kiểm tra IP khi chạy thật qua PS-PL và UIO/mmap.

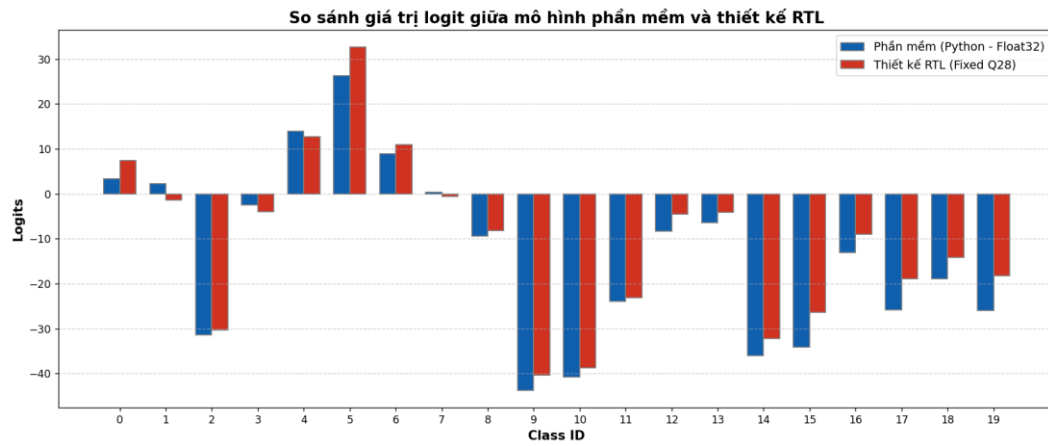
Kết quả validation giảm từ 300/300 ở floating-point xuống 299/300 ở fixed-point, cho thấy lượng tử hóa Q-format chỉ làm sai một mẫu trên tập 300 mẫu. Khi chuyển sang RTL full-pipeline, kết quả còn 295/300, tức phát sinh thêm một số sai khác do toàn bộ chuỗi fixed-point phần cứng gồm MFCC, log xấp xỉ, DCT, Feature RAM và MLP. Tuy nhiên, timeout/error vẫn bằng 0, nên các sai khác này được xem là sai khác phân loại do số học fixed-point chứ không phải lỗi giao thức hoặc treo pipeline.

Ở board-level, kết quả validation đạt $294/300 = 98,00\%$, thấp hơn RTL validation đúng một mẫu. Sau khi chạy thêm toàn bộ dataset 2.000 file trên KV260, hệ thống đạt $1.995/2.000 = 99,75\%$, không ghi nhận timeout/error. Các kết quả này cho thấy luồng triển khai trên KV260 bám sát mô phỏng RTL; chương trình C/Linux, ánh xạ UIO/mmap, cơ chế ghi PCM và đọc result_id hoạt động ổn định trên phần cứng thật.

Khi đánh giá toàn bộ dataset 2.000 mẫu ở mức RTL, hệ thống đạt $1.996/2.000 = 99,80\%$. Đây là kết quả có ý nghĩa thống kê hơn so với các testcase đơn lẻ vì bao phủ toàn bộ 20 class và nhiều mẫu trên mỗi class. Vì vậy, báo cáo sử dụng batch validation 300 file để đối chiếu trực tiếp giữa các mức mô hình, đồng thời sử dụng toàn bộ 2.000 file ở mức RTL và KV260 để đánh giá độ ổn định trên toàn bộ dataset.

Hình 5.24 so sánh 20 giá trị logit ở đầu ra giữa mô hình Python và RTL của một mẫu đại diện ID=05 (Đĩ). Mục đích của hình để quan sát xem lớp có logit lớn nhất ở

hai mức có giữ cùng vị trí hay không. Khi ArgMax không đổi, result_id của RTL sẽ khớp với mô hình tham chiếu dù giá trị logit tuyệt đối có chênh lệch do fixed-point.



Hình 5.24. So sánh giá trị logit giữa mô hình Python và RTL của một mẫu đại diện.

Trục ngang của hình là 20 class ID từ 0 đến 19, trục dọc là giá trị logit đầu ra của MLP. Mỗi class có hai giá trị được so sánh: logit do Python fixed-point/golden model sinh ra và logit do RTL sinh ra. Nếu class có logit lớn nhất ở hai miền giống nhau thì ArgMax giống nhau và result_id cuối cùng khớp, ngay cả khi các giá trị logit tuyệt đối không trùng hoàn toàn.

Đối với một testcase thuộc class ID 05, class đúng tương ứng với nhãn 06_hăm lòn trong Bảng 1.1. Khi logit của class 05 cao nhất, hệ thống dự đoán đúng class 05. Các logit âm ở những class khác không biểu thị lỗi, chúng chỉ là điểm số tuyến tính thấp hơn trong tầng output. Ngược lại, một logit dương cũng không tự động được chọn nếu vẫn nhỏ hơn logit của class khác. Vì vậy, yếu tố quyết định là thứ tự tương đối giữa các logit, đặc biệt là khoảng cách giữa top-1 và top-2.

Sai số giữa mô hình tham chiếu và RTL được tính theo $e_i = \text{logit_RTL},i - \text{logit_ref},i$. Nếu e_i dương, RTL đang tính logit class i lớn hơn mô hình tham chiếu, nếu e_i âm, RTL đang tính nhỏ hơn. Nguồn sai số gồm lượng tử hóa MFCC Q16.16, trọng số Q4.12, bias Q28, làm tròn và dịch bit trong MAC, xấp xỉ log2 bằng LOD/LUT, cũng như sai số tích lũy qua các tầng FFT/Power, Mel, DCT và MLP. Đây là đặc trưng bình thường của hiện thực fixed-point.

MAE cho biết sai số tuyệt đối trung bình của 20 logit, RMSE nhấn mạnh các sai số lớn cục bộ, max error chỉ ra class có độ lệch lớn nhất. Nếu max error xuất hiện ở một class có logit rất thấp, ảnh hưởng đến result_id thường nhỏ. Nếu max error xuất hiện gần class top-1 hoặc top-2, nguy cơ đổi ArgMax cao hơn. Do đó, khi phân tích một testcase sai, cần quan sát đồng thời sai số logit và khoảng cách quyết định giữa hai class có logit cao nhất.

Hình 5.24 minh họa định tính cho quan hệ giữa sai số số học và kết quả phân loại. Accuracy của hệ thống vẫn được kết luận bằng batch expected_id/result_id ở Bảng 5.2, còn hình logit giúp giải thích vì sao một testcase có thể giữ nguyên hoặc thay đổi result_id khi chuyển từ Python sang RTL.

5.5. Tài nguyên, timing và công suất

Các số liệu từ báo cáo synthesis, implementation, timing summary và power estimate của Vivado cho top-level keyword_spotting_top cho biết thiết kế có thể ánh xạ được lên FPGA.

Utilization

Post-Synthesis | Post-Implementation

Graph | Table

Resource	Estimation	Available	Utilization...
LUT	26341	117120	22.49
LUTRAM	1276	57600	2.22
FF	21880	234240	9.34
BRAM	74	144	51.39
DSP	131	1248	10.50
IO	27	189	14.29

Utilization

Post-Synthesis | Post-Implementation

Graph | Table

Resource	Utilization	Available	Utilization...
LUT	26053	117120	22.24
LUTRAM	1173	57600	2.04
FF	21879	234240	9.34
BRAM	74	144	51.39
DSP	131	1248	10.50
IO	27	189	14.29

Hình 5.25. Thống kê tài nguyên phần cứng sau implementation.

5.5.1. Tài nguyên phần cứng sau implementation

Hình 5.25 cho thấy kết quả sử dụng tài nguyên sau synthesis và sau implementation. Sau synthesis, thiết kế sử dụng 26.341 LUT, 1.276 LUTRAM, 21.880 FF, 74 BRAM, 131 DSP và 27 IO. Sau implementation, các giá trị được tối ưu nhẹ còn 26.053 LUT, 1.173 LUTRAM, 21.879 FF, 74 BRAM, 131 DSP và 27 IO.

Tỷ lệ sử dụng sau implementation lần lượt là 22,24% LUT, 2,04% LUTRAM, 9,34% FF, 51,39% BRAM, 10,50% DSP và 14,29% IO.

Bảng 5.3. Thống kê tài nguyên phần cứng sử dụng (Post-Implementation).

Tài nguyên	Used	Available	Utilization	Nhận xét
LUT	26.053	117.120	22,24%	Logic điều khiển, datapath và ROM suy luận
LUTRAM	1.173	57.600	2,04%	ROM nhỏ, bảng tra và bộ nhớ phân tán
FF	21.879	234.240	9,34%	Thanh ghi pipeline, FSM và buffer điều khiển
BRAM	74	144	51,39%	ROM hệ số, bảng tra và bộ đệm đặc trưng
DSP48E2	131	1.248	10,50%	Phép nhân và MAC của MFCC/MLP
IO	27	189	14,29%	Các cổng giao tiếp top-level và tín hiệu debug

Từ Bảng 5.3, BRAM là nhóm tài nguyên có tỷ lệ sử dụng cao nhất với 74/144 khối, tương ứng 51,39%. Nguyên nhân là thiết kế cần lưu nhiều loại dữ liệu cố định và trung gian như hệ số Hamming, hệ số Mel, hệ số DCT, LUT log, trọng số/bias của MLP và Feature RAM 1274 phần tử. Việc dùng BRAM ở mức này là phù hợp với mục tiêu đưa hệ số vào bộ nhớ nội thay vì tính lại bằng logic tổ hợp phức tạp.

DSP48E2 sử dụng 131/1.248 khối, tương ứng 10,50%. Mặc dù hệ thống có nhiều phép nhân trong FFT/Power, Mel, DCT và MLP, tỷ lệ DSP vẫn thấp nhờ các khối DCT và MLP được tổ chức theo hướng MAC tuần tự. Đây là lựa chọn đánh đổi giữa tài nguyên và latency: nếu unroll nhiều phép nhân hơn, latency có thể giảm nhưng DSP và routing sẽ tăng mạnh hơn.

LUT và FF lần lượt chiếm 22,24% và 9,34%, cho thấy logic điều khiển, FSM, datapath phụ trợ và các ROM nhỏ vẫn nằm trong giới hạn của KV260. Sự chênh lệch giữa LUT và FF cũng phản ánh thiết kế dùng nhiều logic tổ hợp cho điều khiển, chọn dữ liệu và bảng tra, trong khi số thanh ghi pipeline chưa chiếm tỷ lệ cao.

5.5.2. Kết quả timing sau implementation

Design Timing Summary			
Setup	Hold	Pulse Width	
Worst Negative Slack (WNS): 0.225 ns	Worst Hold Slack (WHS): 0.010 ns	Worst Pulse Width Slack (WPWS): 4.238 ns	
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	
Total Number of Endpoints: 52394	Total Number of Endpoints: 52394	Total Number of Endpoints: 23528	
All user specified timing constraints are met.			

Hình 5.26. Timing summary sau implementation.

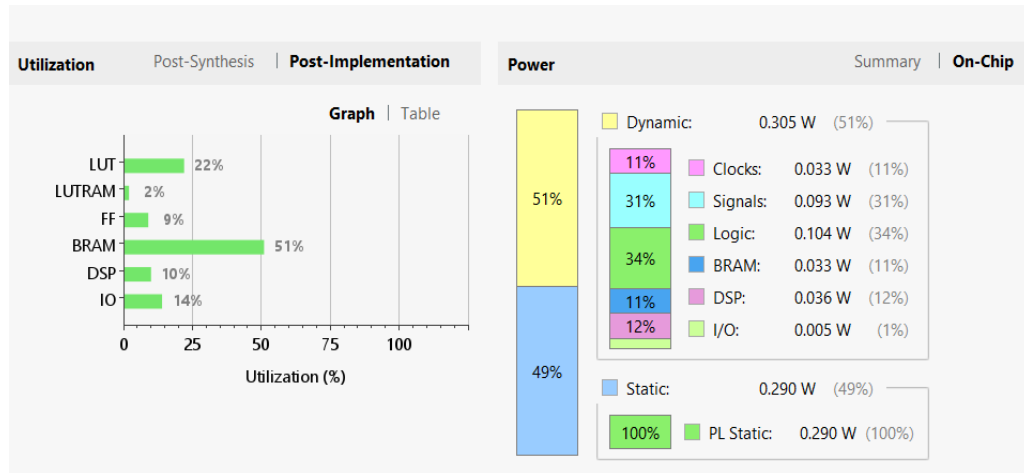
Hình 5.26 ghi nhận kết quả timing sau implementation. Thiết kế đạt WNS = 0,225 ns, TNS = 0 ns và không có endpoint vi phạm setup. Đồng thời, WHS = 0,010 ns và WPWS = 4,238 ns đều dương, cho thấy các ràng buộc setup, hold và pulse width đều được đáp ứng. Với ràng buộc chu kỳ clock 10 ns, đường critical path tương ứng khoảng 9,775 ns, suy ra Fmax xấp xỉ 102,30 MHz.

5.5.3. Phân tích công suất tiêu thụ

Power	Summary On-Chip
Total On-Chip Power:	0.595 W
Junction Temperature:	26.4 °C
Thermal Margin:	58.6 °C (25.0 W)
Effective θ_{JA} :	2.3 °C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low
Implemented Power Report	

Hình 5.27. Tổng công suất on-chip ước lượng bởi Vivado.

Hình 5.27 ghi nhận power estimate của Vivado với tổng công suất on-chip ước lượng 0,595 W, nhiệt độ junction 26,4 °C và thermal margin 58,6 °C. Vì mức confidence của báo cáo là Low, giá trị này chỉ nên dùng như ước lượng ban đầu cho phân tiêu thụ trong chip, không thay thế số đo công suất thực tế của toàn bộ kit.



Hình 5.28. Phân bố công suất động và tĩnh của thiết kế.

Hình 5.28 cho thấy tổng công suất động là 0,305 W, chiếm 51% tổng công suất on-chip, trong khi công suất tĩnh là 0,290 W, chiếm 49%. Trong phần công suất động, logic tiêu thụ 0,104 W, signals tiêu thụ 0,093 W, DSP tiêu thụ 0,036 W, clocks tiêu thụ 0,033 W, BRAM tiêu thụ 0,033 W và I/O tiêu thụ 0,005 W. Phân bố này cho thấy logic/datapath và mạng tín hiệu là hai thành phần động chính của thiết kế.

Phân tích công suất trong khóa luận này được giới hạn ở power estimate sau implementation. Kết quả phản ánh công suất on-chip theo mô hình hoạt động do Vivado suy luận, chưa bao gồm đầy đủ PS/Linux, DDR, nguồn cấp, giao tiếp ngoại vi và phần mềm điều khiển. Vì vậy, các số liệu trong mục này được dùng để đánh giá mức tiêu thụ on-chip của thiết kế, không dùng như số đo công suất toàn bộ board.

Trong báo cáo hậu implementation, tổng công suất on-chip được ước lượng là 0,595 W. Phần dynamic chiếm 0,305 W và phần static chiếm 0,290 W. Các con số này phù hợp để so sánh giữa các thiết kế RTL của cùng đề tài, nếu so với hệ thống khác hoặc với số đo board thật, cần nêu rõ điều kiện đo và phạm vi công suất được tính.

Mức sử dụng tài nguyên sau implementation cho thấy thiết kế dùng 22,24% LUT, 2,04% LUTRAM, 9,34% FF, 51,39% BRAM, 10,50% DSP và 14,29% IO. BRAM là nhóm tài nguyên có tỷ lệ sử dụng cao nhất do hệ thống cần lưu ROM hệ số, bảng tra và bộ đệm đặc trưng. Trong khi đó, tỷ lệ DSP chỉ chiếm 10,50% nhờ kiến

trúc MAC tuần tự ở các khối DCT và MLP, giúp tránh triển khai toàn bộ phép nhân song song.

Đặc thù của hệ thống KWS trong đề tài là dữ liệu âm thanh 16 kHz được xử lý theo frame 512 mẫu, hop 160 mẫu và quyết định phân loại chỉ xuất hiện sau khi đủ 98 frame. Vì vậy MLP không hoạt động liên tục ở mọi chu kỳ clock mà chỉ được kích hoạt sau khi Feature RAM tích lũy đủ vector 1274 phần tử. Cách tổ chức này làm giảm mật độ hoạt động của khối phân loại so với kiến trúc suy luận hoàn toàn song song, đồng thời giữ mức sử dụng DSP ở 131/1.248 khối. Tuy nhiên, các khối front-end như pre-emphasis, framing, windowing, FFT/Power, Mel, Log-Mel và DCT vẫn tạo hoạt động định kỳ theo từng frame nên tiếp tục đóng góp vào công suất động của pipeline.

Khi so sánh với các công trình liên quan, giá trị 0,595 W của đề tài chỉ nên được hiểu là power estimate cho phiên bản full pipeline PCM-MFCC-Feature RAM-MLP trên KV260. Các công trình chỉ tăng tốc mạng nơ-ron hoặc chỉ hiện thực front-end MFCC có phạm vi phân cứng khác nhau, vì vậy power không được dùng để xếp hạng trực tiếp nếu điều kiện đo và phạm vi IP không tương đồng.

Bảng 5.4. Tóm tắt kết quả hậu implementation của thiết kế.

Chỉ số	Giá trị hiện có	Ý nghĩa
LUT	26.053 (22,24%)	Logic điều khiển, datapath và ROM suy luận sau implementation
LUTRAM	1.173 (2,04%)	ROM nhỏ và bảng tra ánh xạ sang logic phân tán
FF	21.879 (9,34%)	Thanh ghi pipeline, FSM và điều khiển
BRAM	74 (51,39%)	ROM hệ số, bảng tra và Feature RAM, là nhóm tài nguyên có tỷ lệ sử dụng cao nhất
DSP	131 (10,50%)	Phép nhân/MAC trong MFCC và MLP
IO	27 (14,29%)	Cổng giao tiếp top-level và tín hiệu debug
Timing	WNS = 0,225 ns, TNS = 0, WHS = 0,010 ns, WPWS = 4,238 ns	Không có endpoint vi phạm setup/hold/pulse width

Chỉ số	Giá trị hiện có	Ý nghĩa
Fmax	$\approx 102,30$ MHz	Suy ra từ critical path khoảng 9,775 ns với clock mục tiêu 10 ns
On-chip power	0,595 W	Ước lượng công suất on-chip của Vivado
Dynamic / Static power	0,305 W / 0,290 W	Công suất động chiếm 51%, công suất tĩnh chiếm 49%

Bảng 5.4 cho thấy thiết kế đáp ứng timing mục tiêu 100 MHz với WNS dương 0,225 ns và TNS bằng 0. Điều này nghĩa là đường tới hạn vẫn còn dư 0,225 ns so với chu kỳ 10 ns, critical path xấp xỉ 9,775 ns nên Fmax ước lượng khoảng 102,30 MHz. Biên timing này không quá lớn, nhưng đủ để kết luận thiết kế đạt ràng buộc clock mục tiêu trong phiên bản implementation hiện tại.

Công suất on-chip 0,595 W là kết quả ước lượng của Vivado, không phải số đo trực tiếp của toàn bộ kit. Dynamic power 0,305 W chiếm 51%, còn static power 0,290 W chiếm 49%. Tỷ lệ dynamic và static gần cân bằng cho thấy workload KWS đã tạo hoạt động chuyển mạch đáng kể trong PL, nhưng công suất nền của chip vẫn chiếm phần lớn do bản chất FPGA/MPSoC. Khi so sánh với hệ thống khác, cần nêu rõ đây là on-chip estimate và không bao gồm đầy đủ PS/Linux, DDR, nguồn cấp và ngoại vi.

5.6. So sánh với các công trình liên quan

So sánh được thực hiện theo nguyên tắc không xếp hạng trực tiếp accuracy giữa các dataset khác nhau. Các công trình tiếng Anh dùng Google Speech Commands hoặc wake-word dataset không cùng hệ quy chiếu với tập từ khóa nhạy cảm tiếng Việt tự xây dựng. Vì vậy, accuracy chỉ là thông tin tham khảo, các chỉ số có giá trị đối chiếu chính là nền tảng, phạm vi pipeline, mô hình, tài nguyên, Fmax và power.

Khi một công trình có ít DSP hơn hoặc Fmax cao hơn, sự khác biệt về mô hình và phạm vi IP được xem là yếu tố phân tích chính. Điểm mạnh của đề tài nằm ở việc hiện thực chuỗi PCM-MFCC-Feature RAM-MLP cho 20 từ khóa/nhân tiếng Việt trên

KV260 và kiểm chứng được từ Python tới RTL/board, không phải tuyên bố vượt trội tuyệt đối so với mọi hệ thống KWS khác.

Bảng 5.5. So sánh định lượng với các công trình liên quan.

Công trình	Ngôn ngữ	Nền tảng	Mô hình / phạm vi	Accuracy/ Fmax	Tài nguyên / Power
Phan Đình Duy et al.	Lệnh tiếng Việt	DE2 + Nios II	FFT, Mel, VQ, bài toán lệnh robot	80-100% theo lệnh, Fmax không công bố	Tài nguyên và power không công bố
Dao et al.	Front-end tiếng nói	Virtex-6 / Spartan-3A DSP	MFCC IP, chỉ đối chiếu chuỗi MFCC	Không có classifier, Fmax không công bố	Tài nguyên và power không công bố
He et al.	GSC, tiếng Anh	Zynq-7020	Modified TENet, int8, accelerator NN, MFCC bằng software	95,36%, 50 MHz	5.575 LUT / 2.631 FF, 64 DSP / 8 BRAM, 0,209 W
Bae et al.	GSC: WUW + lệnh	ZCU104	DS-BTNN, end-to-end Mel spectrogram-classifier	97,1% / 90,5%, 170 MHz	27,3K LUT / 25,5K FF, 31 DSP, power không công bố
Đề tài	20 từ khóa/nhân tiếng Việt	Xilinx KV260	MFCC fixed-point + MLP 1274-32-20, full PCM-MFCC-context-MLP	98,00% board validation; 99,75% board full dataset; 99,80% RTL full dataset; 102,30 MHz	26.053 LUT / 21.879 FF, 131 DSP / 74 BRAM, 0,595 W estimate

Bảng 5.5 chỉ dùng để đối chiếu phạm vi kỹ thuật, không xếp hạng trực tiếp vì các công trình khác nhau về dataset, ngôn ngữ, số lớp và nền tảng triển khai.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận chung

Khóa luận đã hoàn thành các chỉ tiêu chính đặt ra trong đề cương, bao gồm xây dựng bài toán KWS 20 từ khóa/nhãn tiếng Việt, thiết kế IP xử lý chuỗi PCM-MFCC-Feature Buffer-MLP bằng RTL fixed-point, tích hợp lên nền tảng FPGA và kiểm chứng đa mức từ mô hình phần mềm, mô phỏng RTL đến board-level. Kết quả đạt được cho thấy thiết kế đáp ứng mục tiêu chức năng, đạt timing 100 MHz sau implementation và có thể xử lý batch PCM thật mà không ghi nhận timeout/error trong các lần kiểm thử chính.

Kết quả thực nghiệm cho thấy mô hình huấn luyện trên phần mềm với số floating-point 32-bit đạt 300/300 mẫu đúng trên validation và 2.000/2.000 mẫu đúng trên toàn bộ dataset. Sau lượng tử hóa fixed-point, mô hình đạt 299/300 trên validation và 1.999/2.000 trên toàn bộ dataset. Ở mức mô phỏng RTL pre-synthesis, hệ thống đạt 295/300 trên validation và 1.996/2.000 trên toàn bộ dataset. Khi triển khai trên KV260, hệ thống đạt 294/300 trên batch validation và 1.995/2.000 trên toàn bộ dataset, không ghi nhận timeout/error.

Về mặt thuật toán, đề tài đã xây dựng được mô hình tham chiếu phần mềm, quy trình trích xuất MFCC thống nhất với phần cứng và mô hình MLP cho bài toán phân loại 20 từ khóa/nhãn tiếng Việt. Trọng số, bias và các hệ số xử lý tín hiệu được lượng tử hóa sang fixed-point, trong đó bias được đồng bộ ở miền Q28 nhằm giảm sai lệch giữa mô hình fixed-point và RTL.

Về mặt phần cứng, các module chính của pipeline đã được thiết kế, tích hợp và kiểm chứng theo nhiều mức. Waveform từng khối cho thấy giao thức valid/ready, chỉ số index và tín hiệu last được giữ đúng. Ở mức top-level, testbench có thể chạy nhiều file PCM liên tiếp, tự ghi summary/CSV và thống kê timeout/error.

Đóng góp chính của khóa luận được thể hiện ở ba điểm. Thứ nhất, đề tài xây dựng bài toán nhận diện 20 từ khóa/nhãn nháy cảm tiếng Việt trong bối cảnh lớp học, trong đó dữ liệu, nhãn và quy trình đánh giá được tổ chức phù hợp với bài toán Keyword Spotting. Thứ hai, đề tài thiết kế và hiện thực một pipeline end-to-end từ

PCM đến kết quả phân loại, gồm chuỗi MFCC, Feature Buffer 98 frame và MLP 1274-32-20. Thứ ba, toàn bộ hệ thống được triển khai theo số học fixed-point trên FPGA, bao gồm lượng tử hóa hệ số, đồng bộ thang đo, tổ chức ROM/RAM và datapath RTL phù hợp với tài nguyên phần cứng. Bên cạnh các kết quả về accuracy, khóa luận còn báo cáo tài nguyên, timing, power estimate sau implementation và kết quả kiểm thử trên phần cứng FPGA KV260, tạo cơ sở để tiếp tục mở rộng hệ thống sang luồng âm thanh trực tiếp.

Bên cạnh kết quả hiện thực và kiểm chứng trong khóa luận, nhóm cũng đã hoàn thiện một bài báo khoa học từ nghiên cứu của đề tài và gửi tới hội nghị International Conference on Advanced Technologies for Communications (ATC 2026). Tại thời điểm hoàn thiện khóa luận, bài báo đang trong quá trình chờ kết quả phản biện. Bản thảo bài báo đã gửi được đính kèm trong Phụ lục A nhằm cung cấp thêm minh chứng về hướng công bố khoa học và khả năng mở rộng kết quả nghiên cứu của đề tài.

6.2. Hạn chế của đề tài

Về dữ liệu, dataset hiện gồm 20 từ khóa/nhãn và đủ dùng cho quá trình huấn luyện, mô phỏng và kiểm thử trong phạm vi khóa luận. Tuy nhiên, số lượng người nói, điều kiện thu âm và bối cảnh nhiễu vẫn còn giới hạn so với môi trường lớp học thật. Vì vậy, kết quả accuracy chưa nên được xem là đại diện cho mọi tình huống sử dụng thực tế.

Về kiến trúc mô hình, MLP 1274-32-20 có ưu điểm là đơn giản, dễ lượng tử hóa và phù hợp với datapath MAC tuần tự, nhưng đây vẫn là mô hình truyền thẳng với cửa sổ ngữ cảnh cố định. Mô hình không có cơ chế nhớ hồi tiếp hoặc convolution theo thời gian nên khả năng thích nghi với từ khóa bị nói quá nhanh, quá chậm hoặc bị kéo dài còn hạn chế. Ngoài ra, Feature RAM trong phiên bản hiện tại hoạt động theo block 98 frame, cơ chế sliding window liên tục và cập nhật frame mới theo thời gian thực chưa được hoàn thiện ở mức sản phẩm.

Về sai số fixed-point, các phép nhân, cộng, dịch bit, làm tròn, bão hòa và xấp xỉ logarit bằng LOD/LUT đều tạo sai số so với mô hình floating-point. Đối với phần

lớn mẫu kiểm thử, sai số này không làm thay đổi lớp ArgMax. Tuy nhiên, ở những mẫu nằm gần ranh giới quyết định giữa hai lớp, chênh lệch nhỏ ở logit có thể dẫn đến kết quả phân loại khác nhau.

Về công suất, phần công suất trong báo cáo dựa trên ước lượng on-chip của Vivado sau implementation. Kết quả này có giá trị để đánh giá thiết kế trong phạm vi chip và so sánh tương đối giữa các cấu hình cùng điều kiện, nhưng chưa phản ánh đầy đủ toàn bộ hệ thống khi bao gồm PS, DDR, nguồn cấp và ngoại vi.

6.3. Hướng phát triển

6.3.1. Mở rộng và chuẩn hóa dữ liệu

Dataset cần được mở rộng theo số lượng người nói, giới tính, độ tuổi, vùng miền, tốc độ phát âm, khoảng cách thu và điều kiện nhiễu. Việc chia tập dữ liệu nên được thực hiện theo người nói để tránh cùng một giọng xuất hiện đồng thời trong tập huấn luyện và tập kiểm thử. Bên cạnh 20 lớp từ khóa hiện tại, các tập silence, unknown và âm thanh nền nên được bổ sung để đánh giá tỷ lệ báo động giả và khả năng từ chối đầu vào ngoài tập huấn luyện. Khi đó, ngoài accuracy, hệ thống cần báo cáo thêm precision, recall, F1-score, false accept rate và false reject rate trên một tập test độc lập và trên dữ liệu thu trực tiếp trong lớp học.

6.3.2. Mở rộng số lượng từ khóa và cập nhật hệ thống

Khi bổ sung từ khóa mới, hệ thống không chỉ thay đổi bảng ánh xạ nhãn mà phải cập nhật đồng bộ cả mô hình phần mềm và phần cứng. Quy trình đề xuất gồm thu thập và gán nhãn dữ liệu cho các lớp mới, huấn luyện lại mô hình trên toàn bộ lớp cũ và lớp mới, thay số nơ-ron tầng đầu ra từ 20 thành N_{class} , cập nhật ma trận W_2 từ 32×20 thành $32 \times N_{class}$ và vector b_2 tương ứng. Ở mức RTL, cần cập nhật class map, tham số OUTPUT_SIZE, độ rộng result_id theo $\log_2(N_{class})$ khi cần, sau đó xuất lại các ROM trọng số và bias. Thiết kế phải được synthesis, implementation và tạo bitstream lại, đồng thời chạy lại toàn bộ luồng kiểm chứng phần mềm-fixed-point-RTL-FPGA để bảo đảm các lớp cũ không bị suy giảm và các lớp mới được nhận diện đúng.

6.3.3. Nâng cấp mô hình và tối ưu kiến trúc phần cứng

Các nghiên cứu tiếp theo có thể đối chiếu MLP hiện tại với 1D-CNN, DS-CNN hoặc các kiến trúc nhỏ khai thác tốt hơn cấu trúc thời gian-tần số của MFCC. Các kỹ thuật pruning, lượng tử hóa độ rộng thấp và chia sẻ tham số có thể được khảo sát để giảm số phép MAC và dung lượng ROM. Về phần cứng, cần tối ưu nút thắt latency tại MLP Core bằng cách tăng mức song song có kiểm soát, phân chia MAC theo cụm hoặc pipeline các bước đọc ROM, nhân và tích lũy, nhưng vẫn phải giữ tài nguyên DSP, BRAM và timing trong giới hạn KV260. Feature Buffer cũng có thể được chuyển từ cơ chế block 98 frame sang sliding window để tạo kết quả liên tục thay vì chờ hoàn tất từng đoạn âm thanh độc lập.

6.3.4. Tích hợp âm thanh thời gian thực và hoàn thiện đánh giá

Ở mức hệ thống, hướng phát triển quan trọng là tích hợp microphone hoặc audio codec để nhận luồng âm thanh trực tiếp, bổ sung Voice Activity Detection nhằm loại khoảng lặng và sử dụng AXI-Stream hoặc DMA khi cần tăng thông lượng truyền dữ liệu giữa PS và PL. Hệ thống cần được đánh giá trong môi trường thực với nhiều khoảng cách, mức nhiễu và số người nói đồng thời; công suất toàn board cũng nên được đo bằng thiết bị thay vì chỉ dựa trên power estimate của Vivado. Khi triển khai ứng dụng, kết quả KWS cần đi kèm ngưỡng tin cậy, cơ chế xác nhận, lưu log tối thiểu và chính sách bảo vệ dữ liệu âm thanh. Ngoài lớp học, kiến trúc có thể được mở rộng cho các bài toán cảnh báo âm thanh tại thiết bị biên, nhưng kết quả nhận diện chỉ nên được sử dụng như tín hiệu hỗ trợ giám sát, không phải bằng chứng kết luận độc lập.

TÀI LIỆU THAM KHẢO

- [1] A. Thapa, J. Cohen, S. Guffey, and A. Higgins-D'Alessandro, "A review of school climate research," *Review of Educational Research*, vol. 83, no. 3, pp. 357-385, 2013, doi: 10.3102/0034654313483907.
- [2] G. G. Taylor and S. W. Smith, "Teacher Reports of Verbal Aggression in School Settings Among Students With Emotional and Behavioral Disorders," *Journal of Emotional and Behavioral Disorders*, vol. 27, no. 1, pp. 52-64, 2019, doi: 10.1177/1063426617739638.
- [3] S. B. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 28, no. 4, pp. 357-366, 1980, doi: 10.1109/TASSP.1980.1163420.
- [4] L. R. Rabiner and B. H. Juang, *Fundamentals of Speech Recognition*. Prentice Hall, 1993.
- [5] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed. Pearson, 2006.
- [6] Nguyễn Quốc Trung, *Xử lý tín hiệu số*. NXB Đại học Quốc gia Hà Nội, 2013.
- [7] Huỳnh Hữu Tuệ, *Xử lý tín hiệu số và ứng dụng*. NXB Giáo dục, 2008.
- [8] G. Chen, C. Parada, and G. Heigold, "Small-footprint keyword spotting using deep neural networks," *Proc. ICASSP*, pp. 4087-4091, 2014. Truy xuất từ: <https://research.google/pubs/small-footprint-keyword-spotting-using-deep-neural-networks/>.
- [9] T. N. Sainath and C. Parada, "Convolutional neural networks for small-footprint keyword spotting," *Proc. INTERSPEECH*, pp. 1478-1482, 2015. Truy xuất từ: https://www.isca-archive.org/interspeech_2015/sainath15b_interspeech.html.
- [10] S. Arik et al., "Convolutional recurrent neural networks for small-footprint keyword spotting," *arXiv:1703.05390*, 2017.
- [11] Y. Zhang, N. Suda, L. Lai, and V. Chandra, "Hello Edge: Keyword Spotting on Microcontrollers," *arXiv:1711.07128*, 2017.

- [12] P. Warden, “Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition,” arXiv:1804.03209, 2018. Truy xuất từ: <https://arxiv.org/abs/1804.03209>.
- [13] S. Bae, H. Kim, S. Lee, and Y. Jung, “FPGA implementation of keyword spotting system using depthwise separable binarized and ternarized neural networks,” *Sensors*, vol. 23, no. 12, art. 5701, 2023. Truy xuất từ: <https://www.mdpi.com/1424-8220/23/12/5701>.
- [14] K. He et al., “A configurable accelerator for keyword spotting based on small-footprint temporal efficient neural network,” *Electronics*, vol. 11, no. 16, art. 2571, 2022.
- [15] P. D. Duy, V. D. Lung, N. Q. D. Trang, and N. C. Toan, “Speech Recognition on Robot Controller Implemented on FPGA,” *J. Automation and Control Engineering*, vol. 1, no. 3, pp. 274-278, 2013.
- [16] V.-L. Dao, V.-D. Nguyen, H.-D. Nguyen, and V.-P. Hoang, “Hardware Implementation of MFCC Feature Extraction for Speech Recognition on FPGA,” *Advances in Intelligent Systems and Computing*, vol. 538, pp. 248-254, 2017.
- [17] L. D. Pham et al., “An ASIC-Based Artificial Neural Network Applied Real-time Speech Recognition SOPC,” *J. Science and Technology: ICT*, vol. 2, no. 1.
- [18] T. C. Nguyen et al., “A High Performance Dynamic ASIC-Based Audio Signal Feature Extraction (MFCC),” *Proc. ACOMP*, pp. 113-120, 2016.
- [19] D. Lin, S. Talathi, and M. Annapureddy, “Fixed point quantization of deep convolutional networks,” *Proc. ICML*, 2016.
- [20] J. N. Mitchell, “Computer multiplication and division using binary logarithms,” *IRE Trans. Electronic Computers*, vol. EC-11, no. 4, pp. 512-517, 1962.
- [21] AMD, Kria KV260 Vision AI Starter Kit User Guide, UG1089. Truy xuất từ: <https://docs.amd.com/r/en-US/ug1089-kv260-starter-kit>.
- [22] AMD, Zynq UltraScale+ MPSoC Technical Reference Manual, UG1085. Truy xuất từ: <https://docs.amd.com/r/en-US/ug1085-zynq-ultrascale-trm>.

- [23] AMD, Vivado Design Suite User Guide: Synthesis, UG901. Truy xuất từ: <https://docs.amd.com/r/en-US/ug901-vivado-synthesis>.
- [24] AMD, Vivado Design Suite User Guide: Implementation, UG904. Truy xuất từ: <https://docs.amd.com/r/en-US/ug904-vivado-implementation>.
- [25] AMD, Vivado Design Suite User Guide: Power Analysis and Optimization, UG907. Truy xuất từ: <https://docs.amd.com/r/en-US/ug907-vivado-power-analysis-optimization>.
- [26] Arm, AMBA AXI and ACE Protocol Specification. Truy xuất từ: <https://developer.arm.com/documentation/ih0022/latest/>.
- [27] Accellera, SystemVerilog Language Reference Manual.
- [28] P. Kabal, “Audio File Format Specifications: WAVE or RIFF WAVE sound file format,” McGill University, 2022. Truy xuất từ: <https://www.mmsp.ece.mcgill.ca/Documents/AudioFormats/WAVE/WAVE.html>.
- [29] J. W. Cooley and J. W. Tukey, “An algorithm for the machine calculation of complex Fourier series,” Math. Comput., vol. 19, no. 90, pp. 297-301, 1965, doi: 10.1090/S0025-5718-1965-0178586-1.
- [30] S. Young et al., The HTK Book, Cambridge University Engineering Department, 2002. Truy xuất từ: <https://www.danielpovey.com/files/htkbook.pdf>.
- [31] Luan.H.P, “[BASIC SOC] Thiết Kế Phần Cứng & Hệ Thống SoC trên Kria KV260,” YouTube. Truy xuất từ: https://www.youtube.com/watch?v=F1vxzkd7_DI.
- [32] Luan.H.P, “BASIC_SOC_COURSE - PIO Transfer Petalinux Install on FPGA,” GitHub. Truy xuất từ: https://github.com/AISeQLab/BASIC_SOC_COURSE---PIO-Transfer_Petalinux-Install_on_FPGA
- [33] ZipCPU, A Generic Pipelined FFT Core Generator. Truy xuất từ: <https://github.com/ZipCPU/dblcllockfft>

A Low-Power Fixed-Point FPGA Accelerator for Vietnamese Sensitive Keyword Spotting

Ho Van Viet Hoang, Nguyen Phuc Hieu
Faculty of Computer Engineering
University of Information Technology, VNU-HCM
Ho Chi Minh City, Vietnam
{22520461, 22520441}@gm.uit.edu.vn

Truong Van Cuong
Faculty of Computer Engineering
University of Information Technology, VNU-HCM
Ho Chi Minh City, Vietnam
cuongtv@uit.edu.vn

Abstract—This paper presents a low-power fixed-point FPGA accelerator for Vietnamese sensitive keyword spotting (KWS) on edge hardware. The proposed system recognizes 20 Vietnamese keyword classes from 16-bit, 16-kHz PCM audio using a complete PCM-to-result pipeline on the Xilinx Kria KV260 platform. The hardware includes MFCC feature extraction with pre-emphasis, framing, windowing, FFT power spectrum, Mel filtering, logarithm approximation, DCT, a 98-frame feature buffer, and a compact 1274-32-20 multilayer perceptron classifier. Fixed-point arithmetic, ROM-based coefficients, sequential MAC reuse, Q28 bias scaling, and ArgMax-based classification are used to reduce hardware cost while preserving software-to-RTL consistency. On the KV260, the design uses 26,053 LUTs, 21,879 FFs, 131 DSPs, and 74 BRAMs. It closes timing at 100 MHz with a derived Fmax of 102.3 MHz and a Vivado on-chip power estimate of 0.595 W. The RTL full-dataset evaluation reaches 1996/2000 accuracy, while KV260 board-level execution reaches 294/300 on validation and 1995/2000 on the full dataset with zero timeout/error.

Index Terms—Keyword spotting, FPGA, MFCC, MLP, Vietnamese speech recognition, fixed-point arithmetic, low-power edge AI.

I. INTRODUCTION

Keyword spotting (KWS) maps a short audio segment to a predefined keyword set. Its limited output space makes it suitable for edge systems that require local processing, bounded latency, and reduced dependence on cloud services.

This work addresses Vietnamese sensitive keyword spotting for classroom-oriented monitoring. The task differs from standard English command-recognition benchmarks because Vietnamese is tonal and because the target vocabulary contains sensitive Vietnamese expressions rather than generic commands. Accuracy reported on English datasets such as Google Speech Commands is therefore not a direct measure of performance for this task.

The work targets three implementation gaps. First, Vietnamese sensitive-keyword KWS has fewer hardware-oriented references than English command KWS. Second, previous Vietnamese hardware studies often focus on MFCC front-ends, small command sets, or classifier-only systems, rather than a full PCM-to-result FPGA path. Third, an FPGA implementation requires fixed-point consistency across MFCC extraction, buffering, and neural inference; a floating-point Python model alone does not validate hardware behavior.

A compact 1274-32-20 MLP is used as the classifier. This choice is not a claim that MLPs outperform CNN, DS-CNN, or CRNN models in general. It is a hardware trade-off: the MLP maps directly to ROM-stored parameters, fixed-point MAC accumulation, ReLU, and ArgMax. Temporal information is supplied by concatenating 98 MFCC frames into a 1274-dimensional input vector.

The contributions are:

- 1) A 20-class Vietnamese sensitive-keyword KWS dataset and evaluation flow using one-second, 16-kHz PCM utterances.
- 2) An end-to-end FPGA pipeline on KV260, covering PCM input, MFCC extraction, feature buffering, MLP classification, and result register output.
- 3) A fixed-point RTL realization with Q-format selection, ROM-based coefficient storage, LOD-based log approximation, DCT computation, and Q28 MLP bias scaling.

II. RELATED WORK

A. Small-footprint KWS

MFCC is widely used in speech recognition because it compresses the short-time spectral envelope into a compact cepstral vector [1], [2]. Small-footprint DNN KWS reduces memory and computation compared with full ASR [3]. CNN-based KWS exploits local time-frequency patterns [4], while CRNN models add recurrent temporal modeling [5]. Google Speech Commands is a common English benchmark for limited-vocabulary KWS [6]. The “Hello Edge” study reports 95.4% accuracy for a DS-CNN model under microcontroller constraints [7].

These studies are useful algorithmic baselines, but their reported software or microcontroller metrics do not describe the cost of a full FPGA front-end plus classifier pipeline.

B. Hardware KWS Accelerators

KWS accelerators use quantization, specialized datapaths, depthwise separable convolution, and binarized or ternarized models to reduce hardware cost. He et al. reported a configurable INT8 TENet accelerator with 95.36% accuracy on a 12-class Google Speech Commands evaluation [8]. Bae et al. implemented a depthwise separable binarized and ternarized KWS system on FPGA [9]. ASIC work such as Laika shows

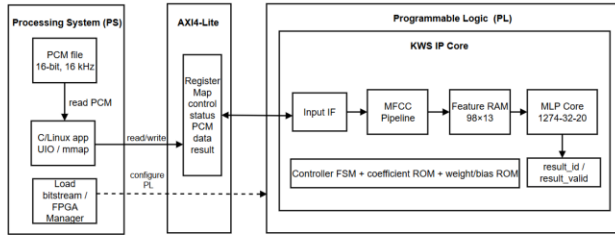


Fig. 1. Overall hardware/software architecture of the proposed FPGA KWS accelerator on KV260.

the benefit of aggressive specialization for always-on KWS [10].

Because these works use different languages, vocabularies, feature assumptions, and evaluation protocols, their accuracy values are not ranked directly against the Vietnamese task in this paper. They are used to compare scope and hardware metrics.

C. Vietnamese Speech Recognition on Hardware

Vietnamese hardware-oriented speech recognition has been studied with robot-command, isolated-word, and MFCC front-end systems. Phan Dinh Duy et al. implemented Vietnamese robot voice-command recognition on Altera DE2 with Nios II, using FFT/Mel/cepstrum features and vector quantization [11]. Lam D. Pham et al. proposed an ASIC/SOPC ANN for 20 isolated Vietnamese words and reported 95.2% accuracy [12]. Nguyen et al. and Dao et al. focused on MFCC feature-extraction hardware rather than end-to-end KWS classification [13], [14]. In contrast, this work implements PCM input, fixed-point MFCC extraction, feature buffering, and MLP classification in one FPGA pipeline.

III. PROPOSED ARCHITECTURE

A. System Overview

The input is 16-bit mono PCM sampled at 16 kHz. Each utterance is nominally one second, or 16,000 samples. With frame length $N = 512$ and hop size $H = 160$, 98 complete frames require

$$N + (98 - 1)H = 16032 \text{ samples.} \quad (1)$$

Zero-padding is applied when needed. In RTL and board-level tests, extra samples are appended only to flush the pipeline and are not included in the 98-frame feature context.

Fig. 1 separates the Processing System (PS), AXI4-Lite register interface, and Programmable Logic (PL). The PS reads PCM files and accesses control, status, PCM-data, and result registers through UIO/mmap. Bitstream loading uses the Linux FPGA manager and is outside the AXI4-Lite data path. The PL implements the KWS IP core, including the input interface, MFCC pipeline, 98-frame feature buffer, MLP core, and result registers.

The MFCC pipeline consists of pre-emphasis, framing, windowing, FFT/power, Mel filterbank, log approximation, and DCT. Each frame produces 13 MFCC coefficients. The

TABLE I
MAIN SYSTEM PARAMETERS

Parameter	Value
Input format	16-bit PCM mono
Sampling rate	16 kHz
Nominal audio length	1 s / 16,000 samples
Samples for 98 frames	16,032 samples
Frame length	512 samples
Hop size	160 samples
FFT size	512 points
FFT bins used	257
Mel filters	40
MFCC coefficients	13 per frame
Context length	98 frames
MLP input size	1274
MLP topology	1274-32-20
Target platform	Xilinx Kria KV260

feature buffer stores 98 frames, giving a 1274-dimensional MLP input. Table I lists the system parameters.

B. MFCC Front-end

The pre-emphasis filter is

$$y[n] = x[n] - \alpha x[n-1], \quad \alpha = 0.97. \quad (2)$$

In hardware, α is quantized to Q1.15:

$$\alpha_{hw} = \text{round}(0.97 \times 2^{15}) = 31785. \quad (3)$$

The framed signal uses a 512-sample Hamming window:

$$w[n] = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N-1}\right), \quad 0 \leq n < N. \quad (4)$$

The 512-point FFT keeps the 257 non-negative-frequency bins. Power is computed as

$$P[k] = \text{Re}(X[k])^2 + \text{Im}(X[k])^2, \quad 0 \leq k \leq 256. \quad (5)$$

The Mel filterbank computes

$$E[m] = \sum_{k=0}^{256} P[k] H_m[k], \quad 0 \leq m < 40, \quad (6)$$

where $H_m[k]$ is the m -th triangular Mel coefficient. The coefficient matrix is generated in Python, quantized to Q1.15, and stored in ROM.

Log compression uses a binary logarithm approximation. For positive energy E ,

$$E = 2^q(1 + f), \quad \log_2(E) = q + \log_2(1 + f). \quad (7)$$

The leading-one detector gives q , and a 256-entry LUT approximates the fractional term. DCT then maps 40 log-Mel values to 13 MFCC coefficients:

$$C[n] = \sum_{m=0}^{39} L[m] \cos\left(\frac{\pi n(m+0.5)}{40}\right), \quad 0 \leq n < 13. \quad (8)$$

TABLE II
MLP PARAMETER COUNT

Parameter	Shape	Count
W_1	1274×32	40,768
b_1	32	32
W_2	32×20	640
b_2	20	20
Total	–	41,460

TABLE III
FIXED-POINT FORMATS USED IN THE DESIGN

Signal/Parameter	Format	Role
PCM input	16-bit signed	Audio sample
Pre-emphasis coefficient	Q1.15	Constant α
Hamming/Mel/DCT coeff.	Q1.15	ROM coeff.
FFT real/imaginary	20-bit signed	FFT output
Power spectrum	48-bit unsigned	$\text{Re}^2 + \text{Im}^2$
Mel accumulator	64-bit signed	Mel MAC
Log-Mel and MFCC	Q16.16	MLP input
MLP weights	Q4.12	Weight ROM
Product $X_i W_i$	Q20.28	MAC product
MLP bias	Q28	Bias scaling
MLP accumulator	64-bit signed	MAC accumulation

C. Feature Buffer and MLP Classifier

The feature buffer stores 98 consecutive MFCC frames. Since each frame has 13 coefficients, the input size is

$$98 \times 13 = 1274. \quad (9)$$

The classifier is a one-hidden-layer MLP:

$$h = \text{ReLU}(W_1 x + b_1), \quad y = W_2 h + b_2, \quad (10)$$

$$\text{result_id} = \arg \max_i y_i, \quad 0 \leq i < 20. \quad (11)$$

Softmax is omitted because it does not change the ArgMax result and would require exponentiation and division. Table II gives the parameter count.

D. Fixed-point Scaling

Table III summarizes the numerical formats. The MLP input X_i is Q16.16 and the weight W_i is Q4.12, so the product has 28 fractional bits. Biases are therefore quantized as

$$B_{hw} = \text{round}(B_{\text{real}} \cdot 2^{28}). \quad (12)$$

This Q28 scaling aligns bias addition with the MAC accumulation scale.

IV. IMPLEMENTATION AND EVALUATION

A. Dataset and Protocol

The dataset contains 20 Vietnamese sensitive keyword classes and 2,000 original utterances, with 100 utterances per class. Each utterance is one second, 16 kHz, 16-bit PCM mono. A 300-utterance validation subset is used for cross-level validation; the full 2,000-utterance set is used for software and RTL regression. Training-time augmentation includes noise

TABLE IV
DATASET CONFIGURATION

Property	Value
Language	Vietnamese
Task	Sensitive keyword spotting
Number of classes	20
Original utterances	2,000
Utterances per class	100
Audio length	1 s
Sampling rate	16 kHz
Input format	16-bit PCM mono
Augmentation	Noise, time shift, volume scaling

TABLE V
CROSS-LEVEL VERIFICATION RESULTS

Stage	Checks	Final result
Software	FP32, fixed/QAT, HEX line-count and scale checks	300/300 val.; 299/300 val.; 2000/2000 and 1999/2000 full
Module	Golden data, waveform, valid/ready/last, index and address ranges	9/9 main modules passed
RTL batch	Self-checking top-level scoreboard with timeout/error logging	295/300 val.; 1996/2000 full; timeout/error = 0
Coverage	Expected/result classes, file coverage, code coverage	20/20 classes; 2000/2000 files; code cov. 100%
SVA	Reset, protocol, result-valid timing, legal result ID, no X/Z result	0 assertion failures
Post-syn.	Functional simulation on synthesized netlist, one sample per class	20/20 passed
KV260	C/Linux UIO/mmap, PCM write, result polling, batch log	294/300 val.; 1995/2000 full; timeout/error = 0

injection, time shift, and volume scaling. Table IV summarizes the dataset.

The dataset is domain-specific and not a public benchmark. Accuracy values are therefore reported for this task and are not used as direct cross-dataset rankings.

B. Verification Methodology

Verification follows the implementation path from Python to RTL and KV260. Numerical checks compare MFCC, log-Mel, MLP logit, and class-ID results against Python references. Control checks cover reset, valid/ready, last/index boundaries, feature-buffer addresses, result-valid timing, timeout handling, and legal result-ID ranges.

At module level, nine main blocks were checked: pre-emphasis, framing, windowing, FFT/power, Mel filterbank, Log-Mel, DCT/MFCC, Feature RAM, and MLP Core. At top level, a self-checking testbench drives PCM files through `audio_valid/audio_ready`, compares `result_id` against expected IDs, and records PASS/FAIL, timeout, and error. Functional coverage tracks expected-class hit, result-class hit, file coverage, code coverage within the regression scope, and selected boundary conditions. SVA checks reset, protocol, result-valid timing, legal result ID, and no X/Z result when valid is asserted.

TABLE VI
CROSS-LEVEL ACCURACY IN THE FINAL EVALUATION

Implementation	Validation	Full dataset
Python floating-point	300/300 = 100.00%	2000/2000 = 100.00%
Python fixed-point/QAT	299/300 = 99.67%	1999/2000 = 99.95%
RTL pre-synthesis	295/300 = 98.33%	1996/2000 = 99.80%
RTL post-synthesis	20/20 = 100.00%	N/R
KV260 board execution	294/300 = 98.00%	1995/2000 = 99.75%

TABLE VII
POST-IMPLEMENTATION RESOURCE UTILIZATION ON KV260

Resource	Used	Available	Utilization
LUT	26,053	117,120	22.24%
FF	21,879	234,240	9.34%
BRAM	74	144	51.39%
DSP	131	1,248	10.50%

C. Classification Accuracy

Table VI reports the final cross-level accuracy. The post-synthesis netlist is checked with 20 representative samples, one per class, because large-batch netlist simulation is slower than behavioral RTL simulation.

The RTL full-dataset regression and KV260 board-level batches both finish with timeout/error = 0. The reduction from Python fixed-point/QAT to RTL is therefore attributed to fixed-point and pipeline-level numerical differences, not to invalid IDs or stalled execution.

D. FPGA Resource Utilization, Timing, and Power

The design was implemented for the XCK26 device on the Kria KV260. Table VII shows post-implementation resource utilization.

The implementation meets the 100-MHz timing constraint. The timing report gives WNS = 0.225 ns and TNS = 0. With a 10-ns target period, the derived critical-path delay is 9.775 ns and the corresponding Fmax is 102.3 MHz. BRAM has the highest utilization ratio because coefficient tables, feature buffering, and MLP parameters are stored on chip. DSP utilization remains 10.50% because the MLP datapath reuses MAC hardware instead of fully unrolling matrix operations.

Vivado post-implementation power analysis estimates 0.595 W total on-chip power: 0.305 W dynamic and 0.290 W static. This is an on-chip estimate, not a board-level measurement. Board-level power would also include PS, DDR, regulators, peripherals, and software activity. The 0.595-W estimate supports a low-power edge-FPGA interpretation only within this on-chip, post-implementation scope.

V. COMPARISON AND DISCUSSION

Tables VIII and IX compare the proposed design with Vietnamese hardware-oriented studies and international KWS accelerators. The proposed design is not the smallest in every metric. Its DSP and BRAM usage are higher than some classifier-oriented or quantized CNN accelerators because this

work includes the MFCC front-end, feature buffer, and classifier in the FPGA pipeline. The comparison therefore supports the scope claim, not a direct accuracy or efficiency ranking across unrelated datasets.

The current system has four limitations. First, the dataset is domain-specific and not public. Second, the board-level evaluation is still limited to the current dataset; larger speaker-independent tests are required for deployment claims. Third, the MLP is selected for hardware regularity, while CNN/DS-CNN/CRNN models may improve robustness under heavier noise or larger speaking-rate variation. Fourth, the low-power claim is based on Vivado on-chip power estimation after implementation; controlled board-level idle/workload measurements are future work.

VI. CONCLUSION

This paper presented a low-power fixed-point PCM-to-result FPGA pipeline for Vietnamese sensitive keyword spotting. The design combines MFCC extraction, ROM-based Mel filtering, LOD-based logarithm approximation, a 98-frame feature buffer, and a 1274-32-20 MLP with Q28 bias scaling. Verification covers Python FP32/fixed models, module-level RTL, top-level batch regression, functional coverage, SVA, post-synthesis functional simulation, and KV260 board execution.

On KV260, the implementation uses 26,053 LUTs, 21,879 FFs, 131 DSPs, and 74 BRAMs, closes 100 MHz timing with WNS = 0.225 ns, and has a Vivado post-implementation on-chip power estimate of 0.595 W. Final accuracy is 300/300 and 2000/2000 for Python FP32, 299/300 and 1999/2000 for Python fixed/QAT, 295/300 and 1996/2000 for RTL pre-synthesis, 20/20 for post-synthesis representative simulation, and 294/300 validation plus 1995/2000 full-dataset execution on KV260, with timeout/error = 0 in RTL and board batches. Future work will expand the dataset, measure board-level idle/workload power, and evaluate CNN/DS-CNN or binarized models.

REFERENCES

- [1] L. R. Rabiner and B. H. Juang, *Fundamentals of Speech Recognition*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1993.
- [2] S. B. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 28, no. 4, pp. 357–366, 1980.
- [3] G. Chen, C. Parada, and G. Heigold, "Small-footprint keyword spotting using deep neural networks," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2014, pp. 4087–4091.
- [4] T. N. Sainath and C. Parada, "Convolutional neural networks for small-footprint keyword spotting," in *Proc. INTERSPEECH*, 2015, pp. 1478–1482.
- [5] S. O. Arik *et al.*, "Convolutional recurrent neural networks for small-footprint keyword spotting," in *Proc. INTERSPEECH*, 2017, pp. 1606–1610.
- [6] P. Warden, "Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition," arXiv preprint arXiv:1804.03209, 2018.
- [7] Y. Zhang, N. Suda, L. Lai, and V. Chandra, "Hello Edge: Keyword Spotting on Microcontrollers," arXiv preprint arXiv:1711.07128, 2017.
- [8] K. He, D. Chen, and T. Su, "A Configurable Accelerator for Keyword Spotting Based on Small-Footprint Temporal Efficient Neural Network," *Electronics*, vol. 11, no. 16, p. 2571, 2022.

TABLE VIII
COMPARISON WITH VIETNAMESE SPEECH-RECOGNITION HARDWARE STUDIES

Work	Task / Platform	Scope / Model	Acc.	Hardware Metrics	Comparison Note
Phan Dinh Duy et al. [11]	Vietnamese robot commands; 9 commands; Altera DE2 + Nios II	Command recognition using FFT/Mel/cepstrum + VQ	80–100%	Verilog + Nios II; no full power table	Robot-command task and VQ classifier, not sensitive-keyword KWS
Lam D. Pham et al. [12]	Vietnamese 20 isolated words; ASIC/SOPC 130 nm	MFCC + ANN classifier/SOPC	95.2%	ASIC/SOPC ANN, 250 MHz on 130 nm	ASIC/SOPC classifier focus, not KV260 FPGA pipeline
Nguyen/Dao et al. [13], [14]	Speech recognition; ASIC or FPGA	MFCC front-end IP	N/R	Dao Virtex-6: 7,353 LUT; 11,567 Reg.; 10 DSP; 8 BRAM; 139 MHz; Nguyen ASIC: 500 MHz	Front-end only; no KWS classifier result
Proposed	Vietnamese; 20 sensitive keywords; 2,000 utterances; Xilinx KV260	End-to-end KWS with fixed-point MFCC + MLP	Board val. 98.00%; board full 99.75%; RTL full 99.80%	102.3 MHz; 26,053 LUT; 21,879 FF; 131 DSP; 74 BRAM; 0.595 W est.	Full PCM–MFCC–feature-buffer–MLP FPGA pipeline

TABLE IX
COMPARISON WITH INTERNATIONAL KWS HARDWARE ACCELERATORS

Work	Dataset / Model Scope	Acc.	Fmax	Resources / Power	Note
He et al. [8]	GSC 12-class evaluation (10 keywords + unknown + silence); classifier-oriented Modified TENet INT8	95.36%	50 MHz	5,575 LUT; 2,631 FF; 64 DSP; 8* BRAM; 0.209 W	BRAM kept as reported; scope is classifier-oriented
Bae et al. [9]	GSC WUW + 10 commands; DS-BTNN KWS system	97.1 / 90.5%	170 MHz	27.3K LUT; 25.5K FF; 31 DSP; BRAM N/R; power N/R	Uses binarized/ternarized DS-CNN-style model
Proposed	Custom 20-class Vietnamese task; end-to-end fixed-point MFCC + MLP	Board val. 98.00%; board full 99.75%; RTL full 99.80%	102.3 MHz	26,053 LUT; 21,879 FF; 131 DSP; 74 BRAM; 0.595 W est.	Includes PCM input, MFCC, feature buffer, classifier, and register output

Note: N/R means not reported or not directly comparable. *He et al. BRAM is kept as reported, not normalized to KV260 blocks. Proposed power is a Vivado on-chip estimate. Accuracy values use different datasets and are listed for context only.

- [9] S. Bae, H. Kim, S. Lee, and Y. Jung, “FPGA Implementation of Keyword Spotting System Using Depthwise Separable Binarized and Ternarized Neural Networks,” *Sensors*, vol. 23, no. 12, p. 5701, 2023.
- [10] J. S. P. Giraldo and M. Verhelst, “Laika: A 5- μ W programmable LSTM accelerator for always-on keyword spotting in 65 nm CMOS,” in *Proc. ESSCIRC*, 2018, pp. 166–169.
- [11] P. D. Duy, V. D. Lung, N. Q. D. Trang, and N. C. Toan, “Speech Recognition on Robot Controller Implemented on FPGA,” *Journal of Automation and Control Engineering*, vol. 1, no. 3, pp. 274–278, 2013.
- [12] L. D. Pham, H. M. Nguyen, D. N. N. T. Nguyen, and T. Hoang, “An ASIC-Based Artificial Neural Network Applied Real-time Speech Recognition SOPC,” *Journal of Science and Technology: Issue on Information and Communications Technology*, vol. 2, no. 1, p. 38, 2016.
- [13] T. C. Nguyen, L. D. Pham, H. M. Nguyen, B. G. Bui, D. T. Ngo, and T. Hoang, “A High Performance Dynamic ASIC-Based Audio Signal Feature Extraction (MFCC),” in *Proc. International Conference on Advanced Computing and Applications (ACOMP)*, 2016, pp. 113–120.
- [14] V.-L. Dao, V.-D. Nguyen, H.-D. Nguyen, and V.-P. Hoang, “Hardware Implementation of MFCC Feature Extraction for Speech Recognition on FPGA,” in *Advances in Information and Communication Technology*, 2017.

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

TP. Hồ Chí Minh, ngày 15 tháng 07 năm 2026

DANH MỤC KIỂM TRA BÁO CÁO KLTN SAU BẢO VỆ

STT	Nội dung kiểm tra	Tình trạng kiểm tra	
		Đã thực hiện	Chưa thực hiện
1	Bìa báo cáo theo mẫu, hướng dẫn	☒	☐
2	Kiểm tra mục lục, danh mục hình ảnh, danh mục bảng biểu, ký hiệu viết tắt	☒	☐
3	Có tóm tắt bằng tiếng Việt và tiếng Anh	☒	☐
4	Hình ảnh và bảng biểu có đầy đủ tên, đánh số thứ tự, phân tích đầy đủ	☒	☐
5	Báo cáo có đánh số trang theo quy định	☒	☐
6	Các chương được bắt đầu bằng một trang mới	☒	☐
7	Kiểm tra, chỉnh sửa lỗi chính tả, kiểu chữ và font chữ	☒	☐
8	Tài liệu tham khảo được trích dẫn đầy đủ theo quy định	☒	☐
9	Có giải trình chỉnh sửa theo góp ý của hội đồng	☒	☐

Xác nhận của GVHD

(Ký và ghi rõ họ tên)



Trương Văn Cường

TP. Hồ Chí Minh, ngày 15 tháng 7 năm 2026

**BẢNG GIẢI TRÌNH CHỈNH SỬA
BÁO CÁO KHOA LUẬN TỐT NGHIỆP SAU BẢO VỆ**

Số quyết định Hội đồng: 710/QĐ-ĐHCNTT ngày 19 tháng 06 năm 2026

Ngày bảo vệ: 06/07/2026

Tên đề tài: NGHIÊN CỨU, THIẾT KẾ VÀ HIỆN THỰC IP NHẬN DIỆN TỪ KHÓA TỤC TĨU TRONG LỚP HỌC

Danh sách sinh viên

Sinh viên 1: Hồ Văn Việt Hoàng MSSV: 22520461

Sinh viên 2: Nguyễn Phúc Hiếu MSSV: 22520441

Giảng viên hướng dẫn: ThS. Trương Văn Cương

Căn cứ trên nội dung Biên bản Hội đồng khóa luận tốt nghiệp ngày 06/07/2026 và các góp ý của các thành viên Hội đồng, nhóm sinh viên thực hiện khóa luận xin phép được giải trình nội dung cập nhật báo cáo chi tiết như sau:

STT	Nội dung góp ý của Hội đồng	Nội dung giải trình cập nhật báo cáo	Trang
1	Rà soát căn lề, chất lượng hình ảnh và bố cục các chương.	Đã canh đều toàn báo cáo, phóng to Hình 3.1 và đặt mỗi chương bắt đầu ở trang mới.	Toàn báo cáo, Hình 3.1: tr. 34, Chương 2–6: tr. 7, 34, 59, 70, 98
2	Báo cáo còn thiếu Abstract bằng tiếng Anh.	Đã bổ sung Abstract tiếng Anh ngay sau phần Tóm tắt tiếng Việt và thống nhất nội dung, số liệu giữa hai bản.	Tr. 2
3	Làm rõ khái niệm pipeline trong thiết kế RTL.	Đã làm rõ chuỗi MFCC xử lý theo kiến trúc streaming; MLP Core sử dụng FSM và MAC tuần tự, không phải fully pipelined.	Tr. 11, 38, 50–51
4	Làm rõ số lượng và cách tổ chức tham số của MLP.	Đã bổ sung $W1 = 1274 \times 32$, $b1 = 32$, $W2 = 32 \times 20$, $b2 = 20$; tổng cộng 41.460 tham số.	Tr. 25–27
5	Làm rõ phương pháp lượng tử hóa sau huấn luyện theo Q-format, gồm Q1.15, Q16.16, Q4.12, Q28 và đồng bộ bias với miền MAC.	Đã trình bày lượng tử hóa sau huấn luyện theo Q-format, gồm Q1.15, Q16.16, Q4.12, Q28 và đồng bộ bias với miền MAC.	Tr. 9–10, 27–30
6	Bổ sung quy trình cập nhật hệ thống khi thêm từ khóa mới.	Đã bổ sung quy trình thu thập dữ liệu, huấn luyện lại, cập nhật class map, OUTPUT_SIZE, W2/b2, xuất lại ROM và tạo bitstream mới.	Tr. 101
7	Làm rõ tính hợp lý của bối cảnh lớp học và khả năng mở rộng ứng dụng.	Đã bổ sung lý do xử lý tại biên, giới hạn phạm vi KWS và hướng mở rộng sang cảnh báo âm thanh hỗ trợ giám sát.	Tr. 3–6, 102

Xác nhận của GVHD
(Ký và ghi rõ họ tên)

Trương Văn Cương

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

Hồ Văn Việt Hoàng
Nguyễn Phúc Hiếu